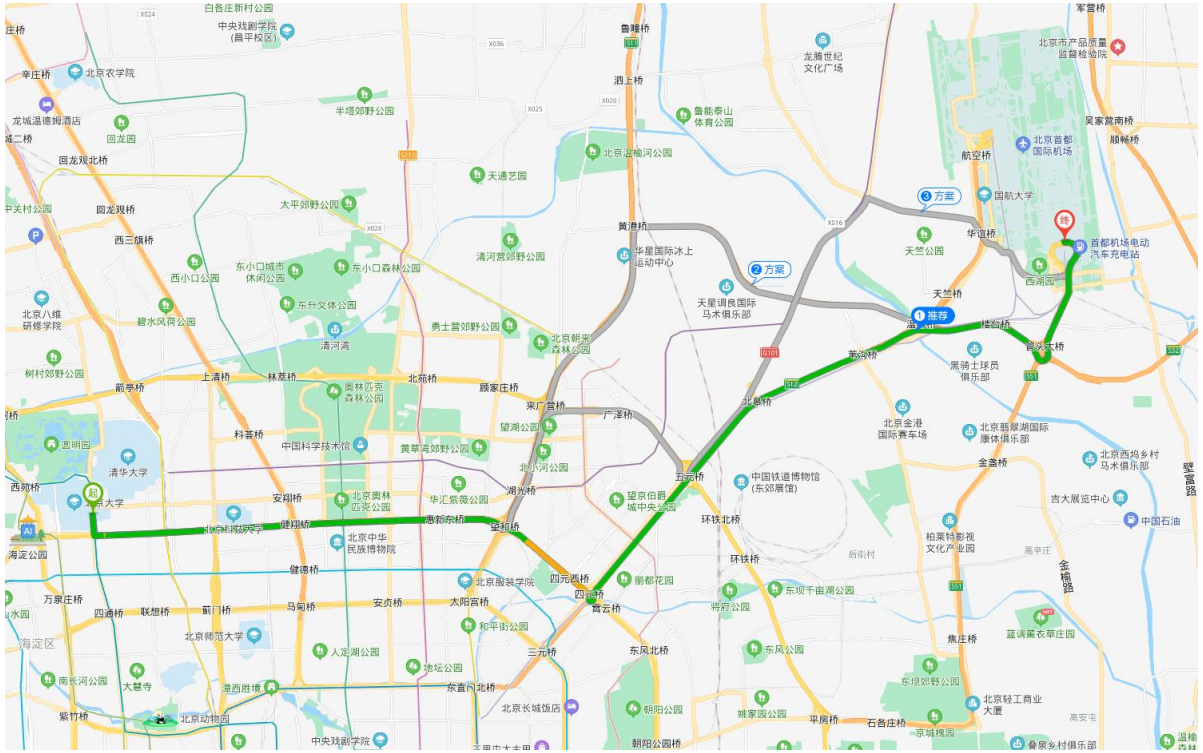


第七章 图

张史梁

地图与路径计算

- 导航路径的计算





课程内容

- 7.1 图的基本概念
- 7.2 图的抽象数据类型
- 7.3 图的存储结构
- 7.4 图的周游（深度、广度、拓扑）
- 7.5 最短路径问题
- 7.6 最小支撑树

7.1 图的基本概念

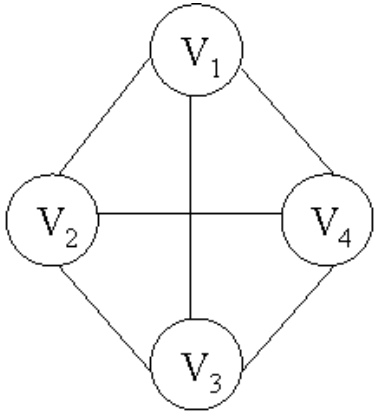
图的定义

- 用G=（V，E）代表一个图
  - V表示有限的顶点集合
  - E表示边集合，是顶点的偶对(边的始点,边的终点)
  - |V|表示顶点的总数， |E|表示边的总数

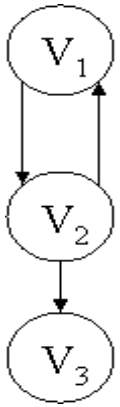
图的不同分类

- 稀疏图（sparse graph）：边数相对较少的图
- 密集图（dense graph）：边数相对较多的图
- 完全图（complete graph）：包括所有可能边的图
- 无向图（undirected graph）：顶点偶对无序的图
- 有向图（directed graph）：顶点偶对有序的图
- 标号图（labeled graph）：各顶点均带有标号的图
- 带权图（weighted graph）：边上标有权的图

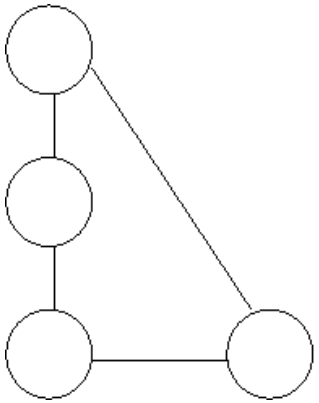
图示



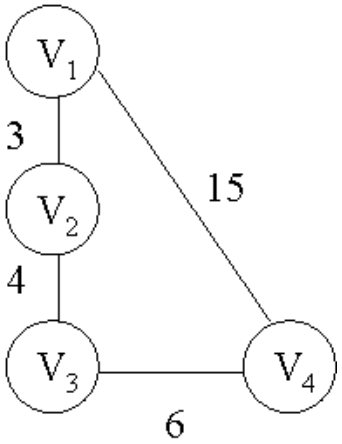
G1



G2



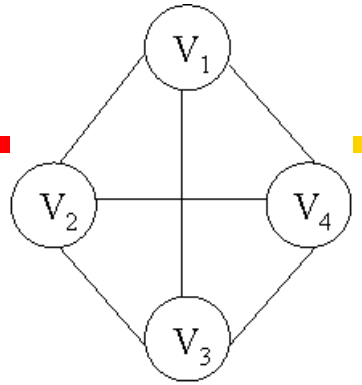
G3



G4

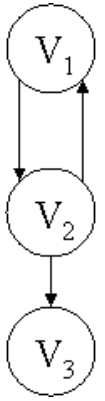
- 在上述四图中
  - G1,G3,G4是无向图
  - G2是有向图
  - G1是完全图：任何具有n个顶点的无向图，边数小于等于n(n-1)/2
  - G1,G2,G4是标号图
  - G4是带权图

图的集合表示

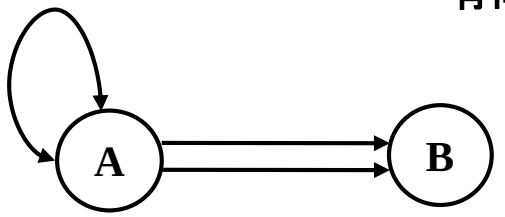


无向图G1

- 无向图G1的集合表示
  - G1=(V,E)
  - V(G1)={V1，V2，V3，V4}
  - E(G1)={(V1,V2),(V1,V3) ,(V1,V4) ,(V2,V3) ,(V2,V4) ,(V3,V4)} //圆括弧
- 有向图G2的集合表示
  - G2=(V,E)
  - V(G2)={V1，V2，V3}
  - E(G2)={<V1,V2>,<V2,V1> ,<V2,V3>} //尖括弧
- 在本章后续图的表示中，请注意
  - 不考虑顶点到自身的边
  - 不允许一条边在图中重复出现

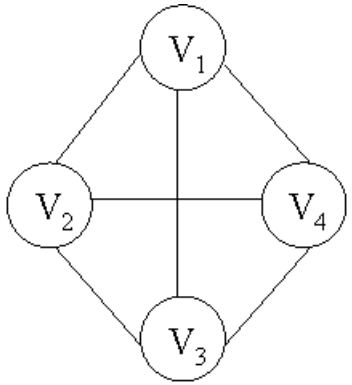


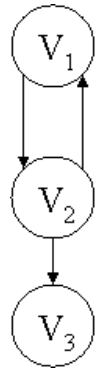
有向图G2



图的相关概念

- 相邻顶点，或邻接点（neighbors）
  - 一条边所连接的两个顶点，称为邻接点
  - 这条边称为相关联的边
- 顶点的度（degree）
  - 定义为与该顶点相关联边的数目
  - 对于有向图G = （V,E）而言
    - 入度(in degree)：以顶点V为终点的边的数目
    - 出度(out degree)：以顶点V为始点的边的数目
    - 终端结点(叶子)：出度为0的顶点
  - 若图G有n个顶点，e条边，di为顶点Vi的度数，则



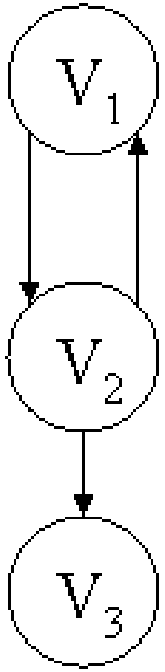


$$e = \frac{1}{2}\sum_{i=1}^{n} d_i$$

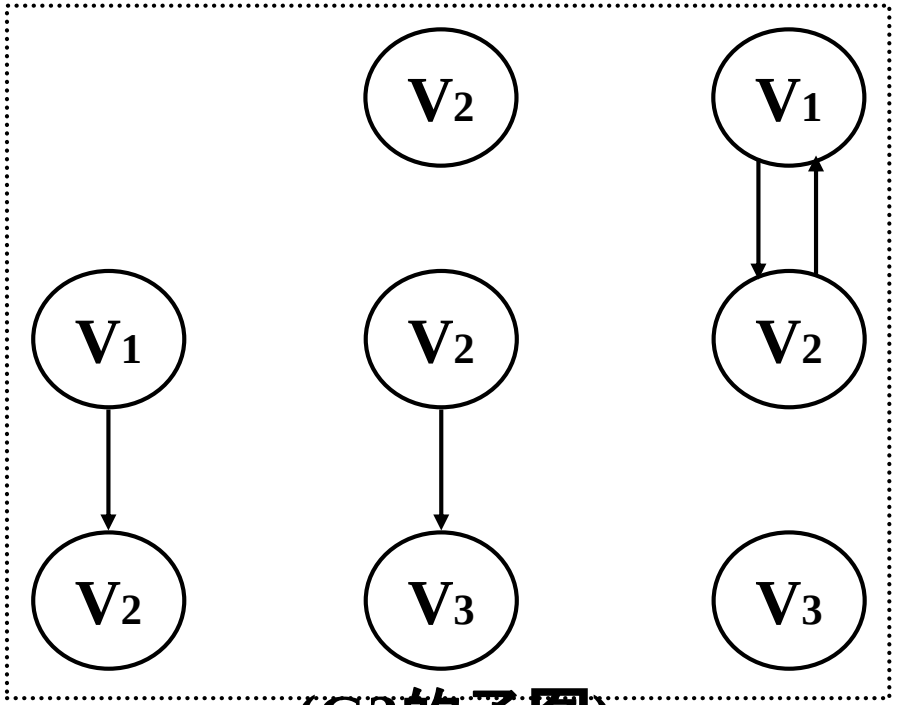
子图

子图（subgraph）

- 图G=(V，E)，G’=(V’，E’)中，若V’⊆V，E’ ⊆E，并且E’中的边所关联的顶点都在V’中，则称图G’是图G的子图



(G2)



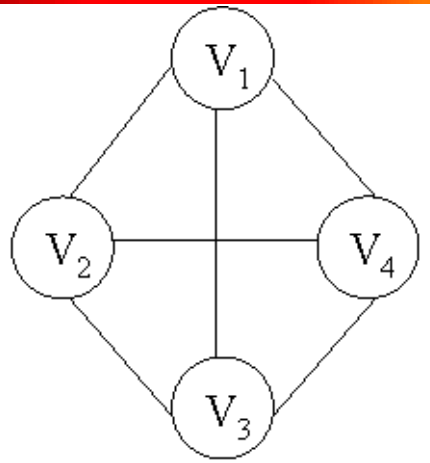
(G2的子图)

图的路径

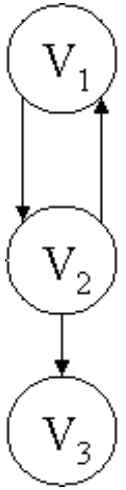
- 路径（path）
  - 在图G=(V，E)中，如果存在顶点序列V_p，V_{i1}，V_{i2}，…，V_{in}，V_q，使得（V_p，V_{i1}），(V_{i1}，V_{i2})，…，(V_{in}，V_q)（若对有向图，则使得<V_p，V_{i1}>，<V_{i1}，V_{i2}>，…，<V_{in}，V_q>）都在E中，则称从顶点Vp到顶点Vq存在一条路径。
- 简单路径（simple path）
  - 路径上除了V_p和V_q可以相同外，其它顶点都不相同
- 路径长度
  - 路径上边的条数
- 回路（也称环）：路经上某个顶点与自身连接
  - 简单回路：首尾顶点相同的简单路径
- 无环图（acyclic graph）
  - 有向无环图（directed acyclic graph，简写为DAG）

图示

- $(V_1,V_2)(V_2,V_3)(V_3,V_4)$可缩写成$V_1,V_2,V_3,V_4$
- 在图G1中
  - V_1,V_2,V_3是一条简单路径
  - V_1,V_3,V_4,V_1,V_3不是一条简单路径
  - V_1, V_2,V_3,V_4,V_2是路径，而且是回路，但不是简单回路
  - V_1, V_2,V_3,V_1是简单路径，而且是简单回路
- 在图G2中
  - V_1,V_2,V_3是一条简单的有向路径
  - V_1,V_2,V_3,V_2则不是路径



(G1)



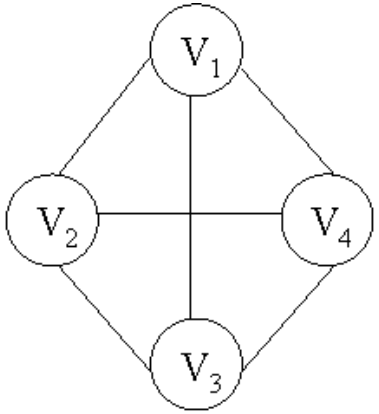
(G2)

图的连通性

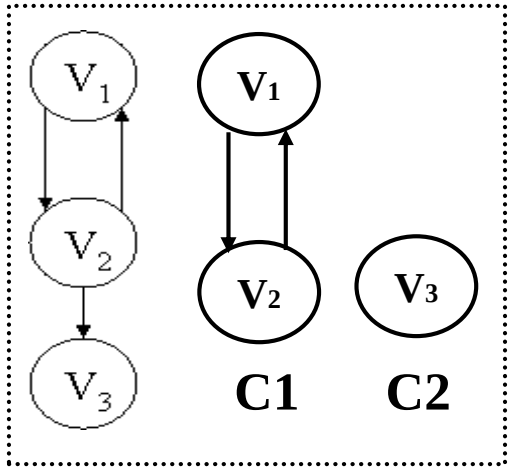
- 有根图
  - 一个有向图中，若存在一个顶点V_0，从此顶点有路径可以到达图中其它所有顶点，则称此有向图为有根图，V_0称作图的根
- 连通图
  - 对无向图G=（V，E）而言，如果从V_1到V_2有一条路径（从V_2到V_1也一定有一条路径），则称V_1和V_2是连通的
  - 若图G中任意两个顶点都是连通的，则无向图G是连通的
- 连通分量（连通分支）
  - 指无向图的最大连通子图

- 强连通性
  - 对有向图G=(V,E)而言，若对于G中任意两个顶点V_i和V_j（$V_i \neq V_j$），都有一条从V_i到V_j的（有向路径），同时还有一条从V_j到V_i的(有向)路径，则称有向图G是强连通的
- 强连通分量
  - 有向图强连通的最大子图
- 网络
  - 带权的连通图。
- 自由树（free tree）
  - 不带有简单回路的无向图，它是连通的，且具有|V|-1条边

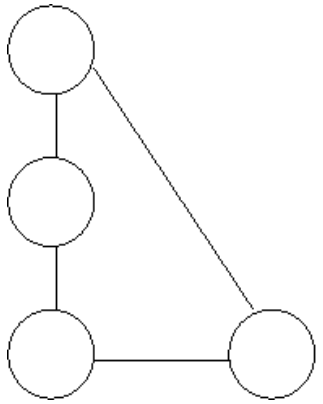
图示



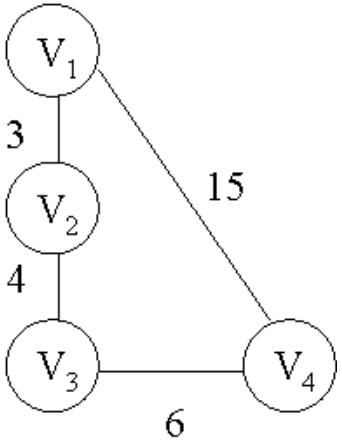
G1



G2

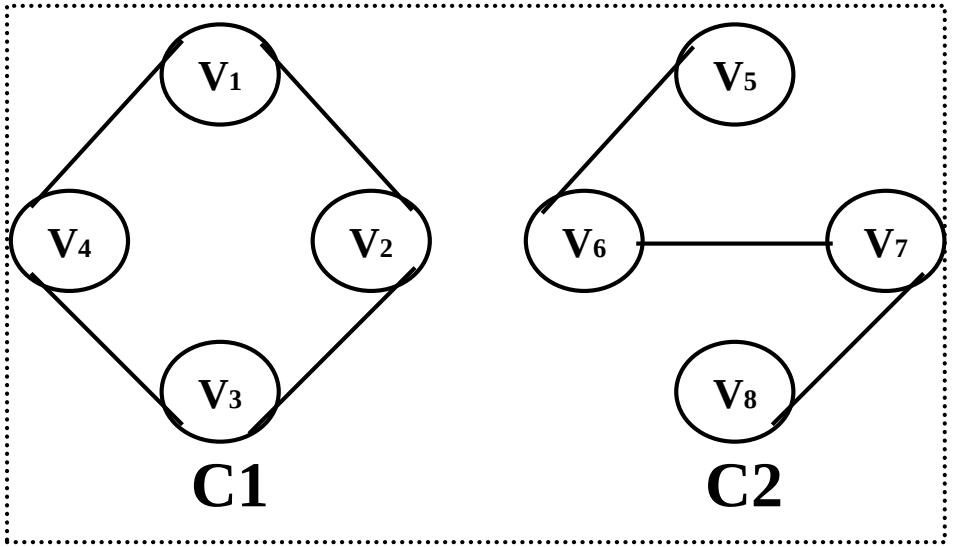


G3



G4

- 在上述四图中
  - G1,G3,G4是都是连通的
  - G5有两个连通分量C1和C2
  - G2不是强连通的
  - G2的两个强连通分量C1和C2
  - G4是网络



G5

7.2 图的抽象数据类型

```
class Graph{                                        //图的ADT
public:
    int VerticesNum();                              //返回图的顶点个数
    int EdgesNum();                                 //返回图的边数
   //返回与顶点oneVertex相关联的第一条边
    Edge FirstEdge(int oneVertex);
    //返回与边PreEdge有相同关联顶点oneVertex的下一条边
    Edge NextEdge(Edge preEdge);
    //添加一条边
    bool setEdge(int fromVertex,int toVertex,int weight);
```

```
    //删一条边
    bool delEdge(int fromVertex,int toVertex);
    //如果oneEdge是边则返回TRUE，否则返回FALSE
    bool IsEdge(Edge oneEdge);
    //返回边oneEdge的始点
    int FromVertex(Edge oneEdge);
     //返回边oneEdge的终点
    int ToVertex(Edge oneEdge);
    //返回边oneEdge的权
    int Weight(Edge oneEdge);
};
```

7.3 图的存储结构

- 7.3.1 相邻矩阵
- 7.3.2 邻接表
- 7.3.3 十字链表

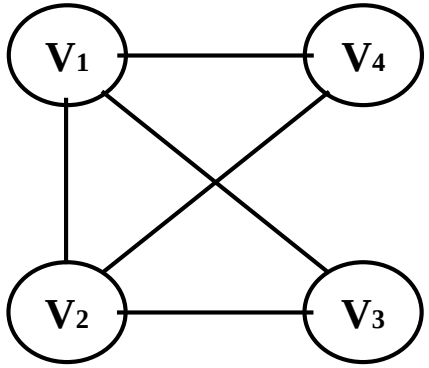
1、图的相邻矩阵表示法

- 相邻矩阵
  - 表示顶点间相邻关系的矩阵
  - 若G是一个具有n个顶点的图，则G的相邻矩阵是如下定义的n×n矩阵

$$A[i,j]=\begin{cases}1, & 若（V_i,V_j）或<V_i,V_j>是图G的边\\ 0, & 若（V_i,V_j）或<V_i,V_j>不是图G的边\end{cases}$$

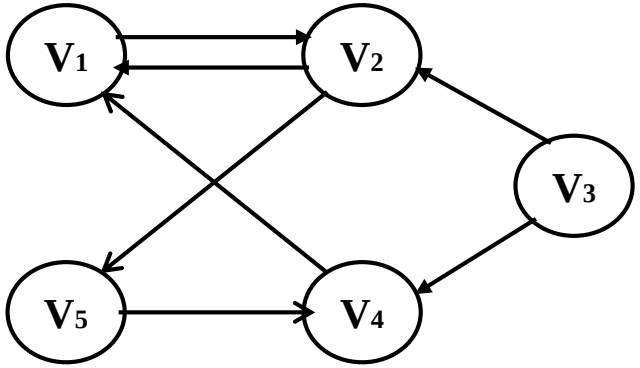
- 相邻矩阵的空间代价为$O(n^2)$，与边数无关

图示



G6

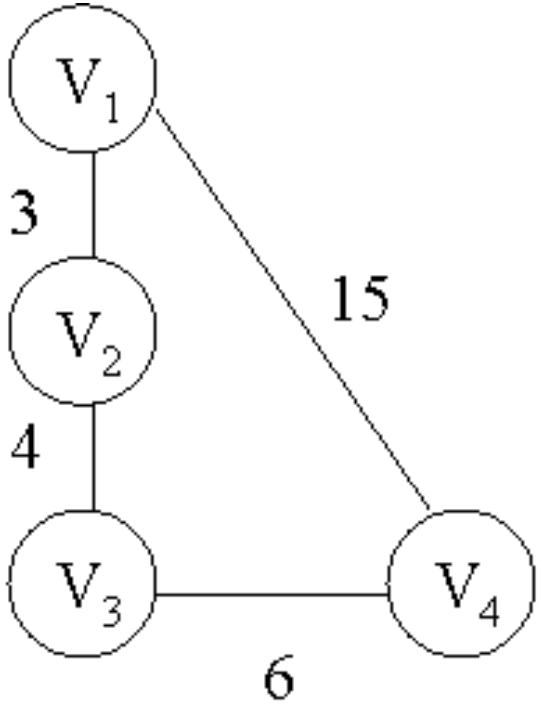
$$A_6 = \begin{bmatrix} 0 & 1 & 1 & 1 \\ 1 & 0 & 1 & 1 \\ 1 & 1 & 0 & 0 \\ 1 & 1 & 0 & 0 \end{bmatrix}$$



G7

$$A_7 = \begin{bmatrix} 0 & 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \end{bmatrix}$$

加权矩阵

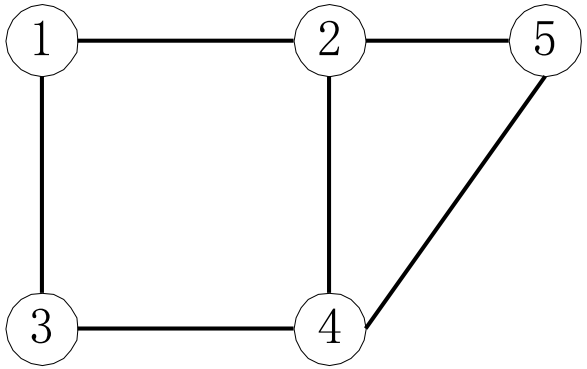


G4

$$A_4 = \begin{bmatrix} 0 & 3 & 0 & 15 \\ 3 & 0 & 4 & 0 \\ 0 & 4 & 0 & 6 \\ 15 & 0 & 6 & 0 \end{bmatrix}$$

性质1

- 从无向图的邻接矩阵可以得出如下结论:
  - 矩阵对称;
  - 第i行或第i列中1的个数为顶点i的度;
  - 矩阵中1的个数的一半为图中边的数目;
  - 容易判断顶点i和顶点j之间是否有边相连



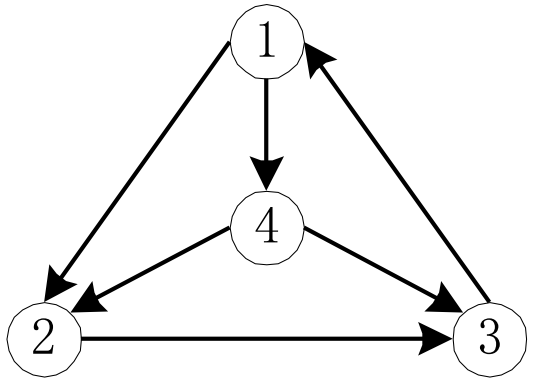
(a)

$$\begin{pmatrix} 0 & 1 & 1 & 0 & 0 \\ 1 & 0 & 0 & 1 & 1 \\ 1 & 0 & 0 & 1 & 0 \\ 0 & 1 & 1 & 0 & 1 \\ 0 & 1 & 0 & 1 & 0 \end{pmatrix}$$

(b)

性质2

- 从有向图的邻接矩阵可以得出如下结论：
  - 矩阵不一定对称；
  - 第i行中1的个数为顶点i的出度；
  - 第i列中1的个数为顶点i的入度；
  - 矩阵中1的个数为图的边数；
  - 容易判断顶点i和顶点j是否有边相连



(a)

$$\begin{pmatrix} 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 \end{pmatrix}$$

(b)

边的基类

```
Class Edge{                                  //边的基类
Public:
    int from, to, weight;
    Edge(){                                  //构造函数
        from=-1; to=-1; weight=0;
    }
    Edge(int f, int t, int w){               //构造函数
        from=f; to=t; weight=w;
    }
    bool operator > (Edge oneEdge){          //定义边比较运算符 “>”
        return  weight>oneEdge.weight;
    }
    bool operator < (Edge oneEdge){          //定义边比较运算符 “<”
        return  weight<oneEdge.weight;
    }
}
```

图的基类

```
Class Graph{                                    //图的基类
Public:
    int numVertex,  numEdge;                    //图的顶点和边的数目
    int *Mark, *indegree;                       //保持顶点是否被访问过和顶点入度的数组

    Graph(int numVert){                                     //构造函数
        numVertex= numVert;
        numEdge=0;
        indegree = new int[numVertex];                      //初始化顶点入度数组;
        Mark = new int[numVertex]; //初始化顶点是否被访问的标志数组;
        for (int i=0; i<numVertex;i++)
        {
                Mark[i]= UNVISITED;
                Indegree[i]=0;
        }
    }
```

```
~Graph(){                                   //析构函数
        delete[] mark;
        delete[] indegree;
}
virtual Edge FirstEdge (int oneVertex)=0;          //返回顶点的第一条边;
virtual Edge NextEdge (Edge preVertex)=0;          //返回当前边的下一条边;
int verticesNum() {return  numVertex};             //返回顶点数;
int EdgesNum() {return  numEdge};                  //返回边数;
bool isEdge(Edge  oneEdge){                        //判断oneEdge是否为边;
        if (oneEdge.weight>0 && oneEdge.weight<INFINITY &&
oneEdge.to>=0)     return true;
        else   return false;
}
```

```
    int FromVertex(Edge oneEdge)                    //返回边oneEdge的始点
        {return oneEdge.from;}
    int ToVertex(Edge oneEdge)                      //返回边oneEdge的终点
        {return oneEdge.to;}
    int Weight(Edge oneEdge)                        //返回边oneEdge的权
        {return oneEdge.weight;}
    virtual void setEdge(int from, int to, int weight)=0;      //设置边
    virtual void delEdge(int from, int to)=0;                  //删除边
};
```

基于邻接矩阵的类表示

```
Class Graphm: public Graph{
private:
    int ** matrix;
Public:
    Graphm(int numVert) : Graph(numVert) {          //构造函数
        int i, j;
        matrix = (int **) new int* [numVertex];     //声明一个相邻矩阵;
        for (int i=0; i<numVertex; i++)             //构造一个相邻矩阵;
            matrix[i]= new int[numVertex];
        for (int i=0; i<numVertex; i++)             //初始化相邻矩阵;
            for (int j=0; j<numVertex; j++)
                matrix[i][j]=0;
    }
```

```
~Graph(){                                   //析构函数
        for (int i=0; i<numVertex; i++)
              delete[] matrix[i];
        delete[] matrix;
}
Edge FirstEdge(int oneVertex) {          //返回顶点oneVertex的第一条边;
        Edge  myEdge;
        myEdge.from = oneVertex;  myEdge.to = -1;
        for (int i=0; i<numVertex; i++) {
              if (matrix[oneVertex][i]!=0){
                    myEdge.to=i;     myEdge.weight = matrix[oneVertex][i];
                    break;
              }
        }
        return myEdge;
}
```

```
Edge NextEdge(Edge preEdge) { //返回与边preEdge有相同顶点的下一条边
        Edge  myEdge;
        myEdge.from = preEdge.from;
        myEdge.to = -1;                          //-1可以判断是非边
        for (int i=preEdge.to+1; i<numVertex; i++) {
            if (matrix[preEdge.from][i]!=0){
                myEdge.to=i;
                myEdge.weight = matrix[oneVertex][i];
                break;
            }
        }
        return myEdge;
    }
```

```
void setEdge(int from, int to, int weight) {          //为图设定一条边；
        if (matrix[from][to]<=0){
                numEdge++;
                indegree[to]++;
        }
        matrix[from][to]=weight;
}
void delEdge(int from, int to) {                      //删掉图的一条边；
        if (matrix[from][to]>0){
                numEdge--;
                indegree[to]--;
        }
        matrix[from][to]=0;                           //0可以判断是非边
}
```

- 稀疏因子
  - 在 m × n 的矩阵中，有 t 个非零元素，则稀疏因子 δ 为:

$$\delta = \frac{t}{m \times n}$$

  - 若 δ 小于 0.05 ， 可认为是稀疏矩阵

2、图的邻接表表示法

- 邻接矩阵表示法
  - 矩阵的规模只与顶点的个数n有关（n^2）
  - 与边无关
  - 由于大量的边不存在，造成空间浪费
- 邻接表表示法
  - 既与顶点有关，又与边有关
  - 顶点表：对应n个顶点，包括顶点数据和指向边表的指针
  - 边链表：对应m条边，包括顶点序号和指向边表下一表目指针

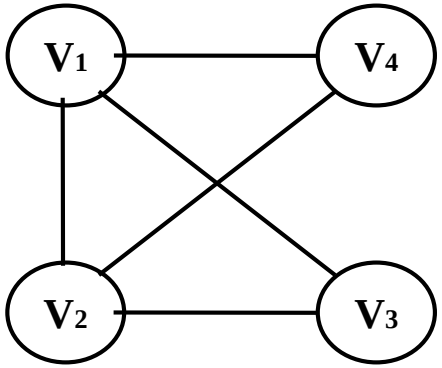
顶点结点

data	firstarc

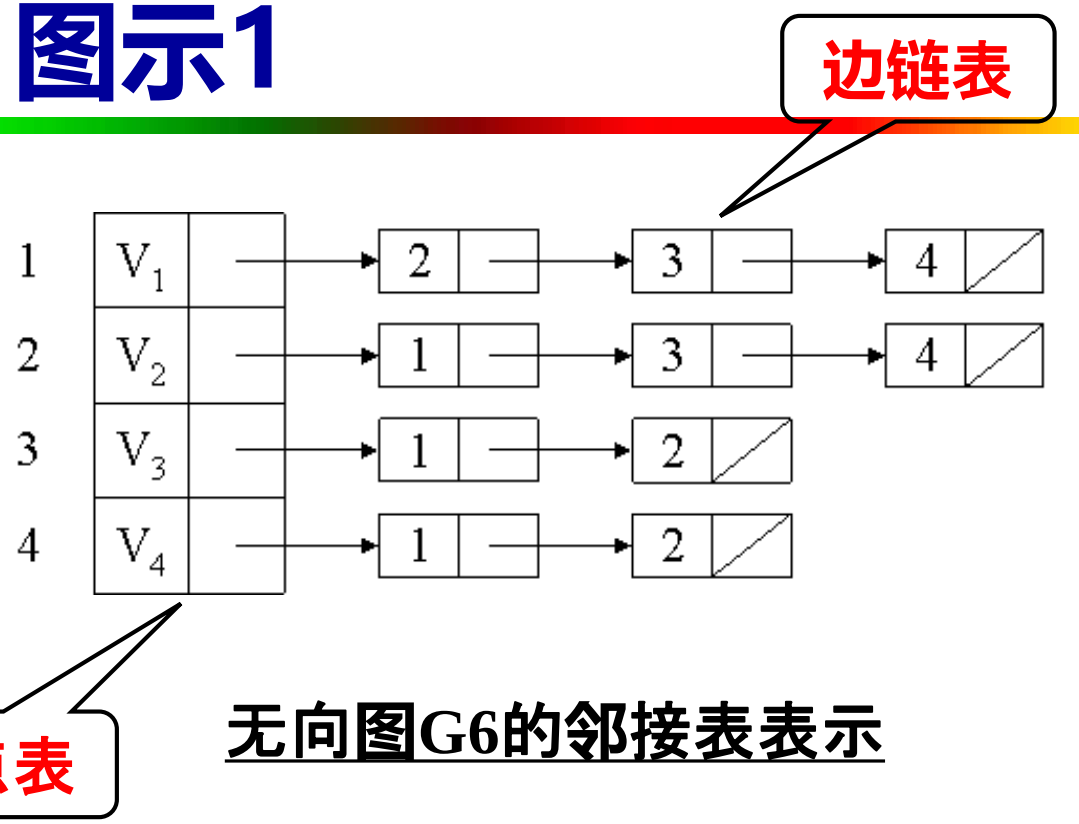
边（或弧）结点

adjvex	nextarc	Info

图示1



无向图G6

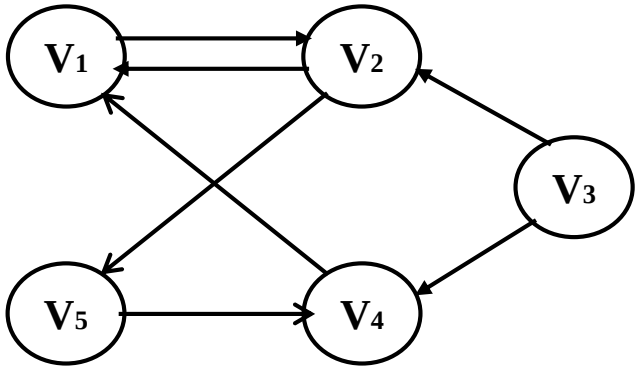


无向图G6的邻接表表示

需要|V|＋2 |E|个存储单元

边链表中表目顺序往往按照顶点编号从小到大排列。

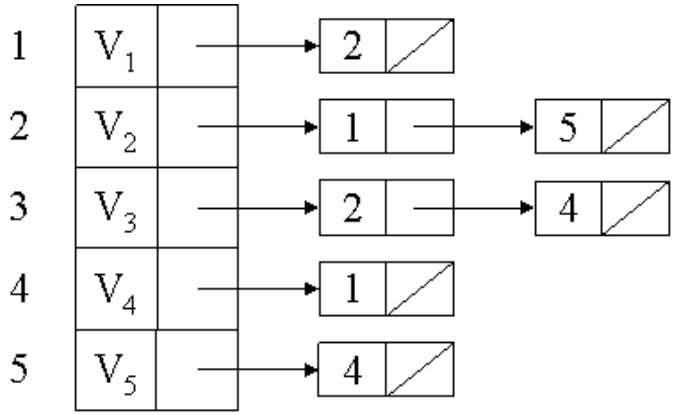
图示2



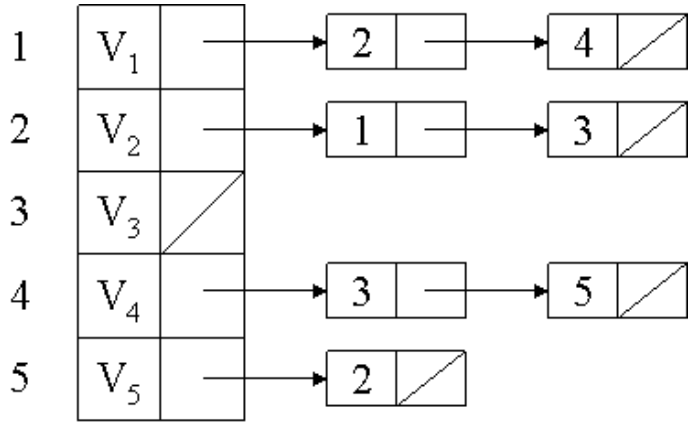
有向图G7

1、保存出边表和入边表之一即可

2、需要|V| + |E|个存储单元



(a)有向图G7的出边表



(b)有向图G7的入边表

图的邻接表空间代价

- n 个顶点 e 条边的无向图
  - 需用 $(n + 2e)$ 个存储单元
- n 个顶点 e 条边的有向图
  - 需用 $(n + e)$ 个存储单元
- 当边数 e 很小时，可以节省大量的存储空间
- 边表中表目顺序往往按照顶点编号从小到大排列

```
Edge firstEdge(int oneVertex)          //返回顶点oneVertex的第一条边
{
    Edge myEdge;
    myEdge.from = oneVertex;
    myEdge.to = -1;
    Link<listUnit> *temp = graList[oneVertex].head; //取对应链表的头
    if (temp->next != NULL)
    {
        myEdge.to = temp->next->element.vertex;
        myEdge.weight = temp->next->element.weight;
    }
    return  MyEdge;
}
```

```
Edge nextEdge(Edge preEdge) { //返回与preEdge有相同顶点的下一条边
        Edge myEdge;
        myEdge.from = preEdge.from;
        myEdge.to = -1;
        Link<listUnit> *temp = graList[preEdge.from].head;
        while (temp->next != NULL && temp->next->element.vertex <=preEdge.to)
                temp=temp->next;
        if (temp->next!=NULL)
        {
                myEdge.to = temp->next->element.vertex;
                myEdge.weight = temp->next->element.weight;
                return  MyEdge;
        }
}
```

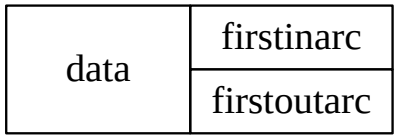
```
Void setEdge(int from, int to, int weight){  //为图设定一条边；
    Link <listUnit> * temp = graList[from].head;
    while (temp->next!=NULL && temp->next->element.vertex<to)
        temp=temp->next;                          //确定要插入边表的位置
    if (temp->next==NULL){                        //为空，在尾部插上这条边
        temp->next=new Link<listUnit>;
        temp->next->element.vertex = to;
        temp->next->element.weight = weight;
        numEdge++;
        indegree[to]++;
        return;
    }
    if (temp->next->element.vertex == to){ //边已经在图中存在，只改权值；
        temp->next->element.weight = weight;
        return;
    }
```

```
    if (temp->next->element.vertex > to){  //不存在，在边表中插入该条边；
        Link <listUnit> * other = temp->next;
        temp->next=new Link<listUnit>;
        temp->next->element.vertex = to;
        temp->next->element.weight = weight;
        temp->next->next = other;
        numEdge++;
        indegree[to]++;
        return;
    }
}//end setEdge;
```

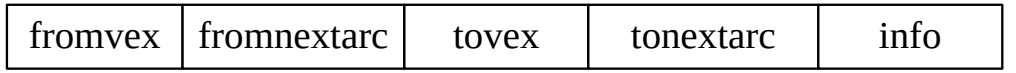
```
Void delEdge(int from, int to){  //删除图的一条边；
    Link <listUnit> * temp = graList[from].head;
    while (temp->next!=NULL && temp->next->element.vertex<to)
        temp=temp->next;                          //确定要插入边表的位置
    if (temp->next==NULL)                         //边在图中不存在；
        return;
    if (temp->next->element.vertex > to)          //边在图中不存在；
        return;
    if (temp->next->element.vertex == to){        //边在图中存在；
        Link <listUnit> * other = temp->next->next;
        delete temp->next;
        temp->next = other;
        numEdge--;
        indegree[to]--;
        return;
    }
  }
}
```

3、十字链表

- 另一种链式存储结构，可看成是邻接表和逆邻接表的结合
- 顶点表：对应图的顶点，由3个域组成
  - data域；
  - firstinarc指针指向第一条以该顶点为终点的边；
  - firstoutarc指针指向第一条以该顶点为起点的边。
- 边链表：对应有向图的每一条边，共5个域：
  - 起点(fromvex)和终点(tovex)的顶点序号；边权值的info域；
  - fromnextarc指针指向下一个以fromvex为起点的边；
  - tonextarc指针指向下一条以tovex为终点的边；

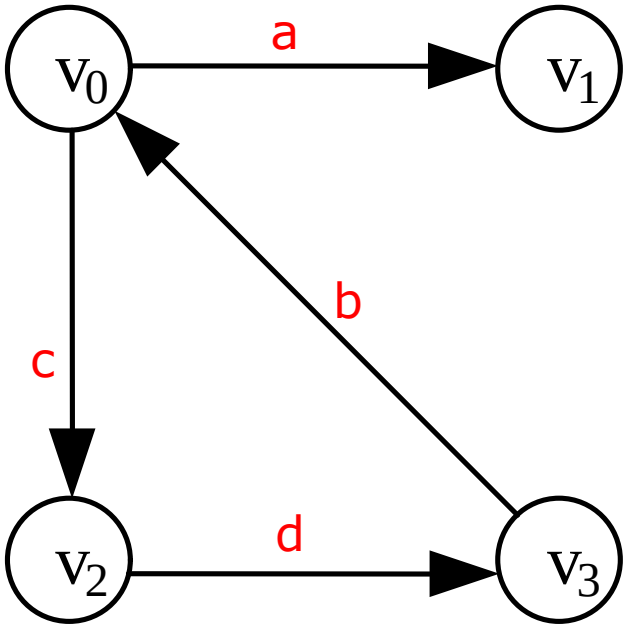


顶点结点

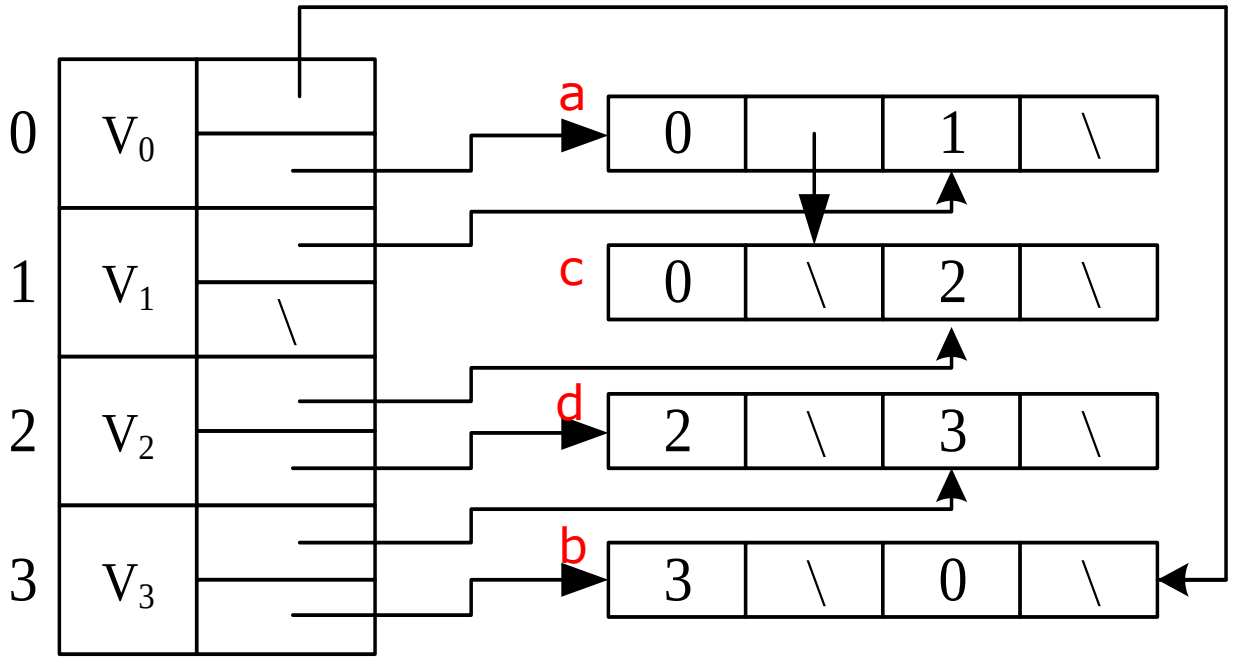


弧(有向边)结点

图示



有向图

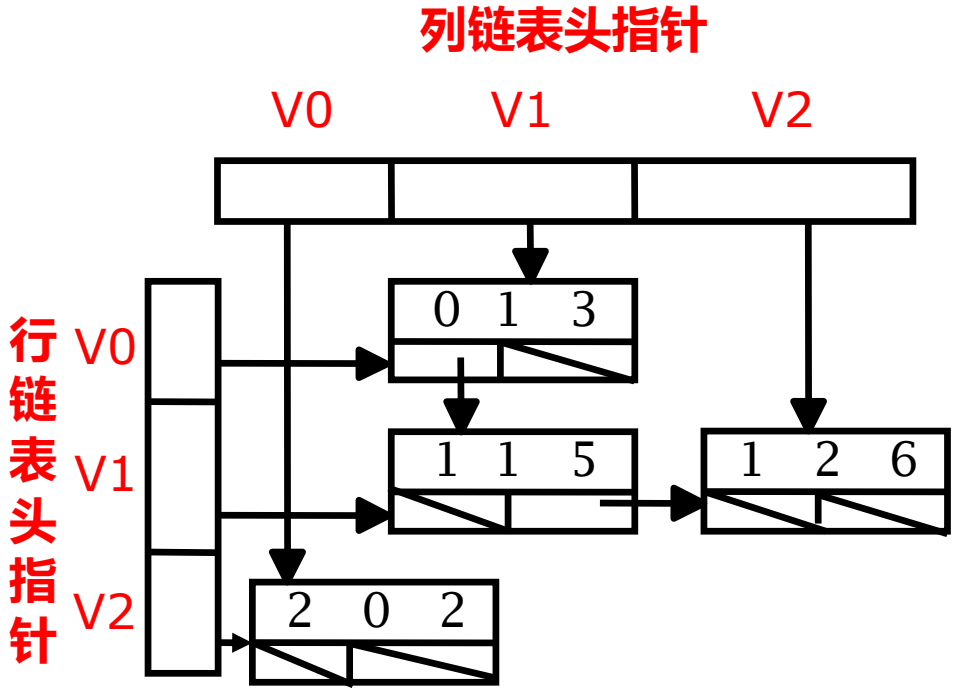


顶点表　　边表

为何称为“十字链表”？

- 十字链表有两组链表组成
  - 行和列的指针序列
  - 每个结点都包含两个指针：同一行的后继，同一列的后继

	V0	V1	V2
V0	0	3	0
V1	0	5	6
V2	2	0	0



7.4 图的周游

- 图周游（Graph Traversal）
  - 给定图G和任一顶点V_0，从V_0出发系统地访问G中所有的顶点，每个顶点访问一次，称为图周游
- 图周游的典型算法
  - 从一个顶点出发，访问其余顶点，需考虑下列情况：
    - 非连通图：从一顶点出发，可能不能到达所有其它的顶点
    - 存在回路的图：也有可能会陷入死循环
  - 解决办法
    - 顶点保留一标志位，初始时标志位置未访问（UNVISITED）
    - 在周游过程中，当顶点被访问时，标志位置已访问（VISITED）

周游算法

```
void graph_traverse(Graph& G)
{  //图的周游算法的实现
    for(int i=0;i<G.VerticesNum();i++)  //初始化标志位
        G.Mark[i]=UNVISITED;
    for(int i=0;i<G.VerticesNum();i++)
        if(G.Mark[i]== UNVISITED)
            do_traverse(G, i); //do_traverse函数
}
```

- 周游是求解图的连通性、拓扑排序和关键路径等问题的基础。

- 两类主要方式
  - 深度优先
  - 广度优先

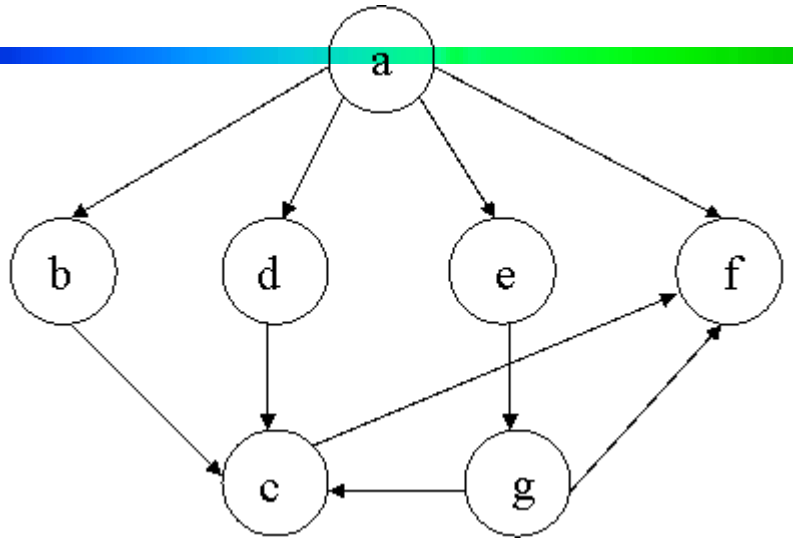
1、深度优先搜索（DFS）

基本思想

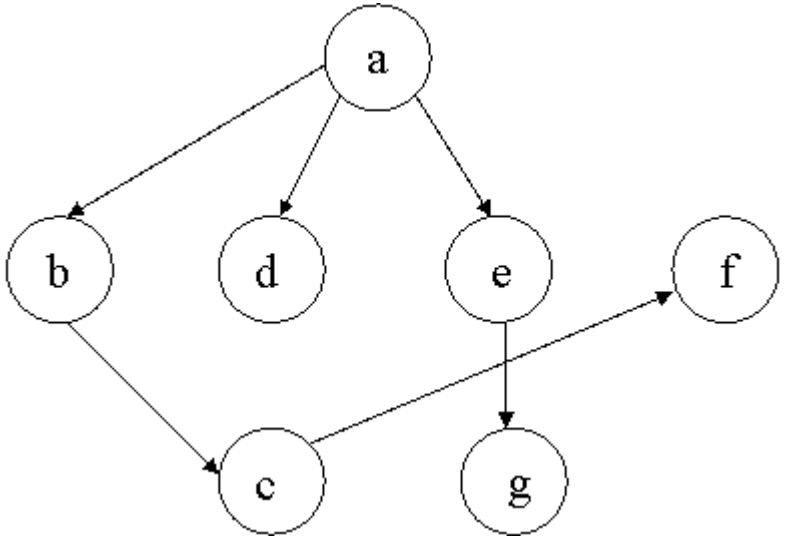
- 选取一个未访问的点 v0 作为源点
  - 访问顶点 v0
  - 递归地深搜遍历 v0 邻接到的其他顶点
  - 重复上述过程直至从 v0 有路径可达的顶点都已被访问过
- 再选取其他未访问顶点作为源点做深搜，直到图的所有顶点都被访问过束

深度优先搜索树（Depth-First Search Tree）

图示

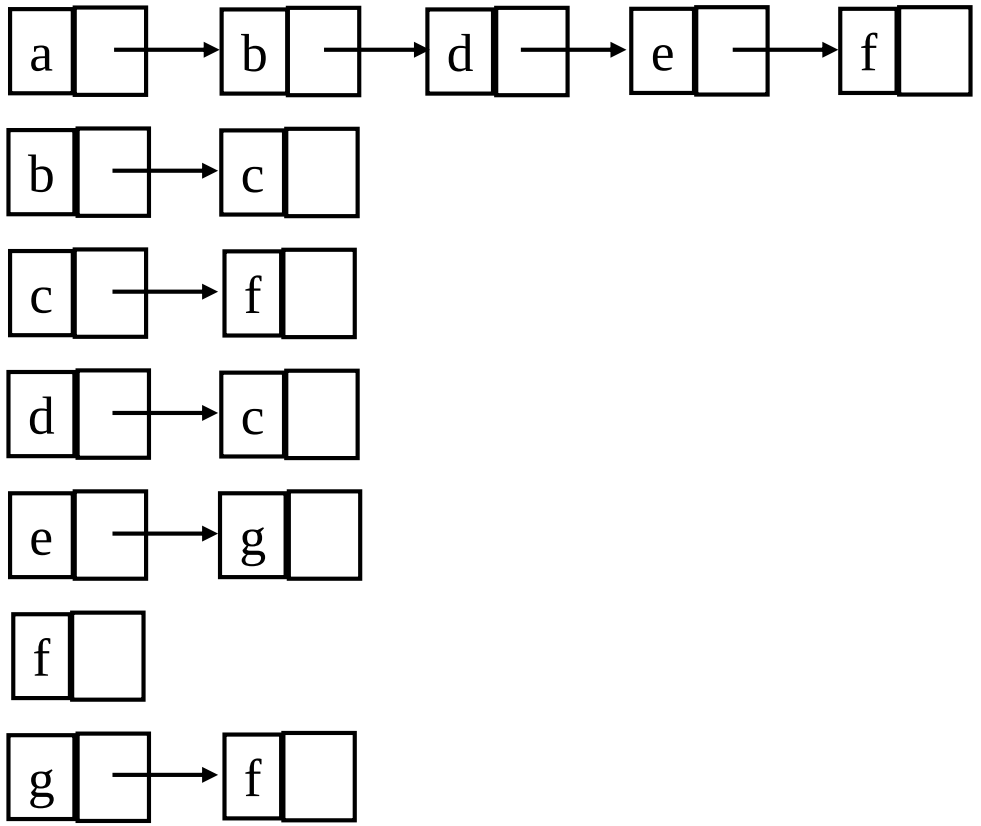


(a)有向图



(c)深度优先搜索树

(b)邻接表



深度优先搜索的顺序：a, b, c, f, d, e, g

深度优先搜索算法

```
void DFS(Graph& G, int V)
{
    Visit(G, V);                          //访问V
    G.Mark[V]= VISITED;                   //访问顶点V，并标记其标志位
    for(Edge e=G. FirstEdge(V); G.IsEdge(e); e=G. NextEdge(e))
        //递归地按照深度优先的方式访问V邻接的未被访问的顶点
    if(G.Mark[G. ToVertices(e)]== UNVISITED)
            DFS(G, G. ToVertices(e));
}
```

复杂性分析

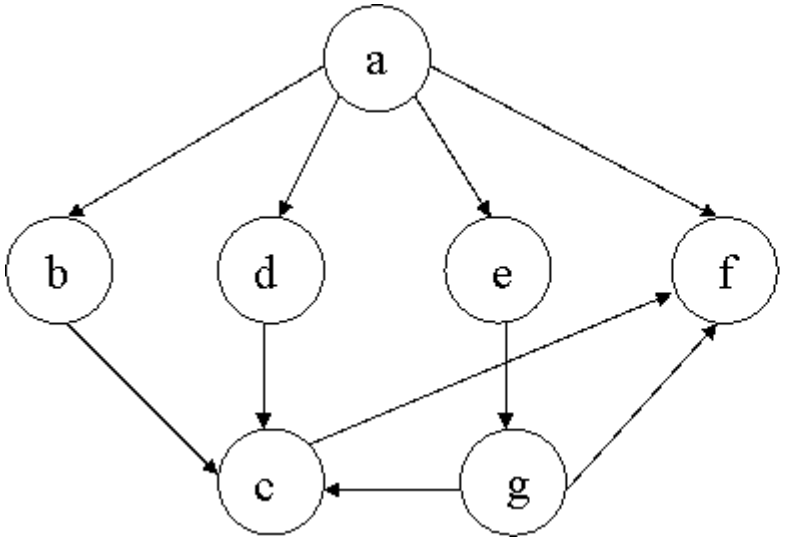
时间复杂度

- DFS对每一条边处理一次（无向图的每条边从两个方向处理），每个顶点访问一次。
- 采用邻接表表示时，有向图总代价为$\Theta(|V|+|E|)$，无向图为$\Theta(|V|+2|E|)$
- 采用相邻矩阵表示时，处理所有的边需要$\Theta(|V|^2)$的时间，所以总代价为$\Theta(|V|+|V|^2)= \Theta(|V|^2)$。

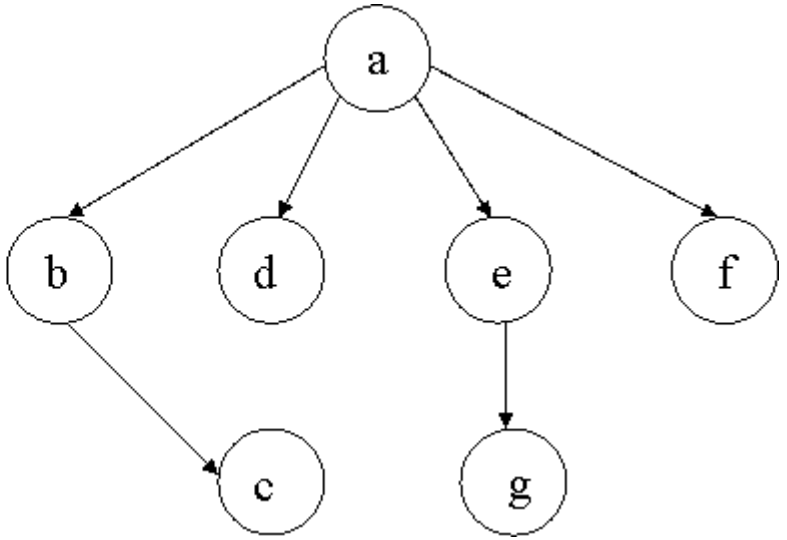
2、广度优先搜索（BFS）

- 基本思想
  - 访问顶点V_0
  - 然后访问V_0邻接到的所有未被访问过的邻居顶点V_{01}，V_{02}，…V_{0i}
  - 再依次访问V_{01}，V_{02}，…V_{0i}邻接到的所有未被访问的邻居顶点
  - 如此进行下去，直到访问遍所有的顶点。
- 广度优先搜索树（breadth-first search tree）

图示

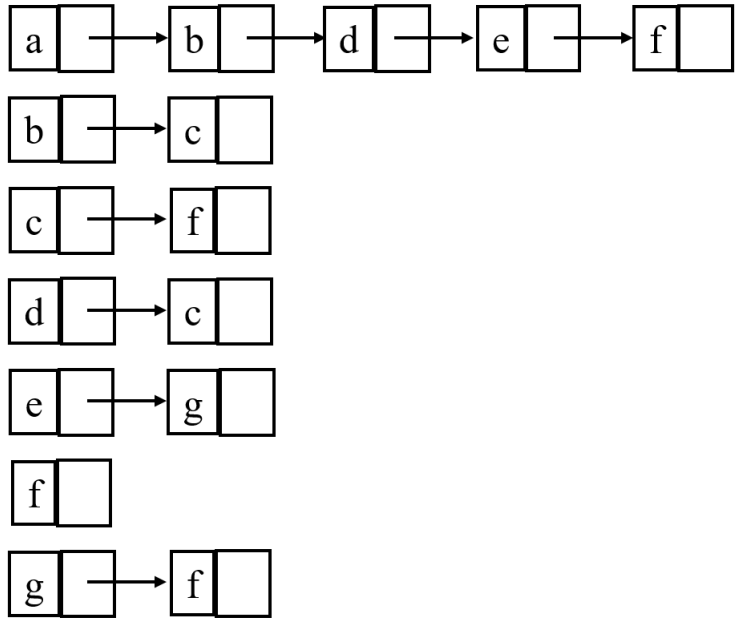


(a)有向图



(b)广度优先搜索树

(b)邻接表



广度优先搜索的顺序:a, b, d, e, f, c, g

广度优先搜索算法

```
void BFS(Graph& G, int V){
    using std::queue;                    //初始化广度优先周游要用到的队列
    queue<int> Q;
    G.Mark[V]= VISITED;                  //访问顶点V，并标记其标志位， V入队
    Visit(G, V); Q.push(V);
    while(!Q.empty()) {                  //如果队列仍然有元素
        int V=Q.front(); Q.pop(); //取顶部元素，并出队
        //将与该点相邻的每一个未访问点都入队
        for(Edge e=G.FirstEdge(V); G.IsEdge(e);e=G.NextEdge(e)){
            if(G.Mark[G.ToVertex(e)]== UNVISITED) {
                G.Mark[G.ToVertex(e)]=VISITED;
                Visit(G, G.ToVertex(e));
                Q.push(G.ToVertex(e)); //入队
            }
        }
    }
}
```

复杂度分析

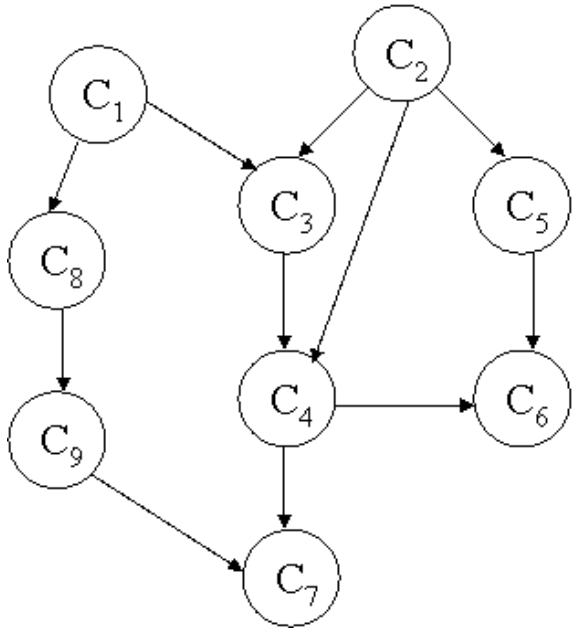
- 广度优先搜索实质上与深度优先相同，只是访问顺序不同而已。
- 二者时间复杂度也相同

3、拓扑排序

问题定义

- **先决条件**：是指以某种**线性顺序**来组织多项任务，以便能够在满足**先决条件**的情况下逐个完成各项任务
- **有向无环图**能够模拟先决条件

先修课程	课程代号	课程名称
	高等数学	C1
	程序设计	C2
C1, C2	离散数学	C3
C2, C3	数据结构	C4
C2	算法语言	C5
C4, C5	编译技术	C6
C4, C9	操作系统	C7
C1	普通物理	C8
C8	计算机原理	C9



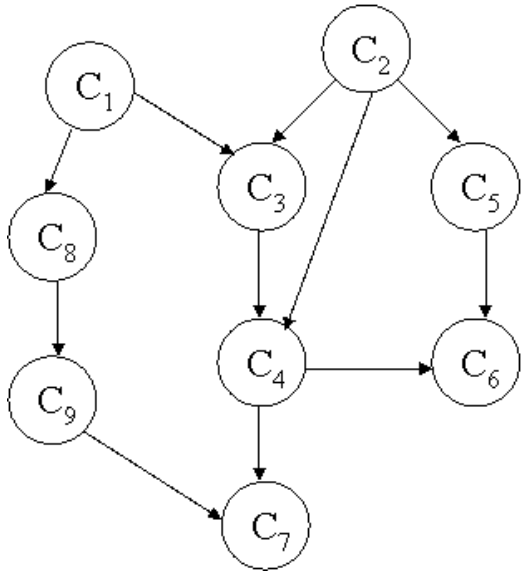
学生课程安排图

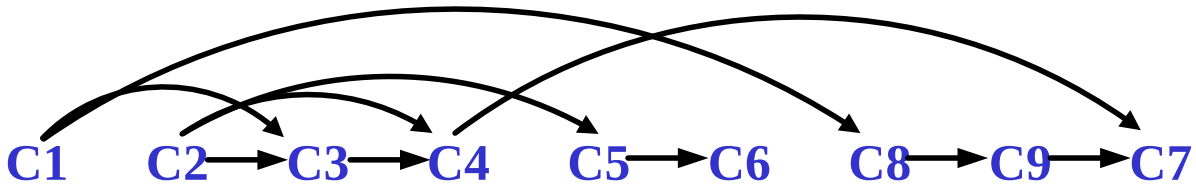
拓扑排序（topological sort）

- 将一个有向无环图中所有顶点在不违反先决条件关系的前提下排成线性序列的过程称为拓扑排序
- 对一个有向无环图G进行拓扑排序，是将G中所有顶点排成一个线性序列，使得图中任意一对顶点u和v，若<u，v> ∈E(G)，则u在线性序列中出现在v之前
- 拓扑排序形成的序列称作拓扑序列

性质1

- 若将图中顶点按拓扑次序排成一行，则图中所有的有向边均是从左指向右的

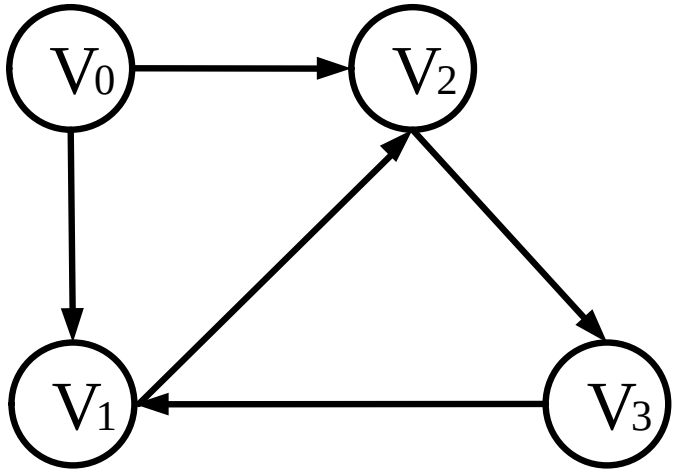


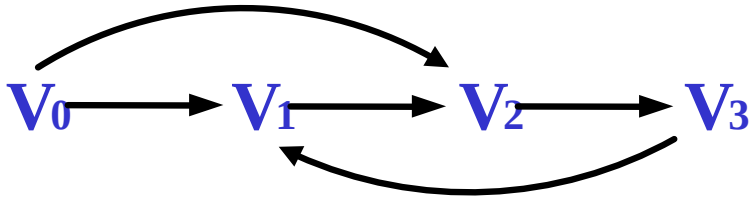


- 拓扑序列不唯一
  - 例如，上图至少可以有如下两个拓扑序列：C1C2C3C4C5C6C8C9C7和C1C8C9C2C5C3C4C7C6

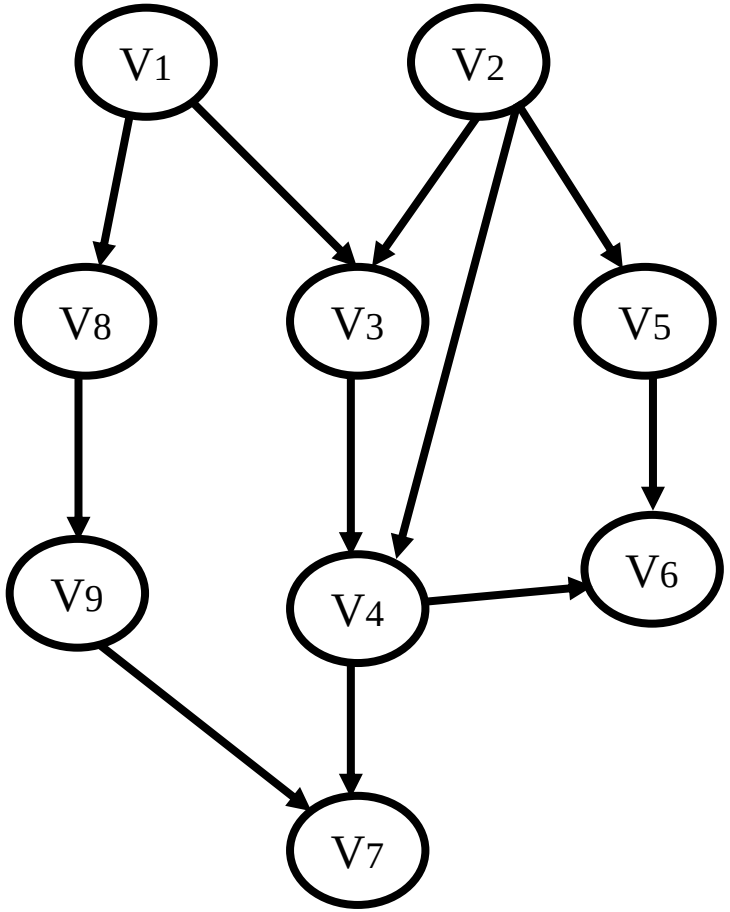
性质2

- 环存在时不存在拓扑序列





拓扑排序的基本思想



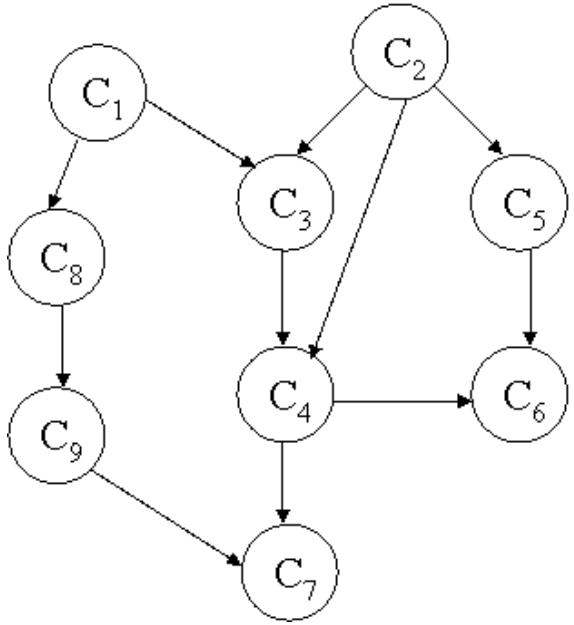
- **限定是有向无环图**
- **拓扑排序方法**
  - 从图中选择一入度为0的顶点并输出
  - 从图中删掉此顶点及其所有的出边（出边关联顶点的入度减1）
  - 回到第（1）步继续执行。
- **环路存在时**
  - 排序结束，仍有顶点没有被输出
  - 但在剩下的图中找不到入度为0的顶点

拓扑排序算法

- 基于邻接矩阵的实现

- 基于邻接表的实现
  - 广度优先排序(BFS-TopSort)
  - 深度优先排序(DFS-TopSort)

基于邻接矩阵表示的拓扑排序



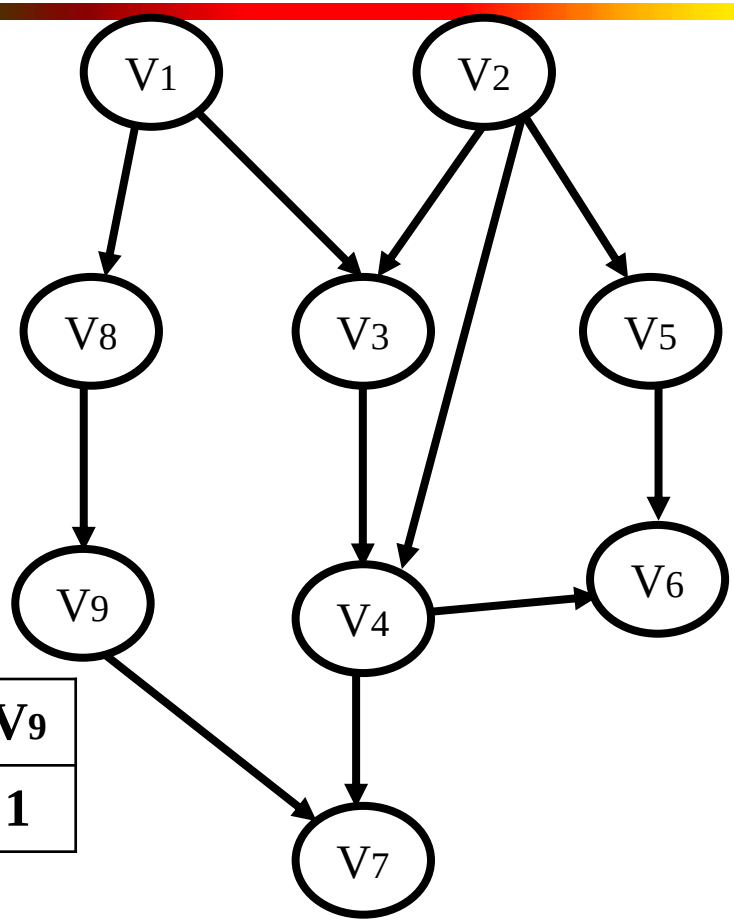
- G中入度为0的是A中全0的列
- 删除某个顶点的出边就是把邻接矩阵中对应的行清0

新序号	1	2	3	4	5	6	9	7	8
行/列号	1	2	3	4	5	6	7	8	9
1	0	0	1	0	0	0	0	1	0
2	0	0	1	1	1	0	0	0	0
3	0	0	0	1	0	0	0	0	0
4	0	0	0	0	0	1	1	0	0
5	0	0	0	0	0	1	0	0	0
6	0	0	0	0	0	0	0	0	0
7	0	0	0	0	0	0	0	0	0
8	0	0	0	0	0	0	0	0	1
9	0	0	0	0	0	0	1	0	0

图G的邻接矩阵A

基于邻接表的拓扑排序：BFS方法

- **BFS-TopSort**
  - **为每个顶点设置一个表示该结点入度字段(indegree)，入度表**
    - **不用检查n*n的矩阵**
    - **直接检查数组就可确定入度为0的顶点**



入度表

顶点	V_1	V_2	V_3	V_4	V_5	V_6	V_7	V_8	V_9
入度	0	0	2	2	1	2	2	1	1

队列

BFS-TopSort算法

```
void TopsortbyQueue(Graph& G) {                  //队列方式实现的拓扑排序
    for(int i=0;i<G.VerticesNum();i++)
        G.Mark[i]=UNVISITED;                     //初始化标记数组
    using std::queue;
    queue<int> Q;                                //初始化队列
    for(i=0; i<G.VerticesNum(); i++) {           //图中入度为0的顶点入队
        if(G.Indegree[i]==0)
            Q.enqueue(i);
    }
    while(!Q.empty()){                           //如果队列中还有图的顶点
        V=Q.dequeue();                           //一个顶点出队
        Visit(G, V);                             //访问该顶点
        G.Mark[V]=VISITED;
```

```
            //边e的终点的入度值减1
            for(Edge e= G.FirstEdge(V); G.IsEdge(e);e=G.NextEdge(e)) {
                    G.Indegree[G.ToVertex(e)]--;
                    if(G.Indegree[G.ToVertex(e)]==0)
                            Q.enqueue(G.ToVertex(e)); //入度为0的顶点入队
            }//end for
        }//end while
        for(i=0; i<G.VerticesNum(); i++) {
            if(G.Mark[i]==UNVISITED){
                Print("图有环");                    //图有环
                break;
            }
        }
    }   //注:在有环的情况下会提前退出，从而可能没处理完所有的边和顶点
```

广度优先排序可以判定有环存在~~

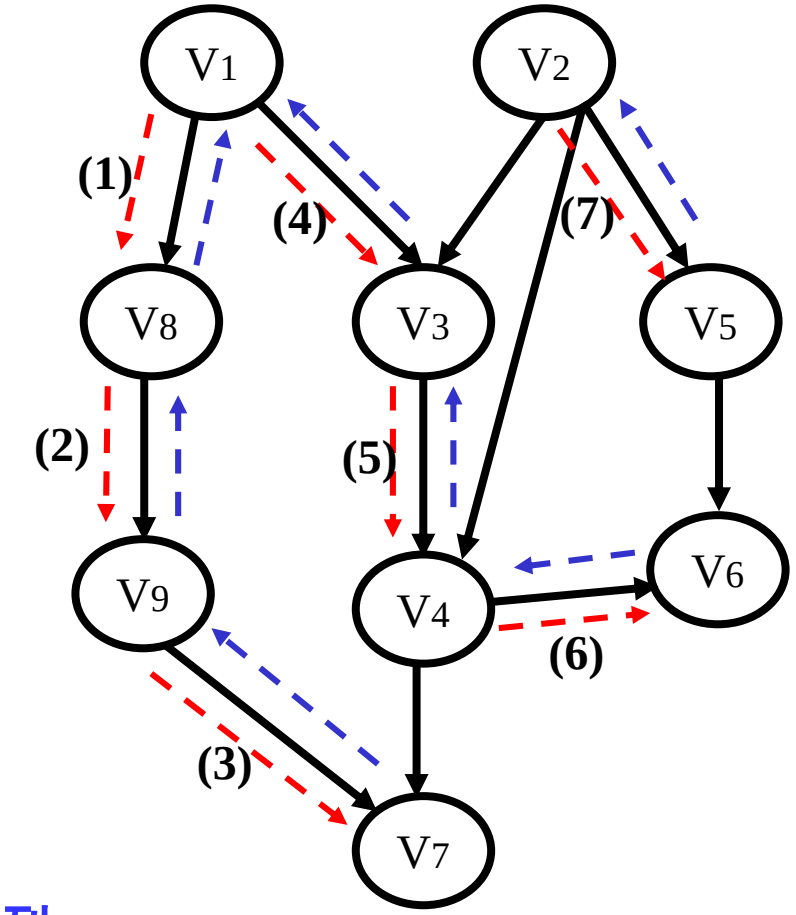
复杂性分析

拓扑排序的时间复杂度

- 采用相邻矩阵时，每次算法需要找所有入度为0的顶点，需要 $\Theta(|V|^2)$的时间，那么对|V|个顶点而言，总代价为$\Theta(|V|^3)$
- 当采用邻接表时，因为在顶点表的每个顶点中可以有一个字段来存储入度，所以只需$\Theta(|V|)$的时间，加上处理边、顶点的时间，总代价为$\Theta(2|V|+|E|)$

基于邻接表的拓扑排序：DFS

- DFS-TopSort
  - 栈的使用
  - 逆序序列



逆序队列

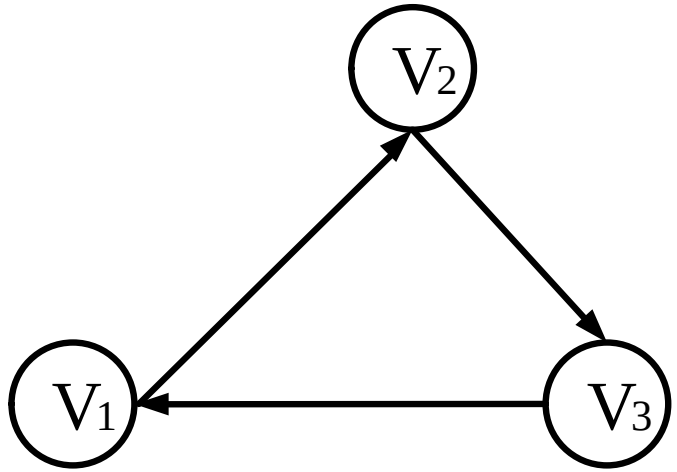
DFS-TopSort算法

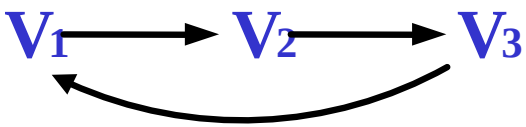
```
void TopsortbyDFS(Graph& G){                    //深度优先拓扑排序,结果逆序
    for(int i=0; i<G.VerticesNum(); i++)        //初始化标志位
        G.Mark[i]=UNVISITED;
    int *result=new int[G.VerticesNum()];       //最终输出的逆序结果
    int tag=0;
    for(i=0; i<G.VerticesNum(); i++)            //对图的所有顶点进行处理
        if(G.Mark[i]== UNVISITED)
            Do_topsort(G,i,result,tag);         //调用递归函数
    for(i=G.VerticesNum()-1;i>=0;i--) {         //逆序输出
        Visit(G, result[i]);
    }
}
```

```
//深度优先搜索实现的拓扑排序
void Do_topsort(Graph& G, int V, int *result, int& tag){
    G.Mark[V]= VISITED;
    //访问V邻接到的所有未被访问过的顶点
    for(Edge e= G.FirstEdge(V); G.IsEdge(e);e=G.NextEdge(e))
        if(G.Mark[G.ToVertex(e)]== UNVISITED)
            Do_topsort(G, G.ToVertex(e), result, tag); //递归调用
    result[tag++]=V;
}
```

环的判断

- 深度优先拓扑排序不能判断环的存在



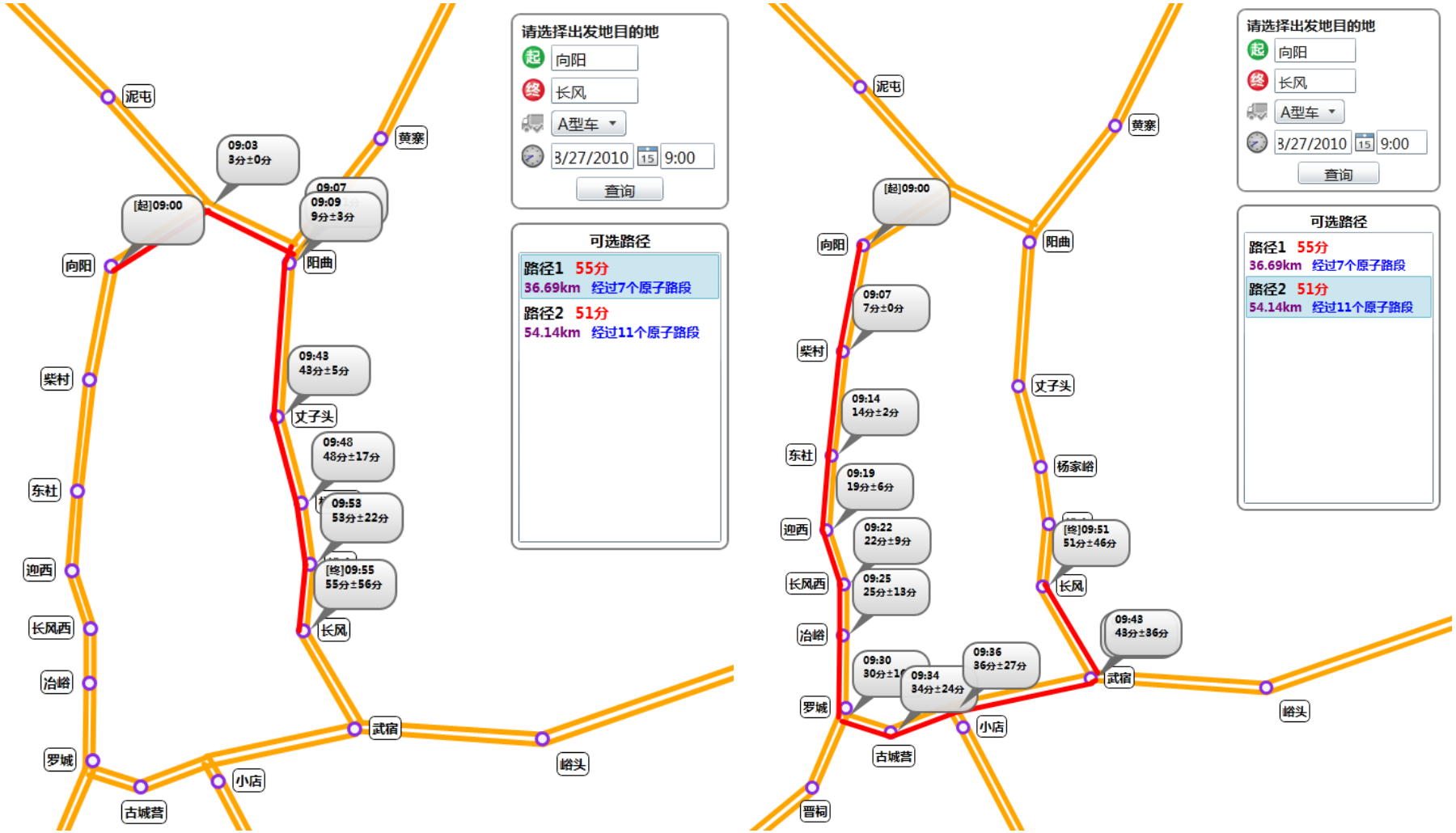


思考题

给出一种实现判断深度优先拓扑排序环存在的算法！

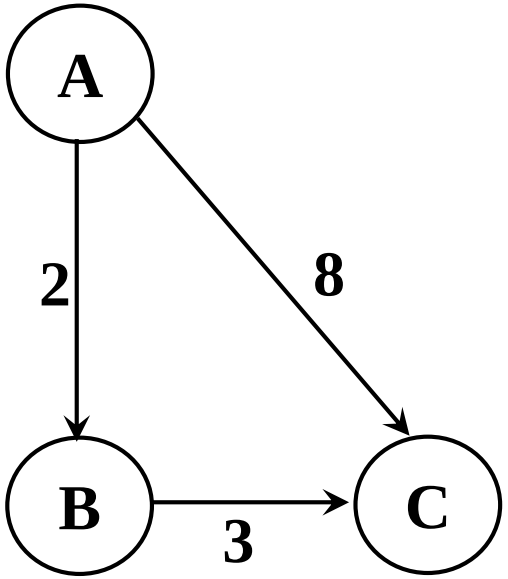
- 设置三个状态：未访问、访问中、已访问
- 如果在DFS过程中遇到一个"访问中"的节点，说明存在环。

7.5 最短路径问题


泥屯
09:03
3分±0分
黄寨
09:07
09:09
9分±3分
[起]09:00
向阳
阳曲
09:43
43分±5分
柴村
丈子头
09:48
48分±17分
东杜
09:53
53分±22分
迎西
[终]09:55
55分±56分
长风西
长风
冶峪
武宿
罗城
小店
峪头
古城营
请选择出发地目的地
起
向阳
终
长风
A型车
3/27/2010
15
9:00
查询
可选路径
路径1 55分
36.69km 经过7个原子路段
路径2 51分
54.14km 经过11个原子路段
泥屯
黄寨
[起]09:00
阳曲
向阳
09:07
7分±0分
柴村
丈子头
09:14
14分±2分
东杜
杨家峪
09:19
19分±6分
迎西
09:22
22分±9分
[终]09:51
51分±46分
长风西
09:25
25分±13分
长风
冶峪
09:43
43分±36分
09:30
09:36
36分±27分
09:34
34分±24分
武宿
罗城
小店
峪头
古城营
晋祠
请选择出发地目的地
起
向阳
终
长风
A型车
3/27/2010
15
9:00
查询
可选路径
路径1 55分
36.69km 经过7个原子路段
路径2 51分
54.14km 经过11个原子路段

问题定义

- 带权图的最短路径问题
  - 即求两个顶点间长度最短的路径
    - 其中：路径长度不是指路径上边数的总和，而是指路径上各边的权值总和
- 管线铺设， 出行线路选择等应用



带权图

- 广度优先遍历本质上就是单位权重图的最短路径搜索问题

最短路径问题求解分类

- 单源最短路径
  - 指对已知图G=（V，E），给定源顶点s∈V，找出s到图中其它各顶点的最短路径
  - 代表性算法：Dijkstra算法（贪心思路）
- 每对顶点间的最短路径
  - 指的是对已知图G=（V，E），任意的顶点V_i，V_j∈V，找出从V_i到V_j的最短路径
  - 代表性算法：Floyd算法（动态规划思路）

Dijkstra（迪杰斯特拉）

- Edsger Wybe Dijkstra，荷兰人
- 20世纪最伟大的计算机科学家之一，曾获1972年图灵奖，与D. E. Knuth并称为20世纪最伟大的计算机科学家
- 主要贡献有如下几个方面：
  - 1)提出“goto有害论”；
  - 2)提出信号量和PV原语；
  - 3)解决了有趣的“哲学家聚餐”问题；
  - 4)最短路径算法(SPF)的创造者；
  - 5)第一个Algol 60编译器的设计者和实现者；
  - 6)THE操作系统的设计者和开发者。

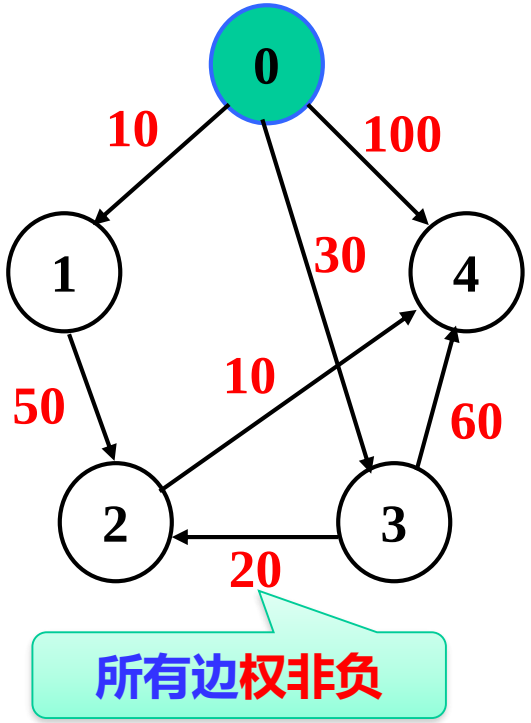


(1930/5/11~2002/8/6)

1、Dijkstra算法

- Dijkstra算法是E.W. Dijkstra于1959年提出的，是目前公认的对**边权非负**情况下的最好算法。
- 基本思想
  - 每次从距离已生成最短路径的节点集“一步之遥”的节点中，选择距离原点V_0最近的边进行延伸
  - 结果由近及远生成以起始点V_0为根的有向树。
- 是一类贪心算法

路径长度递增序



图G中从源点0到其余各点的最短路径

源点	中间结点	终点	路径长度
0		1	10
0		3	30
0	3	2	50
0	3，2	4	60

- 按路径长度递增序产生各顶点最短路径
  - 若按长度递增的次序生成从源点s到其它顶点的最短路径，则当前正在生成的最短路径上除终点以外，其余顶点的最短路径均已生成

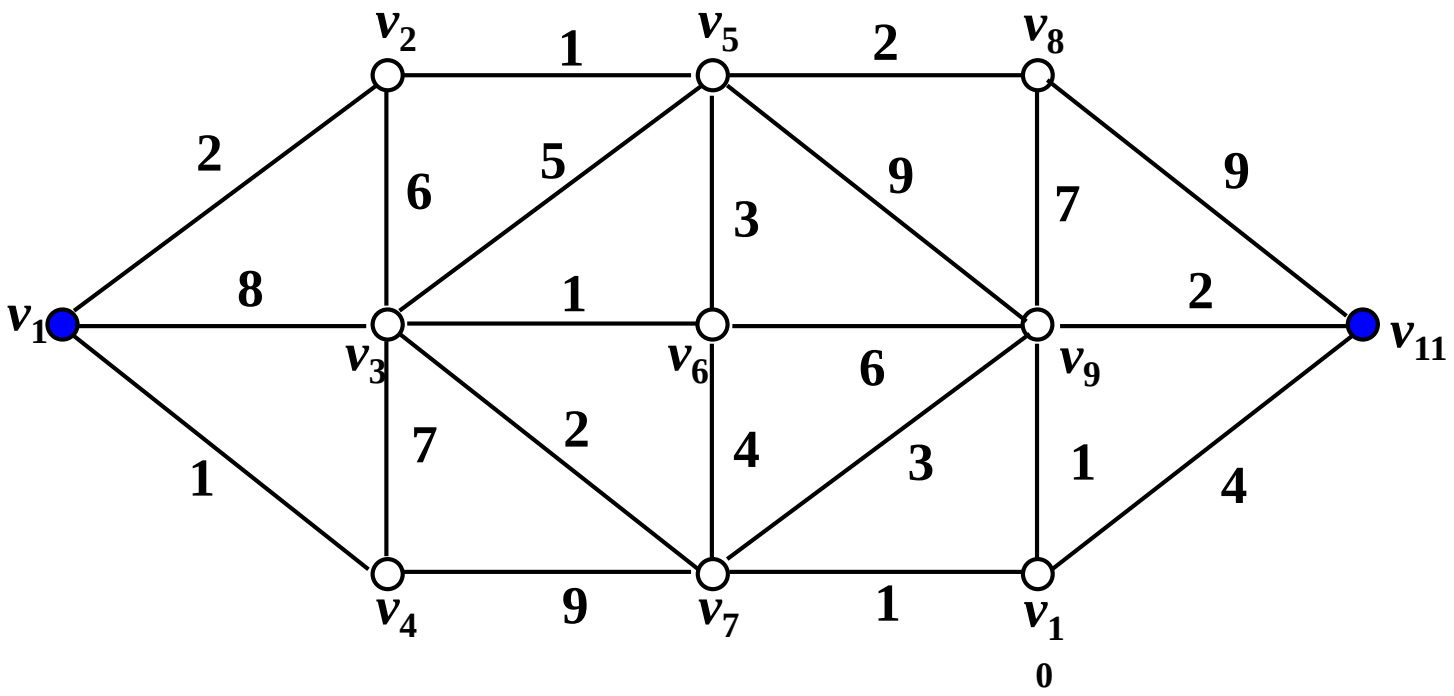
实现策略

- 把图中顶点分成两组
  - 第一组：已确定最短路径的顶点
  - 第二组：尚未确定最短路径的顶点
- 按最短路径长度递增顺序逐个把第二组的顶点加到第一组中
  - 直至从s出发可以到达的所有顶点都包括进第一组
- 在合并过程中，保持s到第一组各顶点的最短路径长度都不大于从s到第二组各顶点的最短路径长度
  - 第一组顶点对应的距离值：从s到该顶点的最短路径长度
  - 第二组顶点对应的距离值：从s到该顶点的值包括第一组的顶点为中间顶点的最短路径长度

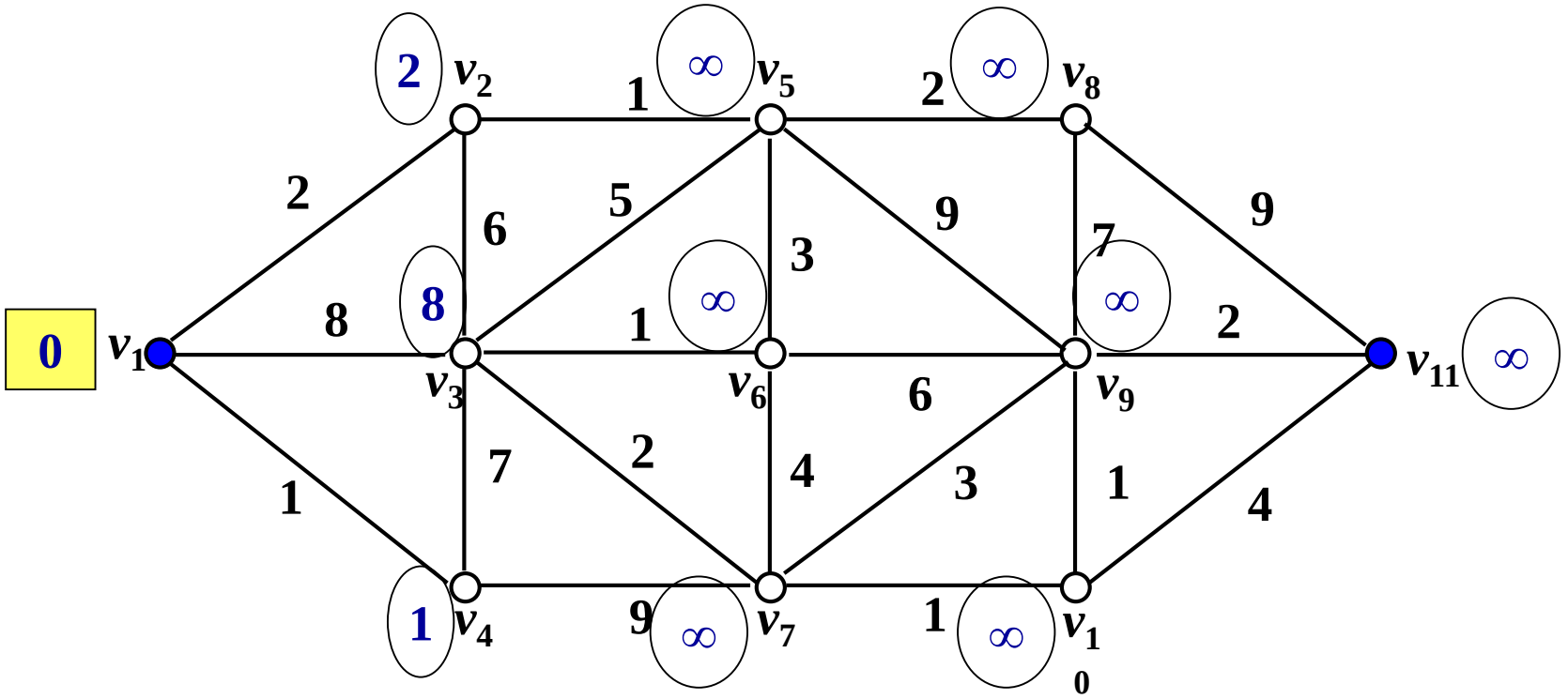
具体过程

- **初始化**：第一组只包括源点s，第二组包括其它所有顶点
  - s距离值为0，第二组顶点的距离值确定如下：
    - 若有边<s，V_i>或（s，V_i），则Vi的距离值为边所带的权，否则为∞
- **过程**：每次从第二组的顶点中选一个其距离值为最小的顶点V_m加入到第一组中
  - 每往第一组加入顶点V_m，要对第二组各顶点的距离值进行一次修正
    - 若使用V_m做中间顶点，使从s到V_i的最短路径比不加V_m的短，则需要修改V_i的距离值
  - 修改后再选距离值最小的顶点加入到第一组中，重复上述过程
  - 结束条件：直到图的所有顶点都包括在第一组中或者再也没有可加入到第一组的顶点存在

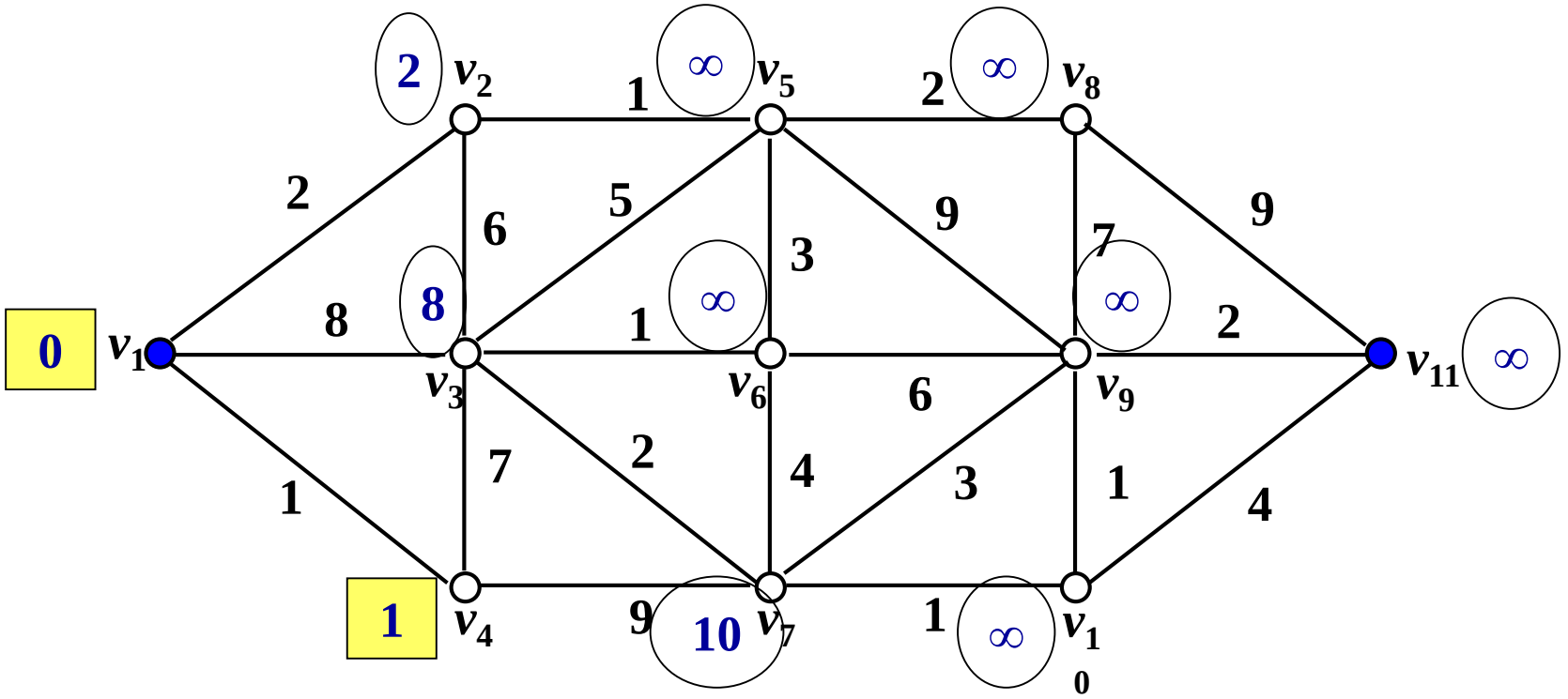
示例



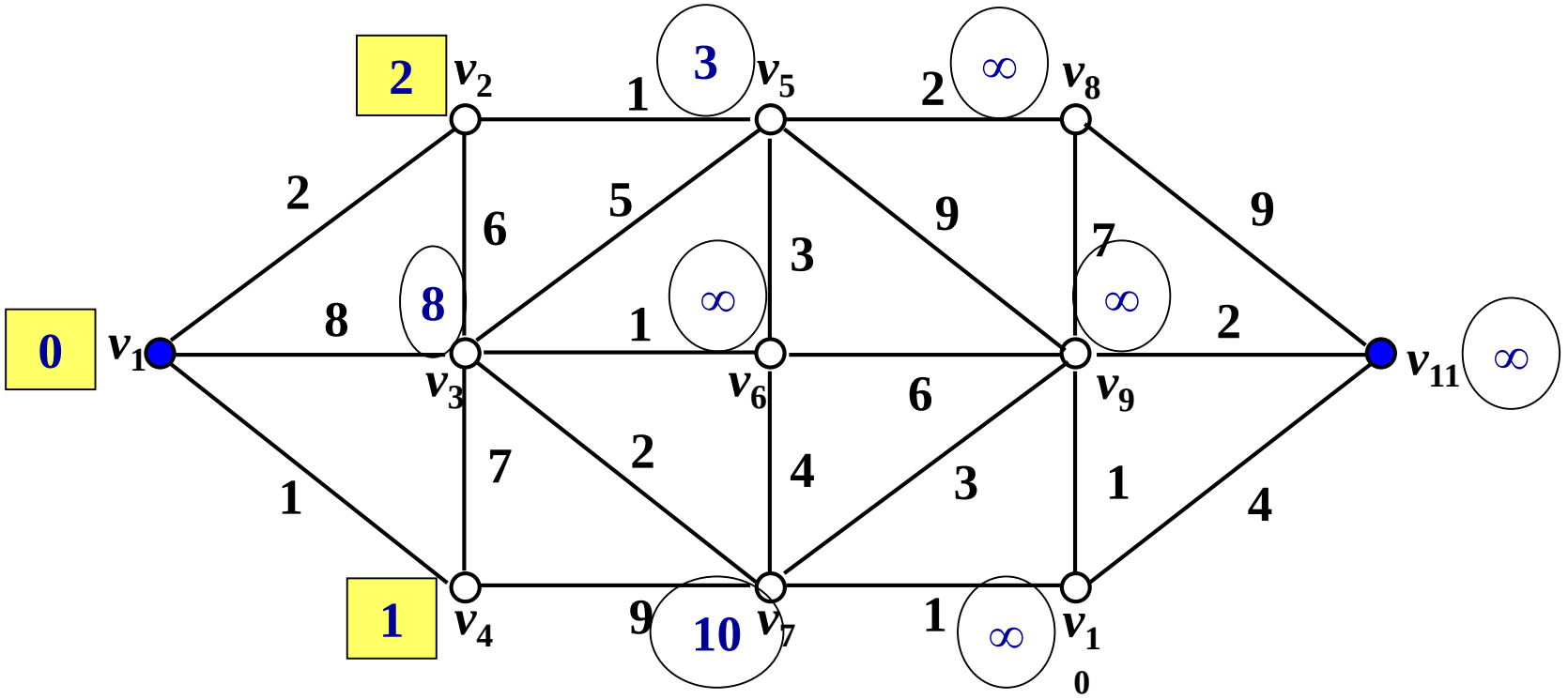
示例



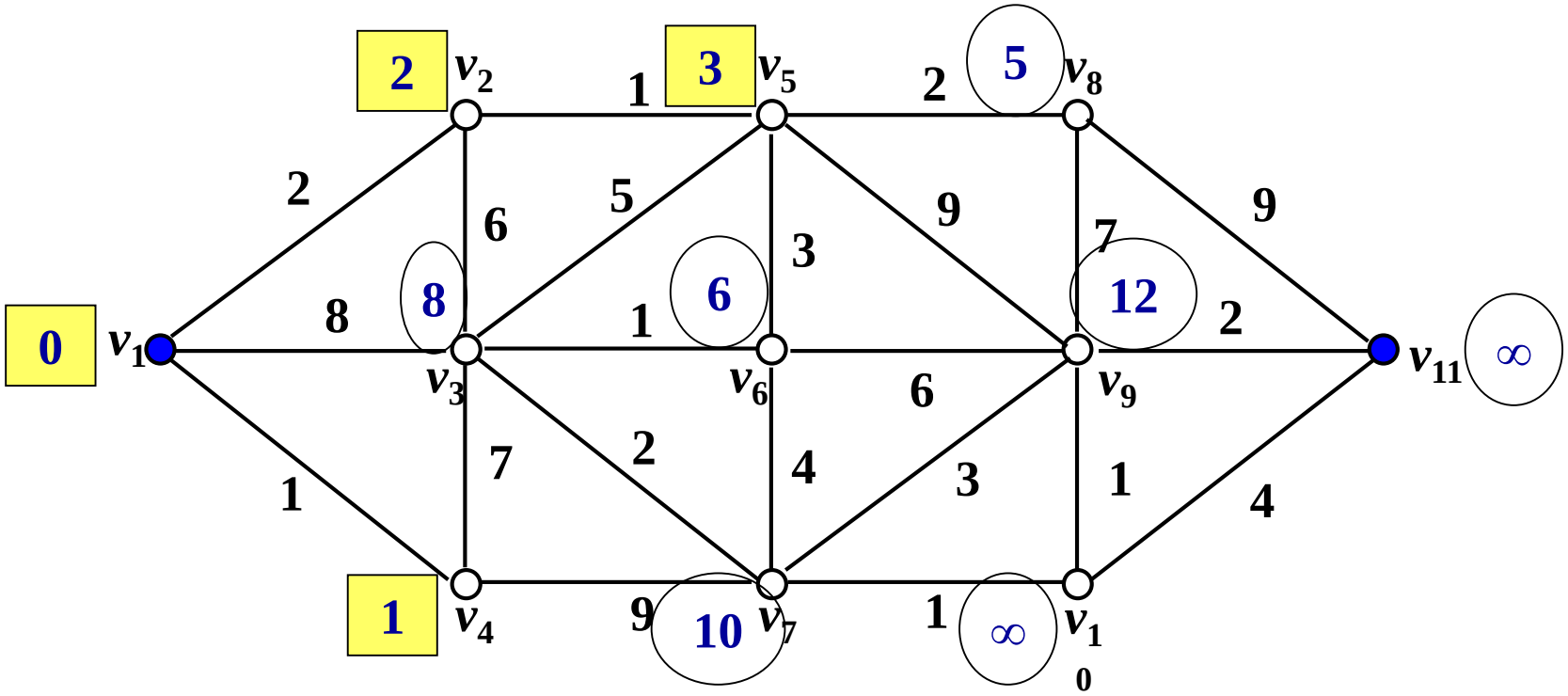
示例


0
v_1
2
v_2
∞
v_5
∞
v_8
8
v_3
∞
v_6
∞
v_9
∞
v_{11}
1
v_4
10
v_7
∞
v_{10}
2
1
2
9
6
5
3
9
7
9
8
1
2
6
7
2
4
3
1
4
1
9
1

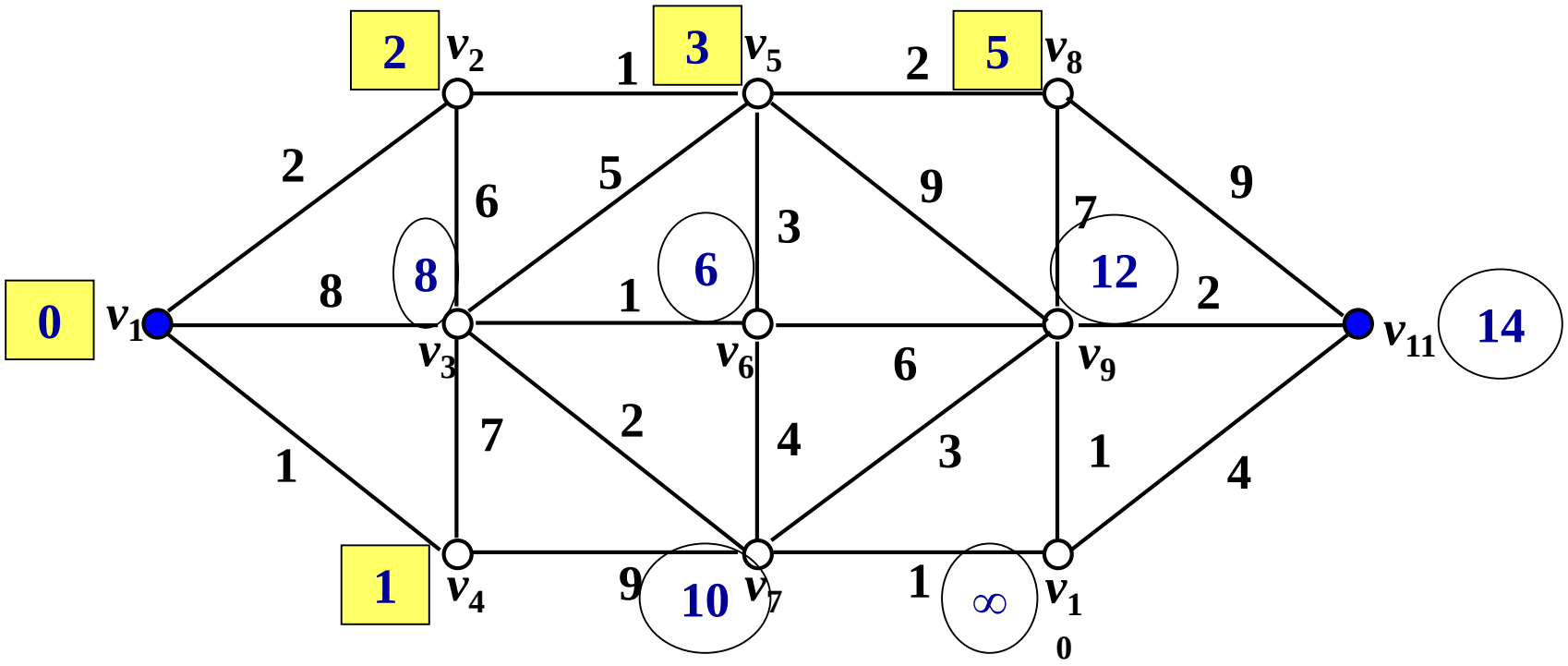
示例


0
v_1
2
v_2
3
v_5
∞
v_8
8
v_3
∞
v_6
∞
v_9
∞
v_{11}
1
v_4
10
v_7
∞
v_{10}
2
1
6
5
9
7
9
8
1
3
2
6
7
2
4
3
1
4
9
1
0

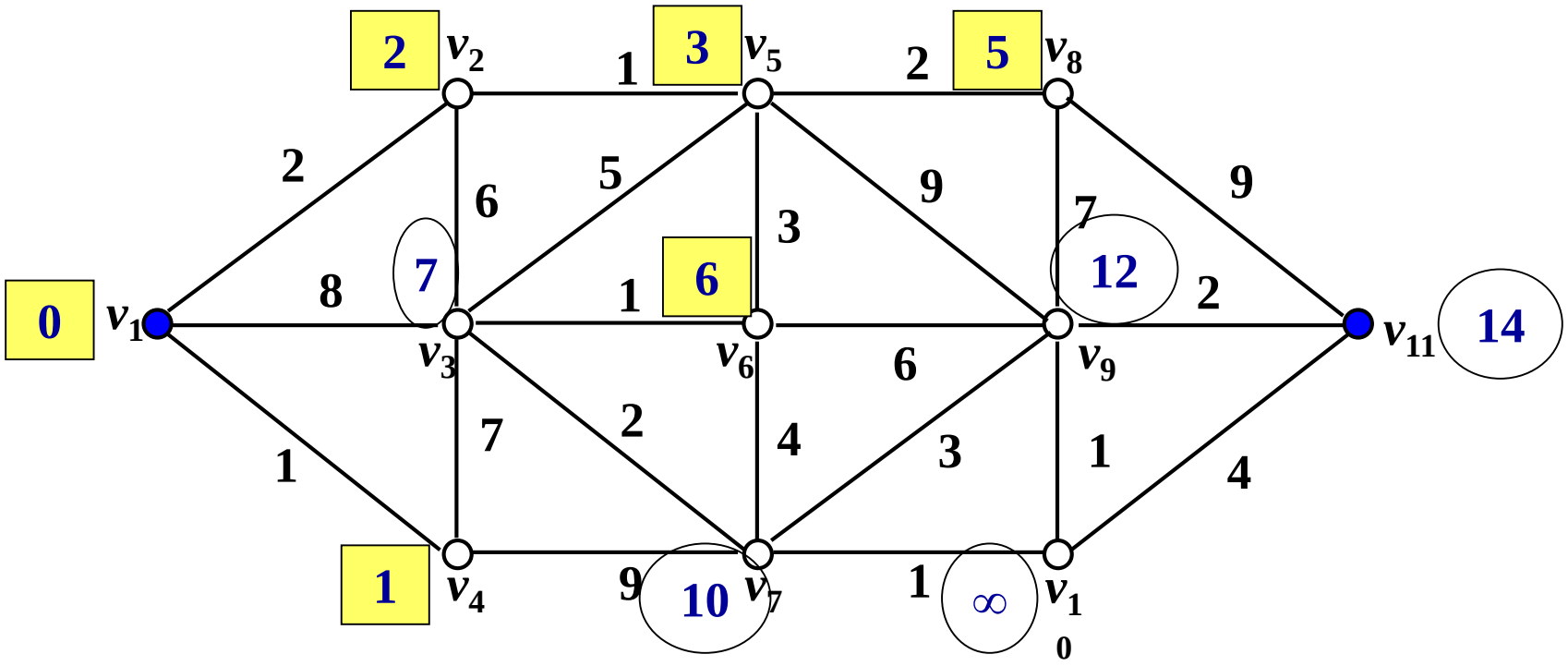
示例


0
v_1
2
v_2
3
v_5
5
v_8
8
6
12
v_3
6
v_6
v_9
v_{11}
∞
1
v_4
10
v_7
∞
v_{10}
2 1 2 9 8 6 5 3 9 7 1 6 2 7 2 4 3 1 1 9 1 4

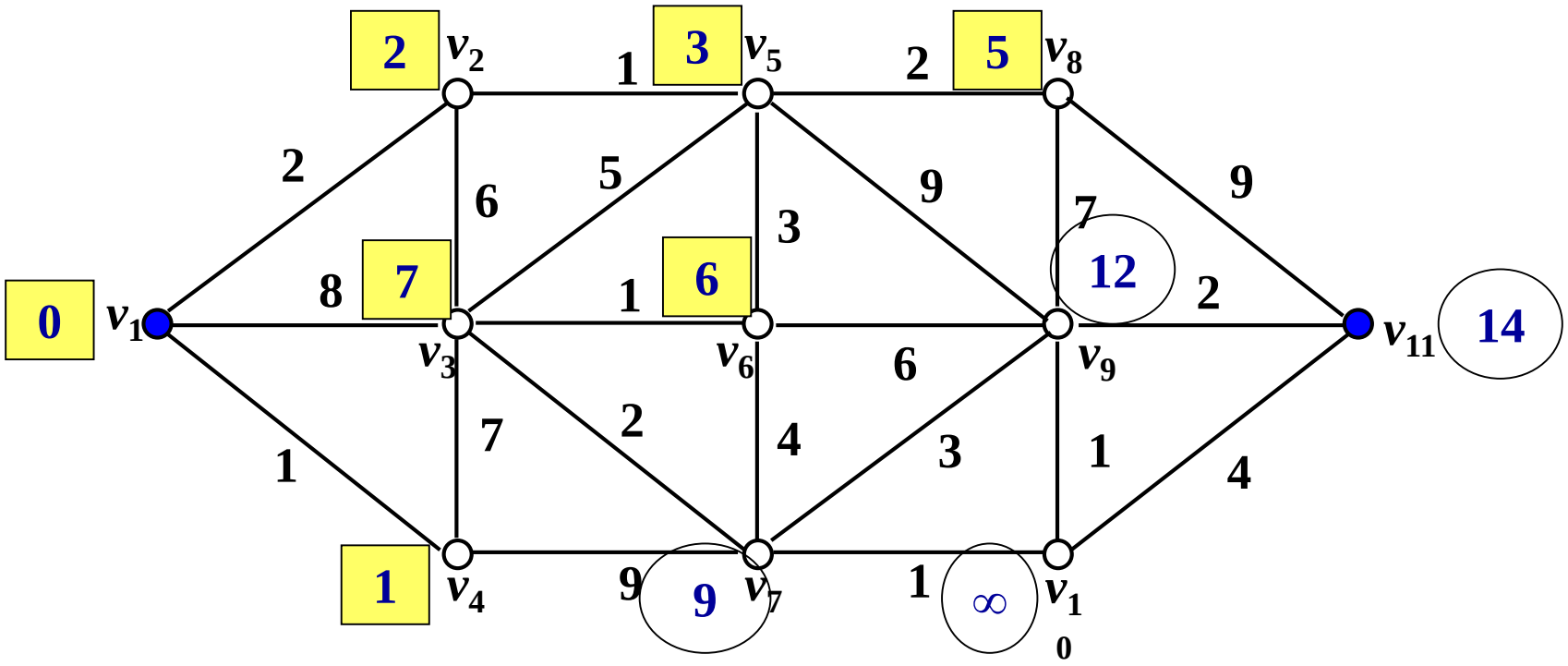
示例


0
v_1
2
v_2
3
v_5
5
v_8
1
2
2
6
5
9
9
7
3
8
8
6
12
1
2
v_3
v_6
v_9
v_{11}
14
6
1
7
2
4
3
1
4
1
v_4
9
10
v_7
1
∞
v_{10}

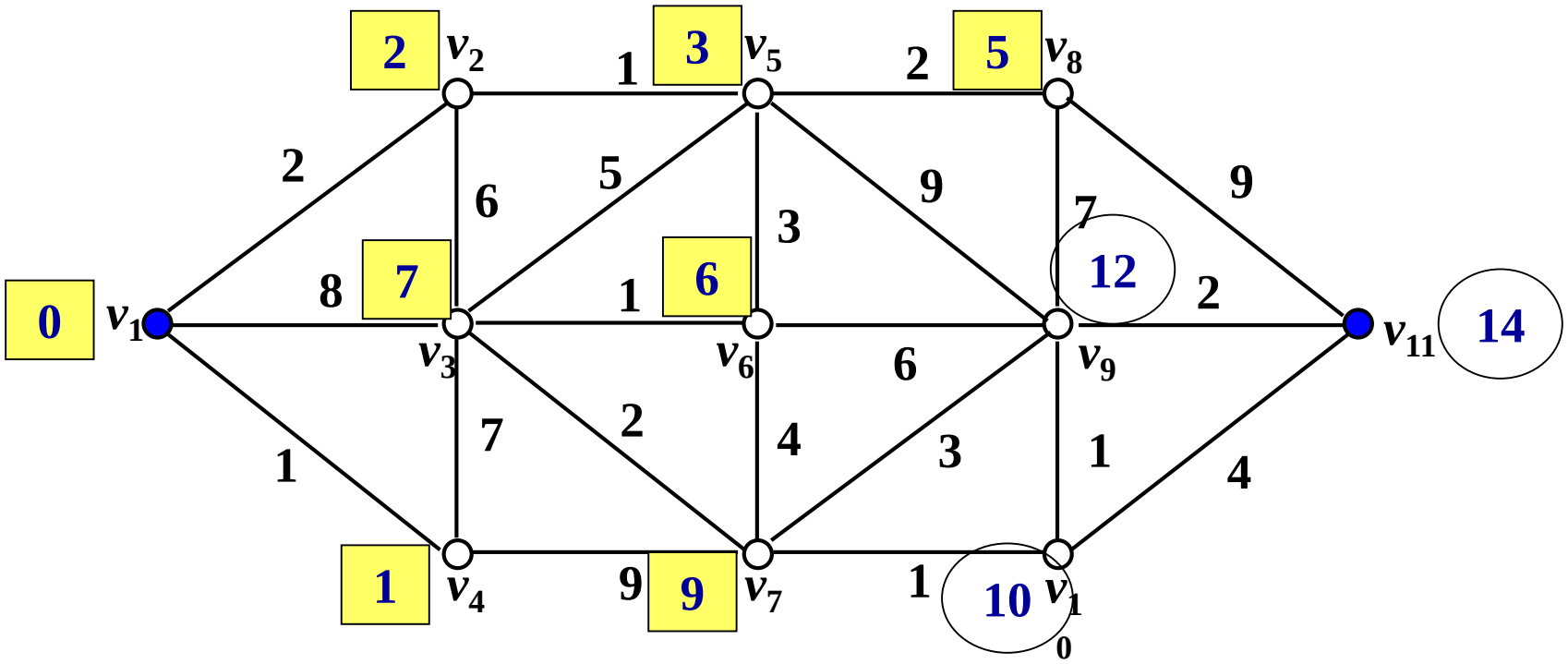
示例


0
v_1
2
v_2
3
v_5
5
v_8
1
2
2
6
5
9
3
7
9
8
7
1
6
12
2
v_3
v_6
v_9
v_{11}
14
6
7
2
4
3
1
4
1
1
v_4
9
10
v_7
1
∞
v_{10}

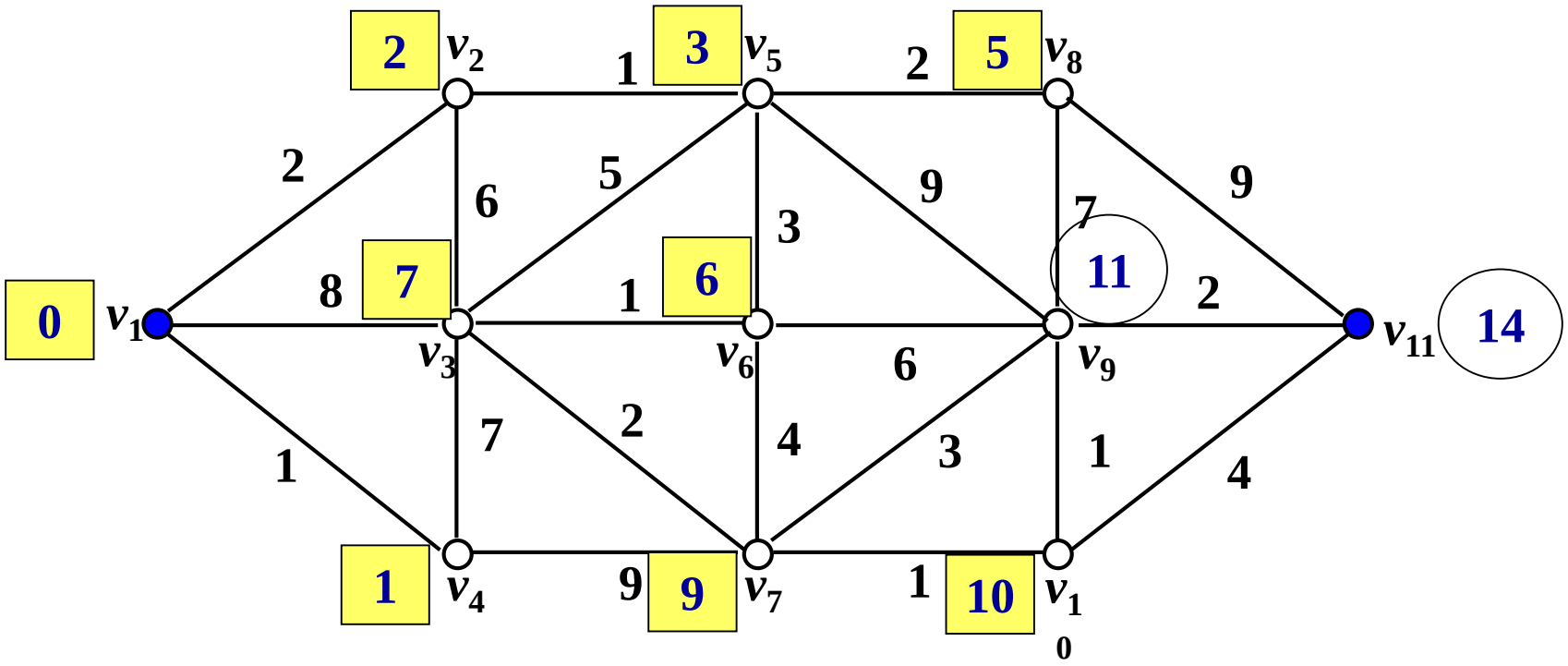
示例


0
v_1
2
v_2
7
8
v_3
1
v_4
3
v_5
6
1
v_6
9
9
v_7
5
v_8
12
7
v_9
1
∞
v_{10}
0
14
v_{11}
2
1
6
5
2
6
3
9
7
2
4
3
1
9
1
4

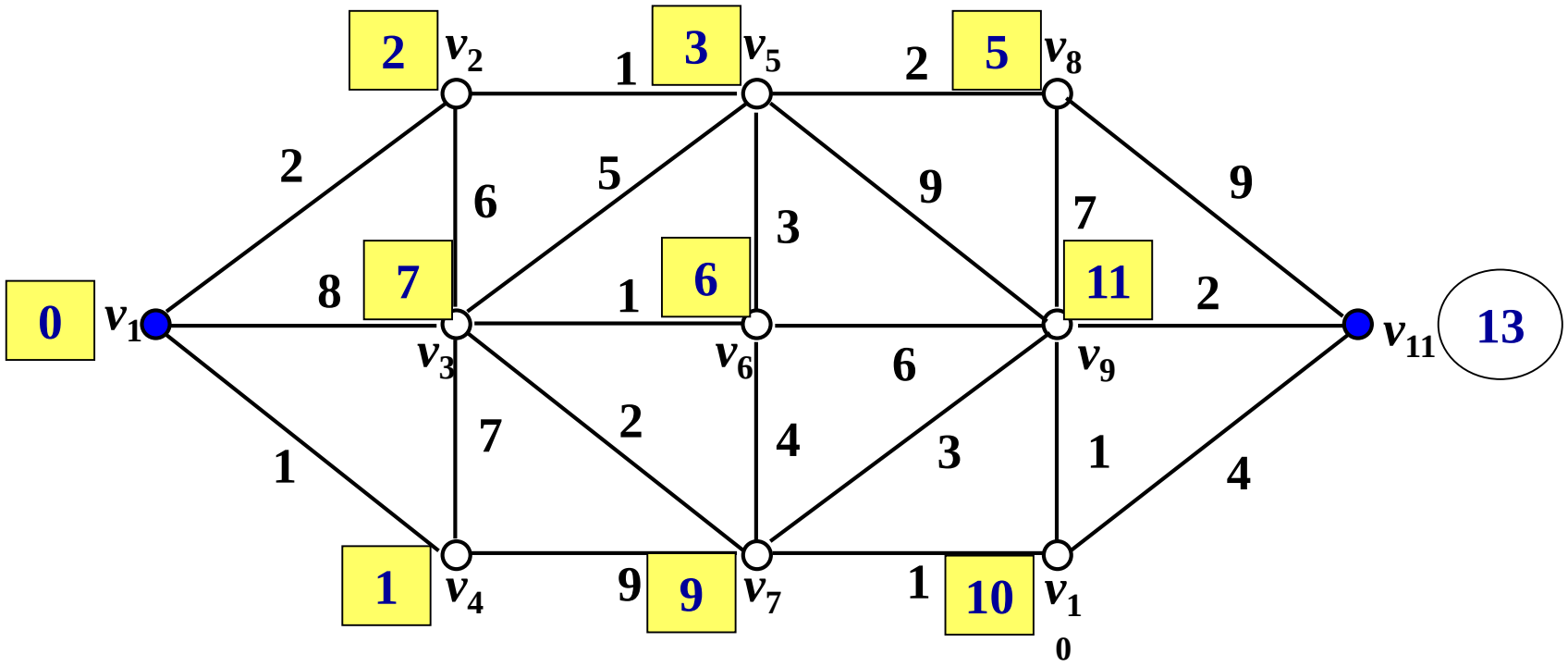
示例


0
v_1
2
v_2
7
8
v_3
1
v_4
3
v_5
6
1
v_6
9
9
v_7
5
v_8
12
7
6
3
v_9
10
1
v_{10}
0
v_{11}
14
2
1
6
5
2
9
9
2
7
4
3
1
4

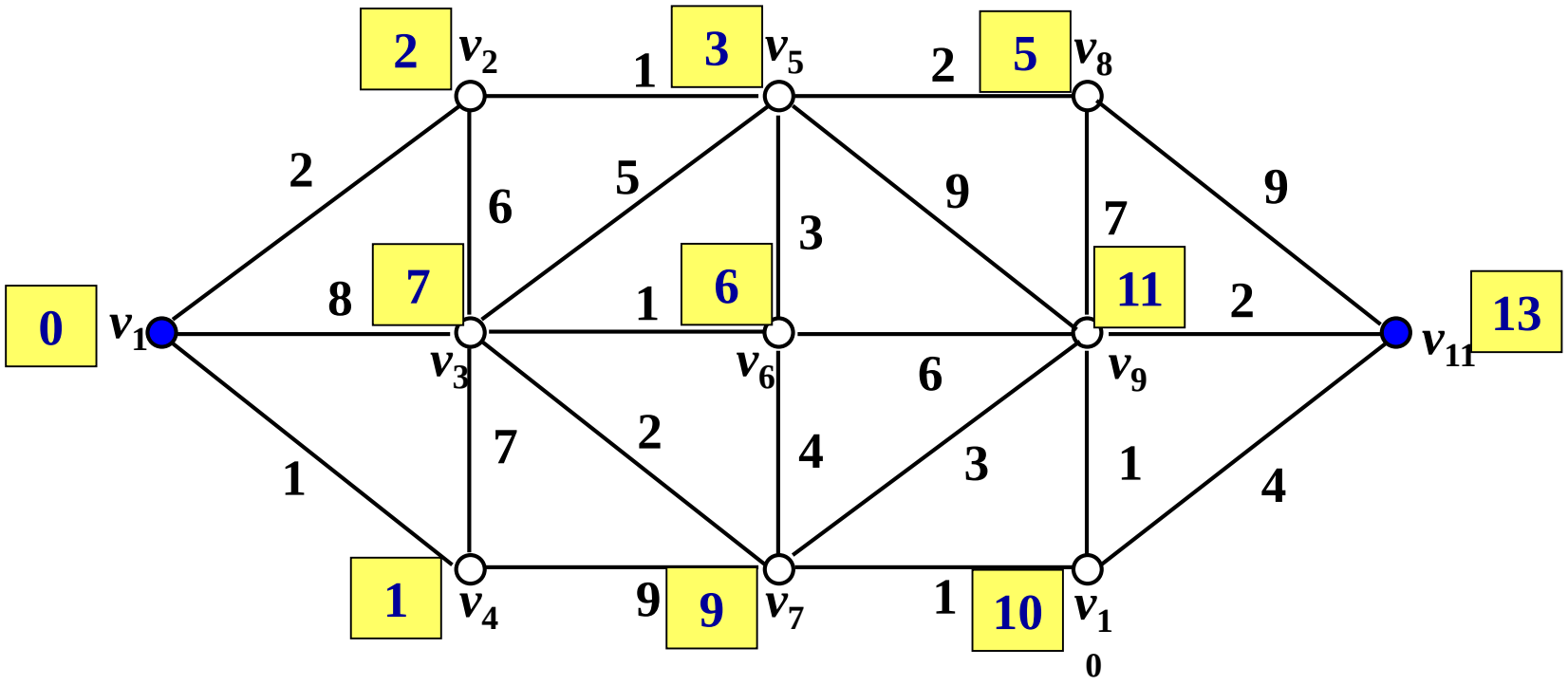
示例


0
v_1
2
v_2
3
v_5
5
v_8
7
11
14
v_{11}
8
7
v_3
1
6
v_6
6
v_9
1
1
v_4
9
9
v_7
1
10
v_{10}
2
1
2
6
5
9
3
9
7
2
4
3
1
4

示例


0
v_1
2
v_2
3
v_5
5
v_8
7
v_3
6
v_6
11
v_9
13
v_{11}
1
v_4
9
v_7
10
v_{10}
2
1
2
9
8
1
6
5
9
3
7
6
7
2
4
3
1
4
1
9
1
0

示例



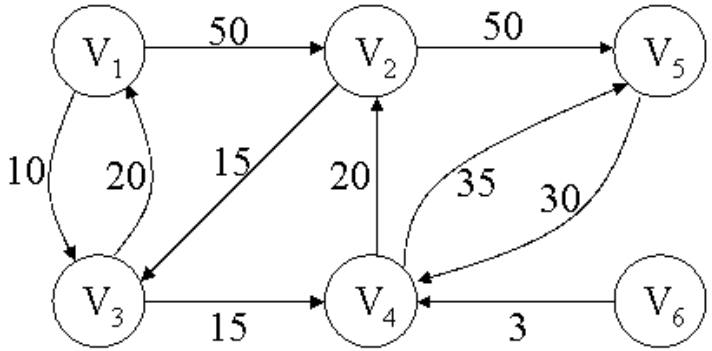
至此， 从v1到其他所有点的路径就全部求出来了。

举例

Dist 数组

最短路径的表示方法

	V_1	V_2	V_3	V_4	V_5	V_6
初始状态	length:∞ pre:1	length:∞ pre:1	length:∞ pre:1	length:∞ pre:1	length:∞ pre:1	length:∞ pre:1
V_1进入第一组	length:0 pre:1	length:50 pre:1	length:10 pre:1	length:∞ pre:1	length:∞ pre:1	length:∞ pre:1
V_3进入第一组	length:0 pre:1	length:50 pre:1	length:10 pre:1	length:25 pre:3	length:∞ pre:1	length:∞ pre:1
V_4进入第一组	length:0 pre:1	length:45 pre:4	length:10 pre:1	length:25 pre:3	length:60 pre:4	length:∞ pre:1
V_2进入第一组	length:0 pre:1	length:45 pre:4	length:10 pre:1	length:25 pre:3	length:60 pre:4	length:∞ pre:1
V_5进入第一组	length:0 pre:1	length:45 pre:4	length:10 pre:1	length:25 pre:3	length:60 pre:4	length:∞ pre:1



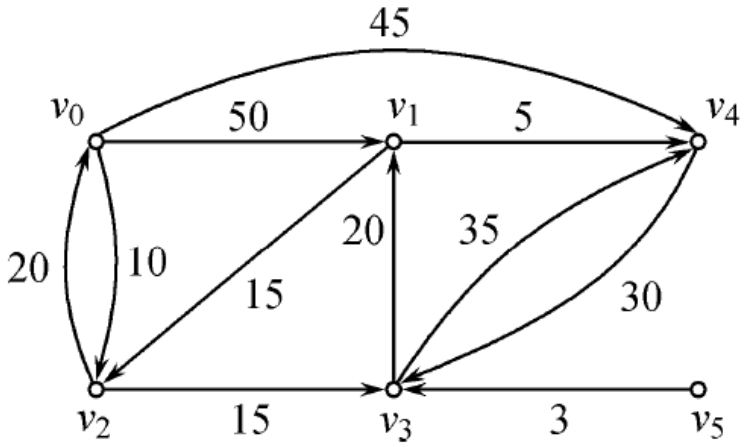
借助一个长度为N的数组，包括：

1）源点到当前节点的路径长度（length）

2）当前节点的前驱节点（pre）

Dijkstra算法-例子

- **已知带权图G如图所示及其邻接矩阵A，求从顶点v0到其它各顶点的最短路径**



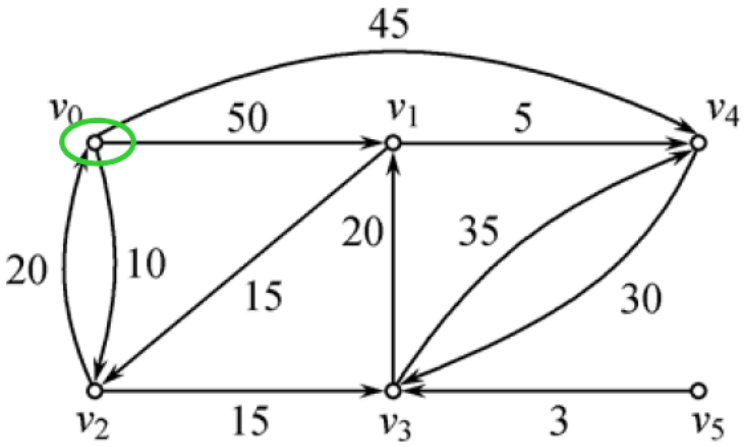
$$\text{arcs} = \begin{bmatrix} 0 & 50 & 10 & \infty & 45 & \infty \\ \infty & 0 & 15 & \infty & 5 & \infty \\ 20 & \infty & 0 & 15 & \infty & \infty \\ \infty & 20 & \infty & 0 & 35 & \infty \\ \infty & \infty & \infty & 30 & 0 & \infty \\ \infty & \infty & \infty & 3 & \infty & 0 \end{bmatrix}$$

Dijkstra算法-例子

① **初始时，集合U中只有顶点v_0，从顶点v_0到其它顶点v_i (i=1,2,…,n-1)的最短路径长度为边(v_0,v_i)的权值。若顶点v_0和v_i不相邻，则假设存在一条从v_0到v_i权为无穷的边**

dist[n]={{0,0},{50,0},{10,0},{MAX,-1},{45,0},{MAX,-1}}

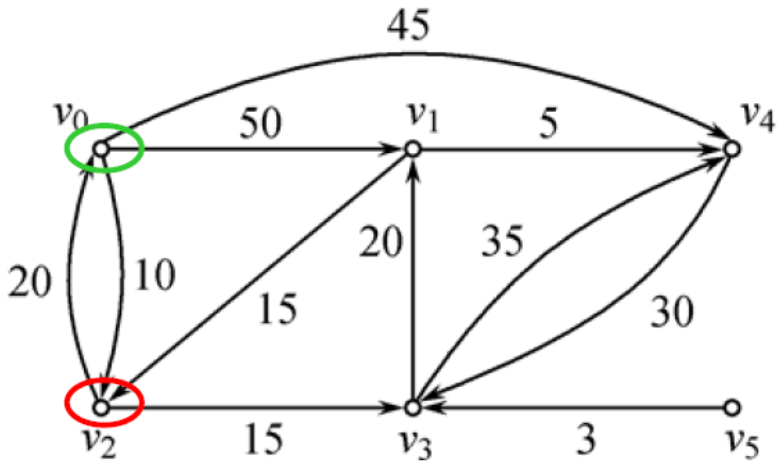
最短路径长度 | 前驱



Dijkstra算法-例子

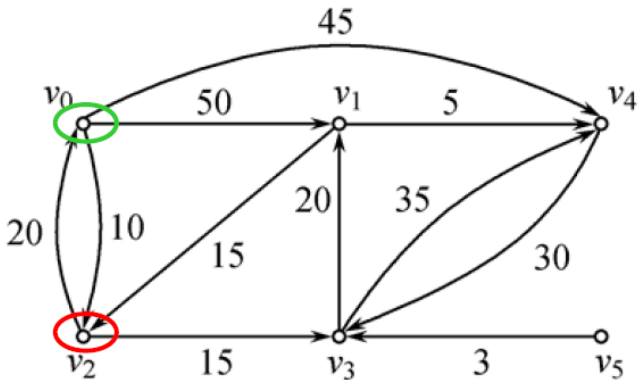
② **在集合V-U中找出距离值最小的顶点v_{min}=2，将其加入集合U，从顶点v_0到顶点v_{min}的最短路径长度就是v_{min}的距离值**

dist[n]={{0,0},{50,0},{10,0},{MAX,-1},{45,0},{MAX,-1}}



Dijkstra算法-例子

dist[n]={{0,0},{50,0},{10,0},{MAX,-1},{45,0},{MAX,-1}}



③ 调整集合V-U中顶点的距离值，对v_i ($v_i \in$ V-U)，如果

dist[i].length > dist[min].length+graph.arcs[min][i]

则将顶点v_i的距离值改为dist[min].length+graph.arcs[min][i]，

并将路径上v_i的前趋顶点改为v_{min}，即：dist[i].prevex=min

min=2

- 因为dist[1].length=50，dist[2].length+graph.arcs[2][1]=10+MAX，顶点v1的距离值不需要调整
- 因为dist[3].length=MAX，dist[2].length+graph.arcs[2][3]=10+15=25，顶点v3的距离值调整为25，其前趋顶点为v2
- 同理，顶点v4，v5的距离值不需要调整

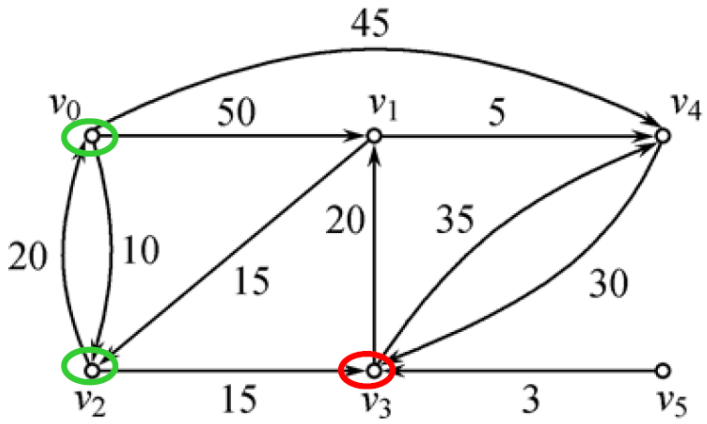
dist[n]为{{0,0},{50,0},{10,0},{25,2},{45,0},{MAX,-1}}

Dijkstra算法-例子

第三步所得结果:
dist[n]为{{0, 0}, {50, 0}, {10, 0}, {25, 2}, {45, 0}, {MAX, -1} }

④ 同理在集合V-U中找出当前距离值最小的顶点v3，并调整集合V-U中顶点的距离值 dist[n]为

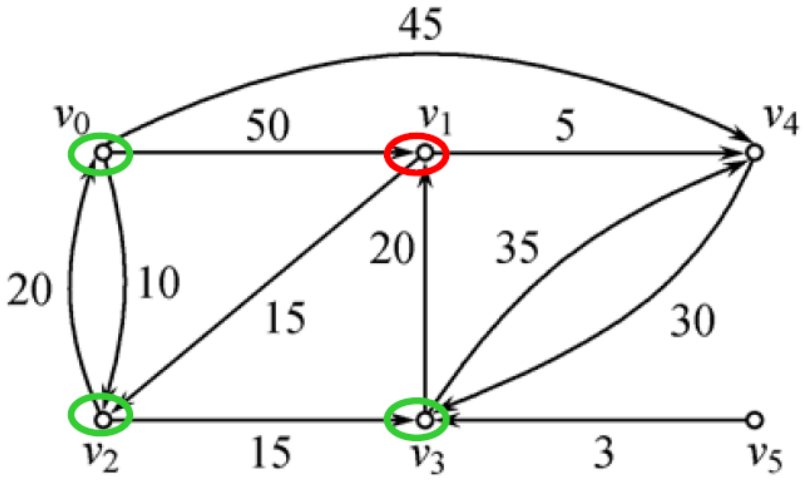
{{0,0},{45,3},{10,0},{25,2},{45,0},{MAX,-1}}调整v1



Dijkstra算法-例子

第四步所得结果:
dist[n]为{{0, 0},{45, 3}, {10, 0}, {25, 2}, {45, 0}, {MAX, -1} }

⑤ 在集合V-U中找出当前距离值最小的顶点v1
dist[n]为{{0,0},{45,3},{10,0},{25,2},{45,0},{MAX,-1}}

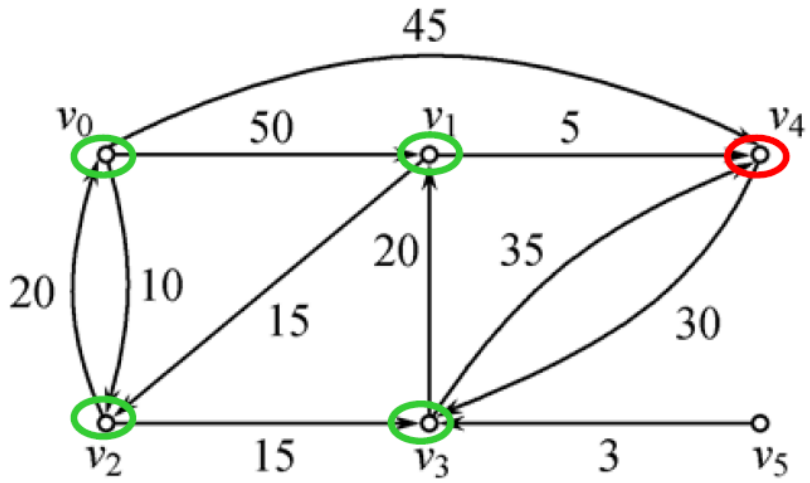


Dijkstra算法-例子

第五步所得结果:

dist[n]为{{0, 0}, {45, 3}, {10, 0}, {25, 2}, {45, 0}, {MAX, -1} }

⑥ 在集合V-U中找出当前距离值最小的顶点v4
dist[n]为{{0,0},{45,3},{10,0},{25,2},{45,0},{MAX,-1}}

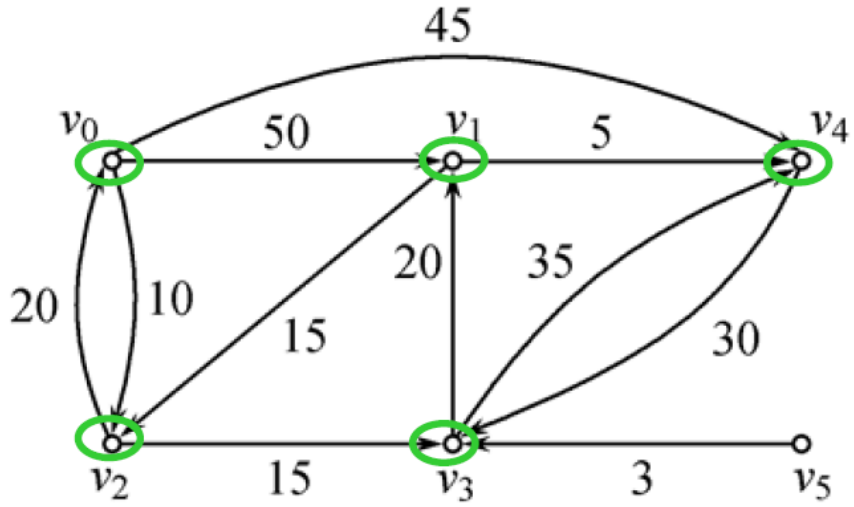


Dijkstra算法-例子

第六步所得结果:

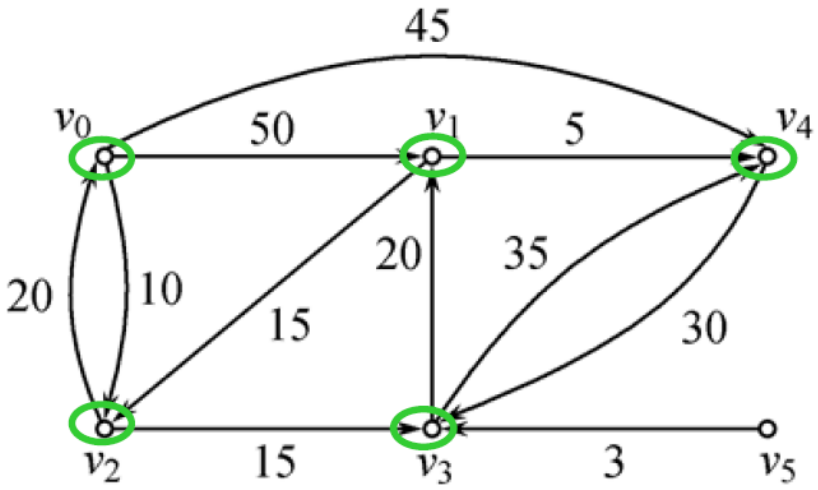
dist[n]为{{0, 0}, {45, 3}, {10, 0}, {25, 2}, {45, 0}, {MAX, -1} }

⑦ 没有可以再加入集合U的顶点了，说明从顶点v0到顶点v5之间无路径相通。



Dijkstra算法

- dist[n]={{0, 0}, {45, 3}, {10, 0}, {25, 2}, {45, 0}, {MAX, -1}}
- 由数组dist的prevex字段得到顶点v0到各顶点的最短路径
  - 如从v0到v1的最短路径，dist[1].prevex=3可知路径上顶点v1的前一个顶点是v3，dist[3].prevex=2可知路径上顶点v3的前一个顶点是v2，dist[2].prevex=0可知路径上前一个顶点是v0，即最短路径为(v0 , v2 , v3 , v1)。
  - 其路径长度为：dist[1].length=45



Dijkstra算法描述

```
class Dist {        // Dist类，Dijkstra和Floyd算法用于保存最短路径信息
public:
    int index;      // 顶点的索引值，仅Dijkstra算法用到
    int length;     // 当前最短路径长度
    int pre;        // 路径最后经过的顶点
};
void Dijkstra(Graph& G,int s, Dist* &D) {
    D=new Dist[G.VerticesNum()];
    for(int i=0;i<G.VerticesNum();i++){          //初始化Mark数组、D数组
        G.Mark[i]=UNVISITED;
        D[i].length= INFINITY;
        D[i].index=i;                            //顶点的索引值;
        D[i].pre=s;                              //路径最后经过的顶点
    }
```

初始化

```
D[s].length=0;                                  //源点为s;
MinHeap<Dist> H(G.EdgesNum());                  //声明一个最小值堆;
H.Insert(D[s]);                                 //以源点s初始化堆;
for(i=0;i<G.VerticesNum();i++){
        Bool FOUND =  false;
        Dist d;
        while (! H.empty()){
                d = H.RemoveMin();
                if( G.Mark[d.index]==UNVISITED)
                        FOUND = true;     break      //把该点加入已访问组;
        }
```

找下一最短路径顶点

```
if (! FOUND)   break;
int v =d.index;   G.Mark[v] = VISITED;  Visit(v);     //打印输出;
for(Edge e=G.FirstEdge(v);G.IsEdge(e);e=G.NextEdge(e)){
        if(D[G.ToVertex(e)].length>(D[v].length + G.Weight(e))){
                D[G.ToVertex(e)].length=D[v].length + G.Weight(e);
                D[G.ToVertex(e)].pre=v;
                H.insert(D[G.ToVertex(e)]);  //入堆;


        }
    }
}//end for;
}
```

更新权值等信息（松弛技术）

时间复杂性分析

- 如果不采用最小堆的方式，而是通过两两比较来扫描D数组
  - 每次寻找权值最小结点，需要进行$|V|$次扫描，每次扫描$|V|$个顶点（$|V|^2$），而在更新D值处总共扫描$|E|$次
  - 总时间代价为$\Theta(|V|^2 + |E|) = \Theta(|V|^2)$
- 如果采用最小堆的方式
  - 每次改变D[i].length，通过先删除再重新插入的方法来改变顶点i在堆中的位置
  - 或者仅为某个顶点添加一个新值(更小的)，作为堆中新元素(而不作删除旧值的操作，因为旧值被找到时，该顶点一定被标记为VISITED，从而被忽略)。
  - 不删除旧值做法的缺点是，在最差情况下，它将使堆中元素数目由$\Theta(|V|)$增加到$\Theta(|E|)$，此时总的时间代价为$\Theta((|V|+|E|)\log|E|)$，因为处理每条边时都必须对堆进行一次重排

2、Floyd算法



- 求每对顶点间的最短路径
  - 方法1：反复执行Dijkstra算法
  - 方法2：Floyd算法
- Floyd算法
  - 该算法名称以创始人之一、1978年图灵奖获得者、斯坦福大学计算机科学系教授罗伯特·弗洛伊德命名。
  - 又称插点法，是一种用于寻找给定的加权图中任意节点对之间的最短路径算法。
  - 是一类动态规划的方法

Floyd算法的基本思路

- 时间复杂度为$O(n^3)$，但算法较为简单，容易理解。
- 对于图G=(V,E)，有n个顶点。求顶点vi到vj的最短路径
  - 首先考虑路径(vi, v0)和(v0,vj)是否存在？如果存在，则比较(vi,vj)和(vi,v0)+(v0,vj)的路径长度，取较短者为当前的最短路径

(vi, vj)

(vi, v0, vj)

  - 其次，考虑从vi到vj是否存在包含v1为中间点的路径？如果存在，则将其与前一次求得的允许v0为中间点的最短路径长度比较，取较短者为当前的最短路径。

(vi, v0, vj)

(vi, {v0, v1}, vj)

Floyd算法的基本思路

- 如果(vi,...,vk)和(vk, ...,vj)分别是从vi到vk和从vk到vj允许k个顶点v0,v1,..., vk-1为中间点的最短路径，则将(vi,...,vk)+(vk, ...,vj) 与已经得到的从vi到vj允许k个顶点v0,v1,...,vk-1为中间点的最短路径进行比较，取较短者为从vi到vj允许k+1个顶点v0,v1,...,vk为中间点的最短路径。

$$(v_i, \{v_0,v_1,\ldots,v_{k-1}\}, v_j)$$

$$(v_i, \{v_0,v_1,\ldots, v_{k-1},v_k\}, v_j)$$

.....

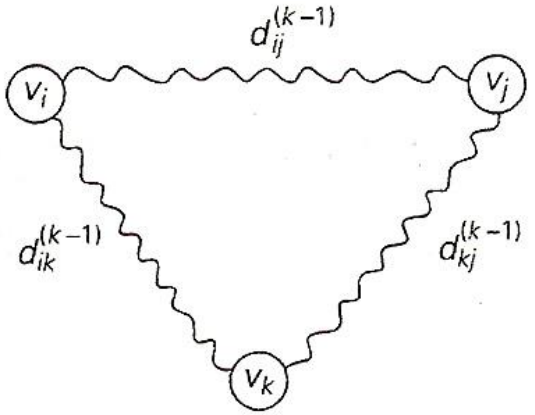
- 依次类推，直到加入顶点vn-1为止，则得到的是vi到vj允许n个顶点v0,v1,...,vn-1为中间点的最短路径。此时，已经考虑了所有顶点为中间点的可能性，因此得到结果。

算法过程

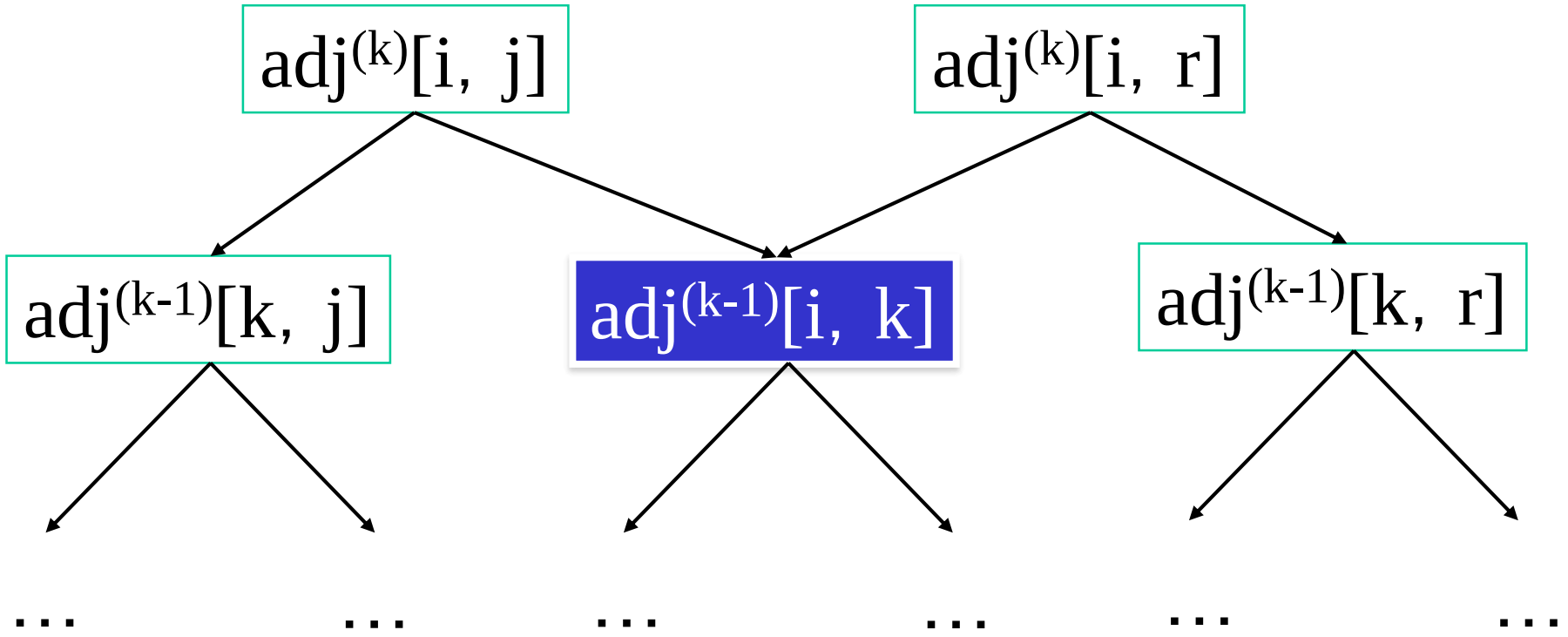
- 假设用相邻矩阵adj表示图
  - 任意两点间距离是边的权，如果两点间没有边直接相连，则权为无穷大（∞）
- 在原图的相邻矩阵$adj^{(0)}$上做n次迭代，递归地产生一个矩阵序列$adj^{(1)}$, $adj^{(2)}$,…, $adj^{(n)}$
  - $adj^{(k)}[i，j]$等于从顶点V_i到顶点V_j中间顶点序号不大于k的最短路径长度
- $adj^{(n)}$包括了所有最终的最短路径

递推公式

- 假设已求得矩阵$adj^{(k-1)}$，那么从顶点V_i到顶点V_j中间顶点的序号不大于k的最短路径有两种情况：
  - 中间不经过顶点V_k，那么就有$adj^{(k)}[i，j]=adj^{(k-1)}[i，j]$
  - 中间经过顶点V_k，那么$adj^{(k)}[i,j]< adj^{(k-1)}[i,j]$，且$adj^{(k)}[i,j]= adj^{(k-1)}[i,k] + adj^{(k-1)}[k,j]$



动态规划递推过程

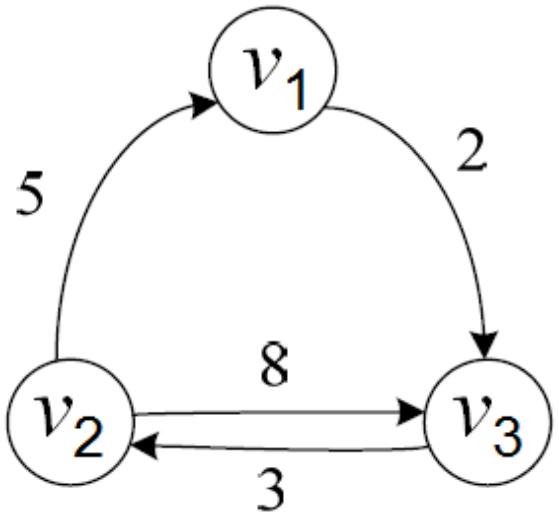


最短路径确定

确定最短路径

- 设置一个n×n的矩阵path，path[i，j]记录顶点v_i到顶点v_j的最短路径上排在顶点v_j前面的那个顶点。
- 即当*k*使得$adj^{(k)}[i,j]$达到最小值，那么就置path[i,j]=path[k,j]
- 如果当前没有最短路径时，就将path[i，j]置为-1。

示例



$$\text{adj} = \begin{bmatrix} 0 & \infty & 2 \\ 5 & 0 & 8 \\ \infty & 3 & 0 \end{bmatrix} \qquad \text{path} = \begin{bmatrix} 1 & -1 & 1 \\ 2 & 2 & 2 \\ -1 & 3 & 3 \end{bmatrix}$$

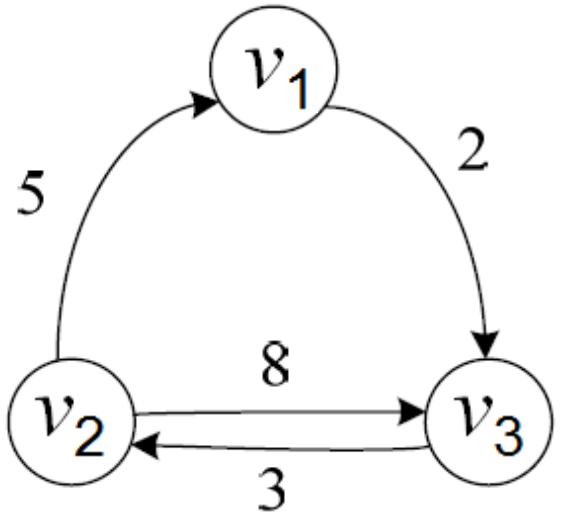
$$\text{adj}^{(1)} = \begin{bmatrix} 0 & \infty & 2 \\ 5 & 0 & 7 \\ \infty & 3 & 0 \end{bmatrix} \qquad \text{path} = \begin{bmatrix} 1 & -1 & 1 \\ 2 & 2 & 1 \\ -1 & 3 & 3 \end{bmatrix}$$

$$\text{adj}^{(2)} = \begin{bmatrix} 0 & \infty & 2 \\ 5 & 0 & 7 \\ 8 & 3 & 0 \end{bmatrix} \qquad \text{path} = \begin{bmatrix} 1 & -1 & 1 \\ 2 & 2 & 1 \\ 2 & 3 & 3 \end{bmatrix}$$

$$\text{adj}^{(3)} = \begin{bmatrix} 0 & 5 & 2 \\ 5 & 0 & 7 \\ 8 & 3 & 0 \end{bmatrix} \qquad \text{path} = \begin{bmatrix} 1 & 3 & 1 \\ 2 & 2 & 1 \\ 2 & 3 & 3 \end{bmatrix}$$

示例

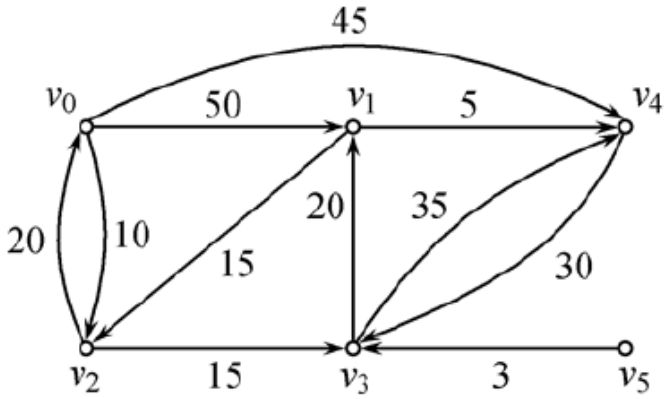
- 例如，想知道v1到v2的最短路径：
  - 由adj[1][2]可知v1到v2的最短路径长度为5
  - 路径由path[1][2]=3可知顶点v2的前一顶点为v3，
  - 由path[1][3]=1可知v3的前一顶点为v1
  - 因此从v1到v2的最短路径为v1→v3→v2



$$\text{adj}^{(3)}=\begin{bmatrix}0 & 5 & 2\\ 5 & 0 & 7\\ 8 & 3 & 0\end{bmatrix} \qquad \text{path}=\begin{bmatrix}\mathbf{1} & \mathbf{3} & \mathbf{1}\\ \mathbf{2} & \mathbf{2} & \mathbf{1}\\ \mathbf{2} & \mathbf{3} & \mathbf{3}\end{bmatrix}$$

Floyd算法—例子

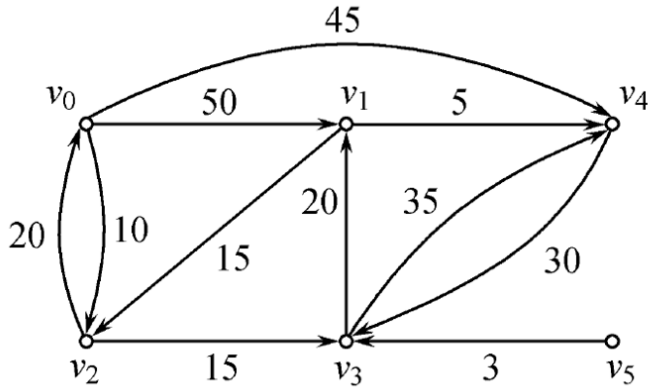
- 初始化



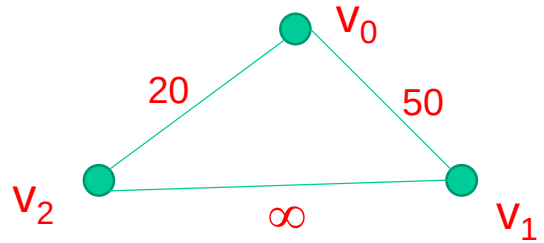
$$A_0 = \begin{bmatrix} 0 & 50 & 10 & \infty & 45 & \infty \\ \infty & 0 & 15 & \infty & 5 & \infty \\ 20 & \infty & 0 & 15 & \infty & \infty \\ \infty & 20 & \infty & 0 & 35 & \infty \\ \infty & \infty & \infty & 30 & 0 & \infty \\ \infty & \infty & \infty & 3 & \infty & 0 \end{bmatrix}$$

$$\text{nextvex}_0 = \begin{bmatrix} 0 & 1 & 2 & -1 & 4 & -1 \\ -1 & 1 & 2 & -1 & 4 & -1 \\ 0 & -1 & 2 & 3 & -1 & -1 \\ -1 & 1 & -1 & 3 & 4 & -1 \\ -1 & -1 & -1 & 3 & 4 & -1 \\ -1 & -1 & -1 & 3 & -1 & 5 \end{bmatrix}$$

Floyd算法—例子

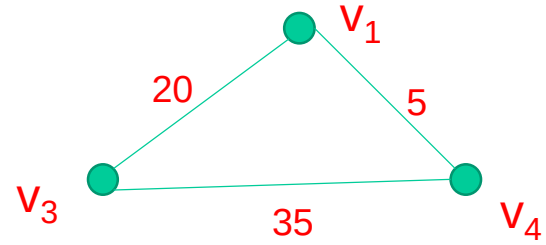


$$A_0 = \begin{bmatrix} 0 & 50 & 10 & \infty & 45 & \infty \\ \infty & 0 & 15 & \infty & 5 & \infty \\ 20 & \infty & 0 & 15 & \infty & \infty \\ \infty & 20 & \infty & 0 & 35 & \infty \\ \infty & \infty & \infty & 30 & 0 & \infty \\ \infty & \infty & \infty & 3 & \infty & 0 \end{bmatrix}$$



加入第一个顶点 v_0，$A_1[i][j]=\min(A_0[i][j], A_0[i][0]+A_0[0][j])$　$0 \leqslant i \leqslant n-1, 0 \leqslant j \leqslant n-1$

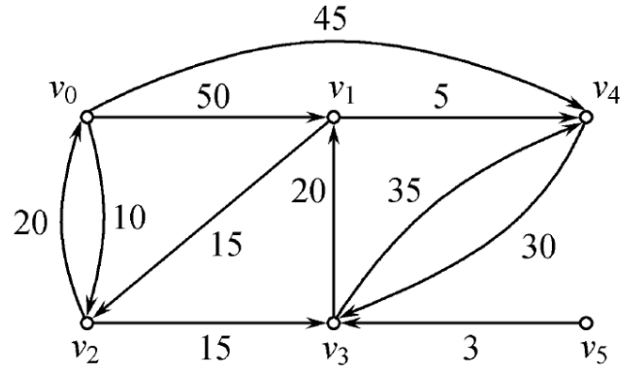
$$A_1 = \begin{bmatrix} 0 & 50 & 10 & \infty & 45 & \infty \\ \infty & 0 & 15 & \infty & 5 & \infty \\ 20 & 70 & 0 & 15 & 65 & \infty \\ \infty & 20 & \infty & 0 & 35 & \infty \\ \infty & \infty & \infty & 30 & 0 & \infty \\ \infty & \infty & \infty & 3 & \infty & 0 \end{bmatrix} \qquad \text{nextvex}_1 = \begin{bmatrix} 0 & 1 & 2 & -1 & 4 & -1 \\ -1 & 1 & 2 & -1 & 4 & -1 \\ 0 & 0 & 2 & 3 & 0 & -1 \\ -1 & 1 & -1 & 3 & 4 & -1 \\ -1 & -1 & -1 & 3 & 4 & -1 \\ -1 & -1 & -1 & 3 & -1 & 5 \end{bmatrix}$$



加入顶点 v_1，$A_2[i][j]=\min(A_1[i][j], A_1[i][1]+A_1[1][j])$　$0 \leqslant i \leqslant n-1, 0 \leqslant j \leqslant n-1$

$$A_2 = \begin{bmatrix} 0 & 50 & 10 & \infty & 45 & \infty \\ \infty & 0 & 15 & \infty & 5 & \infty \\ 20 & 70 & 0 & 15 & 65 & \infty \\ \infty & 20 & 35 & 0 & 25 & \infty \\ \infty & \infty & \infty & 30 & 0 & \infty \\ \infty & \infty & \infty & 3 & \infty & 0 \end{bmatrix} \qquad \text{nextvex}_2 = \begin{bmatrix} 0 & 1 & 2 & -1 & 4 & -1 \\ -1 & 1 & 2 & -1 & 4 & -1 \\ 0 & 0 & 2 & 3 & 0 & -1 \\ -1 & 1 & 1 & 3 & 1 & -1 \\ -1 & -1 & -1 & 3 & 4 & -1 \\ -1 & -1 & -1 & 3 & -1 & 5 \end{bmatrix}$$

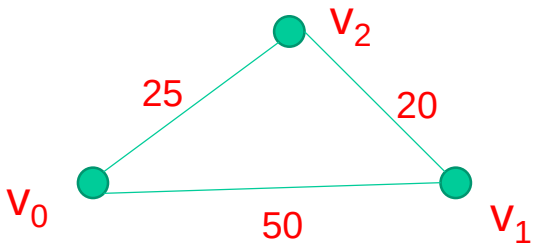
Floyd算法—例子



$$A_2 = \begin{bmatrix} 0 & 50 & 10 & \infty & 45 & \infty \\ \infty & 0 & 15 & \infty & 5 & \infty \\ 20 & 70 & 0 & 15 & 65 & \infty \\ \infty & 20 & 35 & 0 & 25 & \infty \\ \infty & \infty & \infty & 30 & 0 & \infty \\ \infty & \infty & \infty & 3 & \infty & 0 \end{bmatrix}$$

加入顶点 v_2，$A_3[i][j]=\min(A_2[i][j], A_2[i][2]+A_2[2][j])$　$0 \leqslant i \leqslant n-1, 0 \leqslant j \leqslant n-1$

$$A_3 = \begin{bmatrix} 0 & 50 & 10 & 25 & 45 & \infty \\ 35 & 0 & 15 & 30 & 5 & \infty \\ 20 & 70 & 0 & 15 & 65 & \infty \\ 55 & 20 & 35 & 0 & 25 & \infty \\ \infty & \infty & \infty & 30 & 0 & \infty \\ \infty & \infty & \infty & 3 & \infty & 0 \end{bmatrix} \qquad \text{nextvex}_3 = \begin{bmatrix} 0 & 1 & 2 & 2 & 4 & -1 \\ 2 & 1 & 2 & 2 & 4 & -1 \\ 0 & 0 & 2 & 3 & 0 & -1 \\ 1 & 1 & 1 & 3 & 1 & -1 \\ -1 & -1 & -1 & 3 & 4 & -1 \\ -1 & -1 & -1 & 3 & -1 & 5 \end{bmatrix}$$



加入顶点 v_3，$A_4[i][j]=\min(A_3[i][j], A_3[i][3]+A_3[3][j])$　$0 \leqslant i \leqslant n-1, 0 \leqslant j \leqslant n-1$

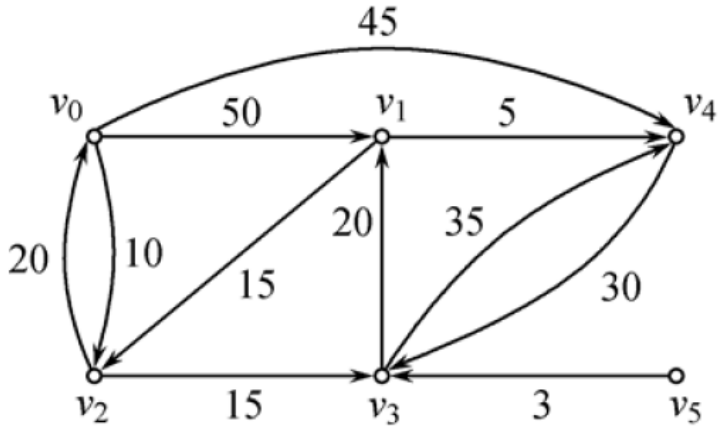
$$A_4 = \begin{bmatrix} 0 & 45 & 10 & 25 & 45 & \infty \\ 35 & 0 & 15 & 30 & 5 & \infty \\ 20 & 35 & 0 & 15 & 40 & \infty \\ 55 & 20 & 35 & 0 & 25 & \infty \\ 85 & 50 & 65 & 30 & 0 & \infty \\ 58 & 23 & 38 & 3 & 28 & 0 \end{bmatrix} \qquad \text{nextvex}_4 = \begin{bmatrix} 0 & 2 & 2 & 2 & 4 & -1 \\ 2 & 1 & 2 & 2 & 4 & -1 \\ 0 & 3 & 2 & 3 & 3 & -1 \\ 1 & 1 & 1 & 3 & 1 & -1 \\ 3 & 3 & 3 & 3 & 4 & -1 \\ 3 & 3 & 3 & 3 & 3 & 5 \end{bmatrix}$$

A_5、A_6与 A_4相同，nextvex_5、nextvex_6与 nextvex_4相同。

Floyd算法—例子

$$A_4 = \begin{bmatrix} 0 & 45 & 10 & 25 & 45 & \infty \\ 35 & 0 & 15 & 30 & 5 & \infty \\ 20 & 35 & 0 & 15 & 40 & \infty \\ 55 & 20 & 35 & 0 & 25 & \infty \\ 85 & 50 & 65 & 30 & 0 & \infty \\ 58 & 23 & 38 & 3 & 28 & 0 \end{bmatrix}$$

$$\text{nextvex}_4 = \begin{bmatrix} 0 & 2 & 2 & 2 & 4 & -1 \\ 2 & 1 & 2 & 2 & 4 & -1 \\ 0 & 3 & 2 & 3 & 3 & -1 \\ 1 & 1 & 1 & 3 & 1 & -1 \\ 3 & 3 & 3 & 3 & 4 & -1 \\ 3 & 3 & 3 & 3 & 3 & 5 \end{bmatrix}$$



- 求v0到v1的最短路径：
- 由A[0][1]可知v0到v1的最短路径长度为45，
- 最短路径为v0→v2→v3→v1
  - 由nextvex[0][1]=2可知顶点v0的下一顶点为v2，
  - 由nextvex[2][1]=3可知v2的下一顶点为v3，
  - 由nextvex[3][1]=1可知v3的下一顶点为v1，

时间复杂度为$O(n^3)$，算法简单，容易理解。

Floyd算法实现

```
void Floyd(Graph& G, Dist** &D){
    int i,j,v;                                          //i, j, v作为计数器
    D=new Dist*[G.VerticesNum()];
    for(i=0; ;i<G.VerticesNum();i++)                    //申请数据D的空间
            D[i]=new Dist[G.VerticesNum()];
    for(i=0;i<G.VerticesNum();i++)                      //初始化D数组
        for(j=0;j<G.VerticesNum();j++)
            if(i==j){
                D[i][j].length=0;                       //权值矩阵
                D[i][j].pre=i;                          //path矩阵
            }else {
                D[i][j]=INFINITY;
                D[i][j].pre=-1;
            }
}
```

初始化

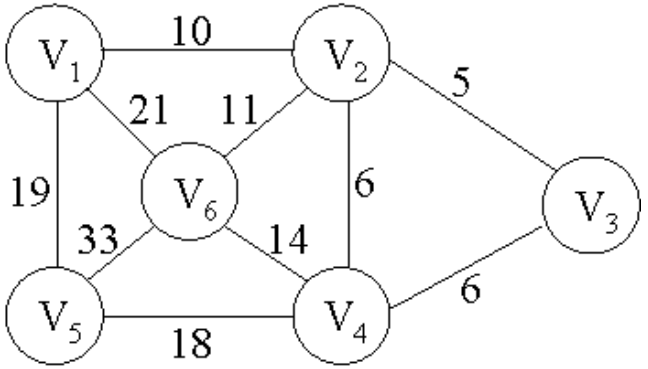
```
for(v=0;v<G.VerticesNum();v++)          //矩阵初始化，仅初始化邻接顶点
    for(Edge e=G.FirstEdge(v); G.IsEdge(e); e=G.NextEdge(e)){
        D[v][G.ToVertex(e)].length=G.Weight(e);
        D[v][G.ToVertex(e)].pre=v;
    }
//如果两顶点间最短路径经过顶点v，则有权值进行更新！
for(v=0;v<G.VerticesNum();v++)
    for(i=0;i<G.VerticesNum();i++)
        for(j=0;j<G.VerticesNum();j++)
            if((D[i][v].length +D[v][j].length) < D[i][j].length){
                D[i][j].length =D[i][v].length +D[v][j].length;
                D[i][j].pre=D[v][j].pre;
            }
}
```

权值 路径更新

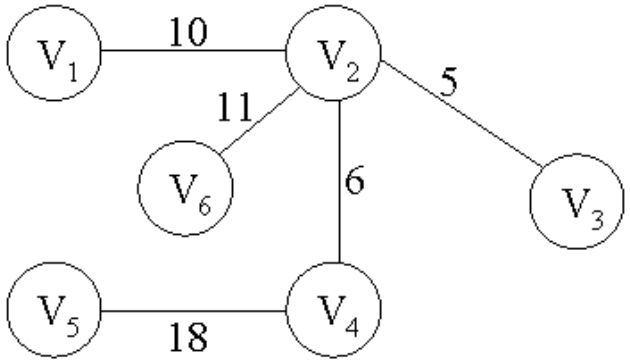
- 时间复杂性：三重for循环，复杂度是O(n^3)，适合稠密图

7.6 最小支撑(生成)树

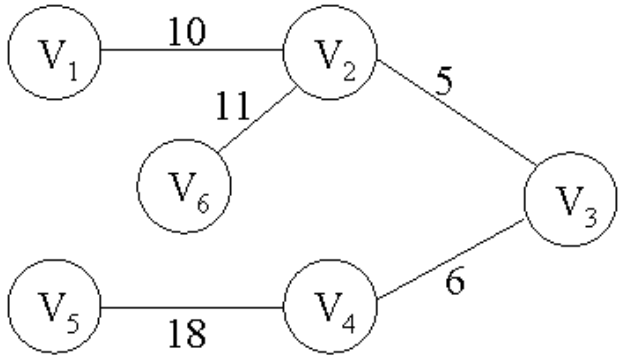
- 公路网的造价问题



(a)交通网络



(b) 最小支撑树



(c) 最小支撑树

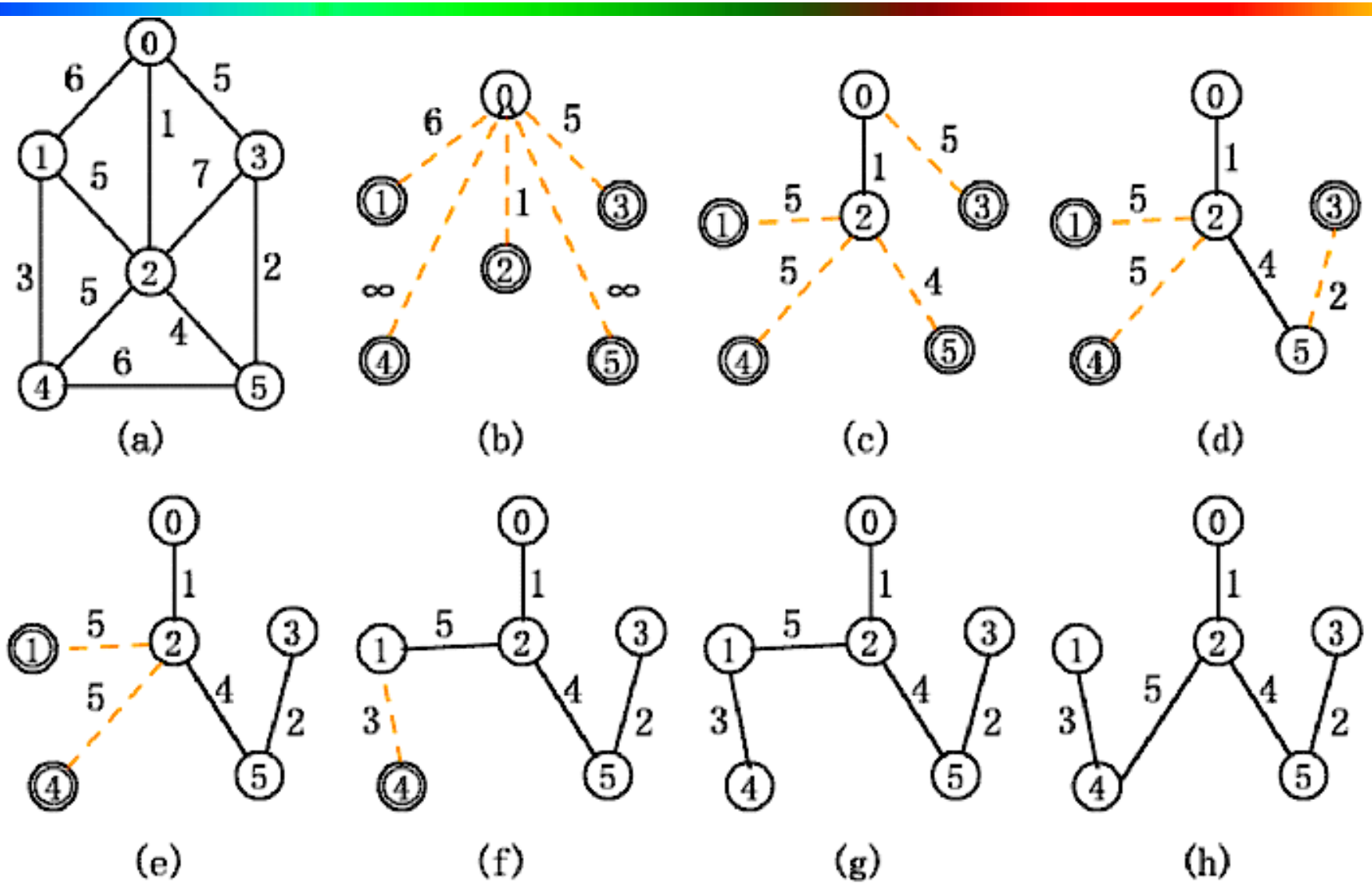
最小支撑树

- 最小支撑树（Minimum-cost Spanning Tree，MST）
  - 对于带权的连通无向图G，其最小支撑树是一个包括G的所有顶点和部分边的图，这部分的边满足下列条件：
    - 保证图的连通性
    - 边权值总和最小
- 代表算法
  - Prim算法
  - Kruskal算法

1、Prim算法

- 具体操作
  - 从图中任意一个顶点开始，把这个顶点包括在MST里
  - 然后，在那些其一个端点已在MST里，另一个端点还未在MST里的边中，找权最小的一条边（相同边存在，任选择其一），并把这条边和其不在MST里的那个端点包括进MST里
  - 如此进行下去，每次往MST里加一个顶点和一条权最小的边，直到把所有的顶点都包括进MST里
- MST不唯一，但是最小权值是确定的

举例

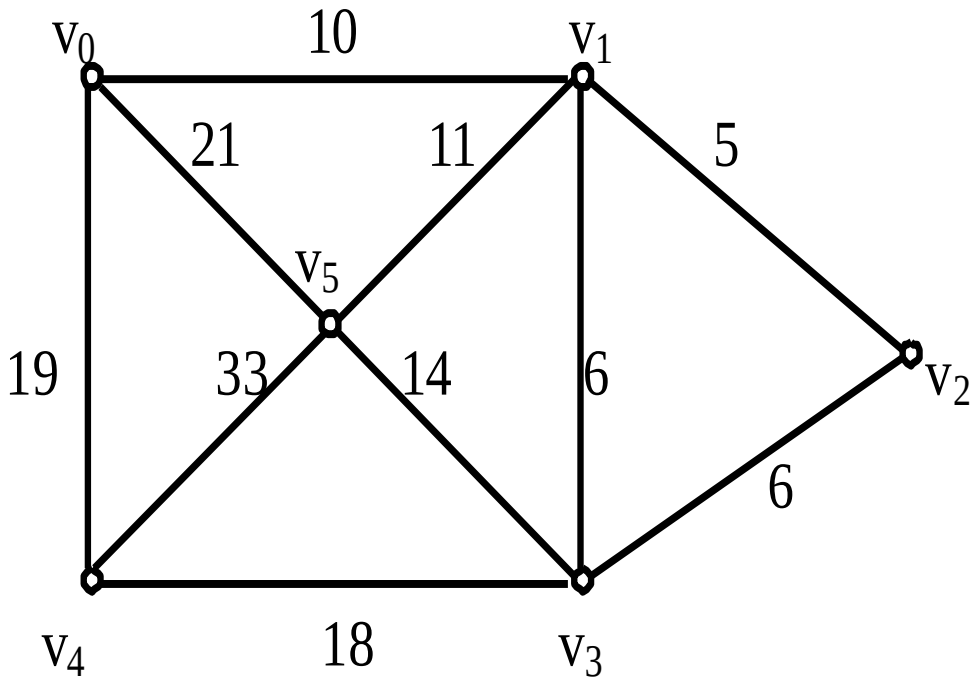


Prim算法构造最小生成树的过程

例子

已知带权图G及其邻接矩阵如图所示

请构造该图的最小生成树



$$\begin{bmatrix} 0 & 10 & \infty & \infty & 19 & 21 \\ 10 & 0 & 5 & 6 & \infty & 11 \\ \infty & 5 & 0 & 6 & \infty & \infty \\ \infty & 6 & 6 & 0 & 18 & 14 \\ 19 & \infty & \infty & 18 & 0 & 33 \\ 21 & 11 & \infty & 14 & 33 & 0 \end{bmatrix}$$

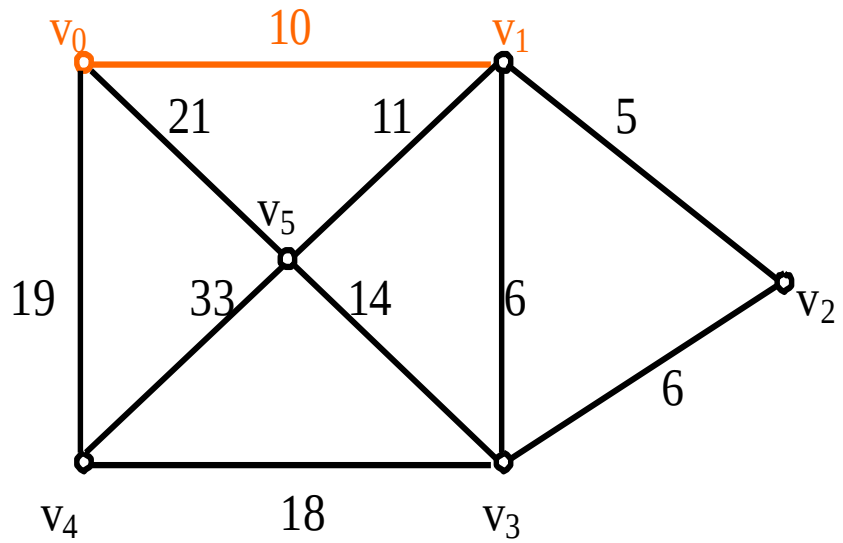
- n=6，只有顶点v_0在最小生成树中。

  mst[5]={{0,1,10}, {0,2,∞}, {0,3,∞}, {0,4,19}, {0,5,21} }

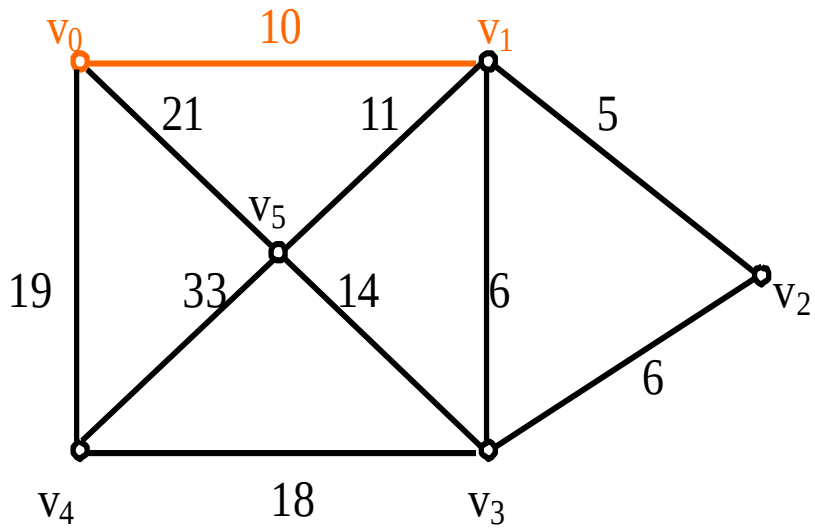
  U= {V_0}

- 在mst[0]到mst[4]中找出权值最小的边mst[0]，即(v0, v1)，将顶点v1及边(v0, v1)加入最小生成树。

  U= {V_0 , V_1}



mst[5]={{0,1,10}, {0,2,∞}, {0,3,∞}, {0,4,19}, {0,5,21} }



- **调整mst[1]到mst[4]**

  (v_1, v_2)=5 小于(v_0, v_2) {0,2,∞} 调整

  (v_1, v_3)=6 小于(v_0, v_3) {0,3,∞} 调整

  (v_1, v_4)=∞ 大于(v_0, v_4){0,4,19} 不需要调整

  (v_1, v_5)=11 小于(v_0, v_5){0,5,21} 调整

  mst[5]={{0,1,10}, {1,2,5}, {1,3,6}, {0,4,19}, {1,5,11} }

mst[5]={{0,1,10}, {1,2,5}, {1,3,6}, {0,4,19}, {1,5,11} }

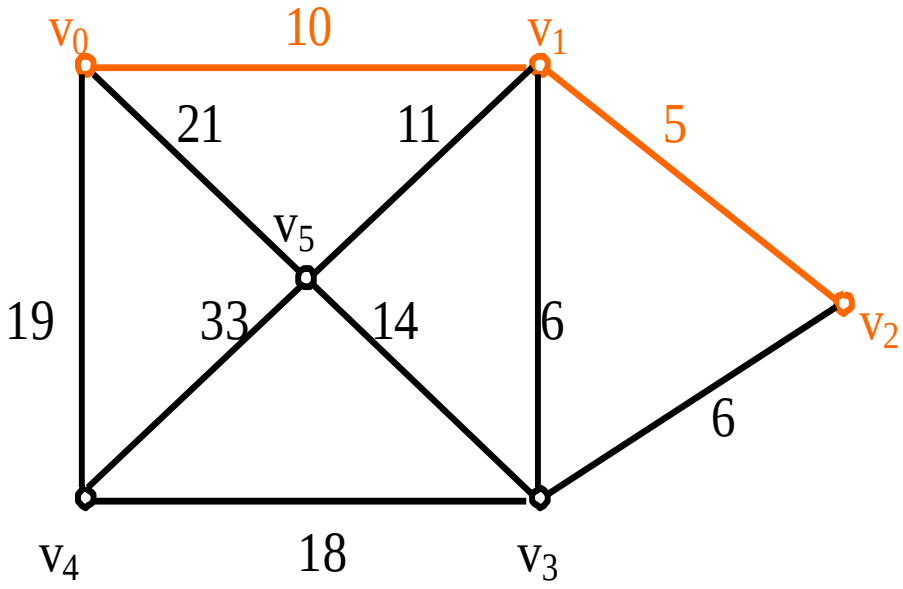
U= {V_0 , V_1, V_2}

- 在mst[1]到mst[4]中找出权值最小的边mst[1]，即(v_1, v_2)，将顶点v_2及边(v_1, v_2)加入最小生成树
- 调整mst[2]到mst[4]

(v_2, v_3)=6 不小于(v_1, v_3)　　{ 1,3,6}　　不需要调整

(v_2, v_4)=∞ 大于(v_0, v_4)　　{0,4,19}　　不需要调整

(v_2, v_5)=∞ 大于(v_1, v_5)　　{1,5,11}　　不需要调整

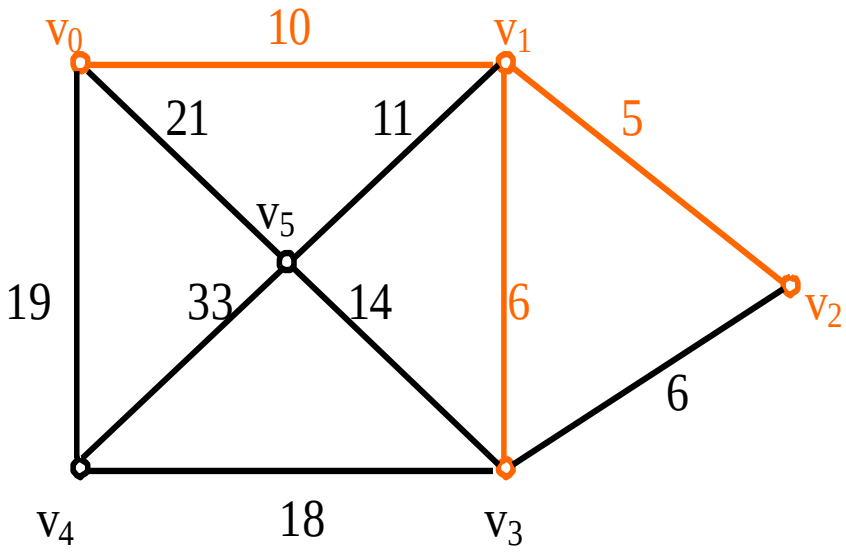


mst[5]={{0,1,10}, {1,2,5}, {1,3,6}, {0,4,19}, {1,5,11} }

mst[5]={{0,1,10}, {1,2,5}, {1,3,6}, {0,4,19}, {1,5,11} }

- 在mst[2]到mst[4]中找出权值最小的边mst[2]，即(v_1, v_3)，将顶点v_3及边(v_1, v_3)加入最小生成树

U= {V_0 , V_1, V_2 , V_3}



- 调整mst[3]到mst[4]

(v_3, v_4)=18 小于(v_0, v_4)　{0,4,19}　调整

(v_3, v_5)=14 大于(v_1, v_5)　{1,5,11}　不需要调整

mst[5]={{0,1,10}, {1,2,5}, {1,3,6}, {3,4,18}, {1,5,11} }

mst[5]={{0,1,10}, {1,2,5}, {1,3,6}, {3,4,18}, {1,5,11} }

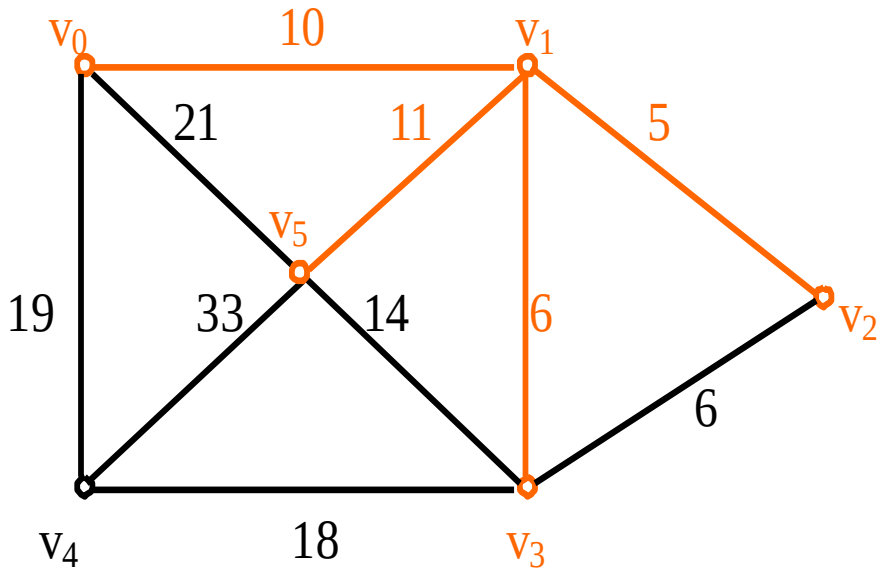
- 在mst[3]到mst[4]中找出权值最小的边mst[4]，即(v_1, v_5)，
  将顶点v_5及边(v_1, v_5)加入最小生成树。互换mst[3]和mst[4]

mst[5]={{0,1,10}, {1,2,5}, {1,3,6}, {1,5,11} , {3,4,18}}

- 调整mst[4]

  (v_5, v_4)=33 大于(v_3, v_4)　{3,4,18} 不需要调整

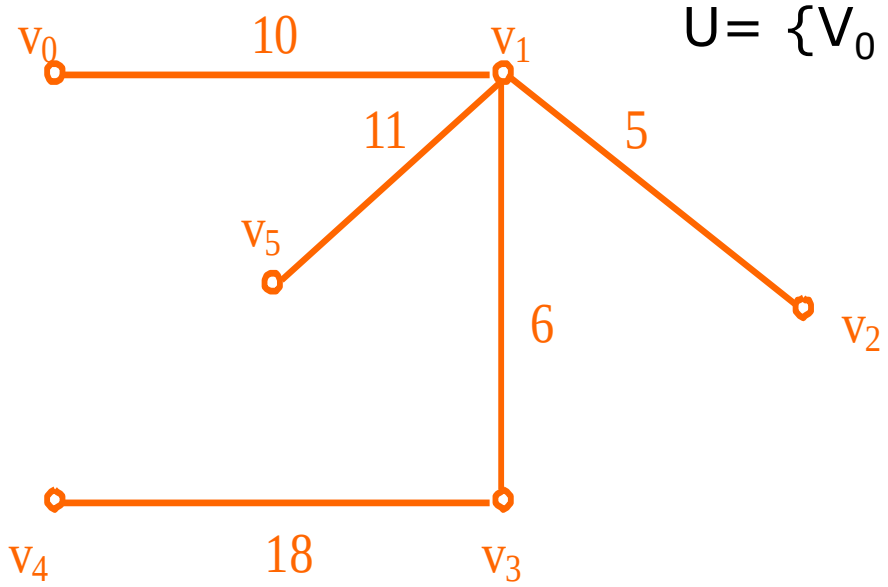
  mst[5]={{0,1,10}, {1,2,5}, {1,3,6}, {1,5,11} , {3,4,18}}



U= {V_0 , V_1, V_2 , V_3, , V_5}

- 将mst[4]加入最小生成树。

- 最小生成树如图所示



U= {V_0 , V_1, V_2 , V_3, , V_5 , V_4}

Prim算法实现

```
void Prim(Graph& G, int s, Edge* &MST ) {
    int MSTtag=0;                                   //最小支撑树边的标号
    Edge *MST=new Edge[G.VerticesNum()-1];
    MinHeap<Edge> H(G.EdgesNum());
    for(int i=0;i<G.VerticesNum();i++)   //初始化Mark数组、距离数组
        G.Mark[i]=UNVISITED;
    int v=s;                                        //开始顶点
    G.Mark[v]=VISITED;                              //开始顶点首先被标记
    do{   //将以v为顶点，另一顶点未被标记的边插入最小值堆H
        for(Edge e= G. FirstEdge(v);G.IsEdge(e);e=G. NextEdge(e))
            if(G. Mark[G. ToVertex(e)]==UNVISITED)
                H.Insert(e);
        bool Found=false;
        while(!H.empty()) {                         //寻找下一条权最小的边
            e=H.RemoveMin();
```

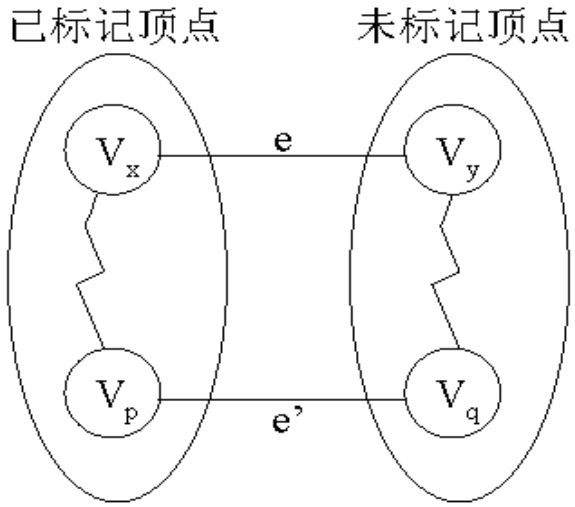
```
                if(G. Mark[G. ToVertex(e)]==UNVISITED){
                        Found=true;
                         break;
                }
            } //end while
            if(!Found){
                Print("不存在最小支撑树。");
                 delete [] MST;                          //释放空间
                 MST=NULL;                               //MST是空数组
                return;
            }
            v= G. ToVertex(e);
            G.Mark[v]=VISITED;       //在顶点v的标志位上做已访问的标记
            AddEdgetoMST(e,MST,MSTtag++); //将边e加到MST中
        }  while(MSTtag < (G. VerticesNum()-1))
    }
```

证明

- 引理：用Prim算法构造的生成树是MST
- 首先证明这样一个结论：
  - 设T(V*，E*)是连通无向图G=(V，E)的一棵正在构造的生成树，又E中有边e=(V_x，V_y)，其中$V_x \in V^*$，V_y不属于V*，且e的权W(e)是所有一个端点在V*里，另一端不在V*里的边的权中最小者，则一定存在G的一棵包括T的MST包括边e=(V_x，V_y)。

反证法

1. 设G中任何包括T的MST都不包括e=(V_x，V_y)，且设T’是一棵这样的MST
2. 由于T’是连通的，因此有从V_x到V_y的路径V_x，…，V_y
3. 把边e=(V_x，V_y)加进树T’，得到一个回路V_x，…，V_y，V_x
4. 上述路径V_x，…，V_y中必有边e’=(V_p，V_q)，其中$V_p \in V^*$，V_q不属于V*，由条件知边的权W(e’)≥W(e)，从回路中去掉边e’，回路打开，成为另一棵生成树T”，T”包括边e=(V_x，V_y)，且各边权的总和不大于T’各边权总和
5. 因此T”是一棵包括边e的MST，与假设矛盾，即证明了我们的结论

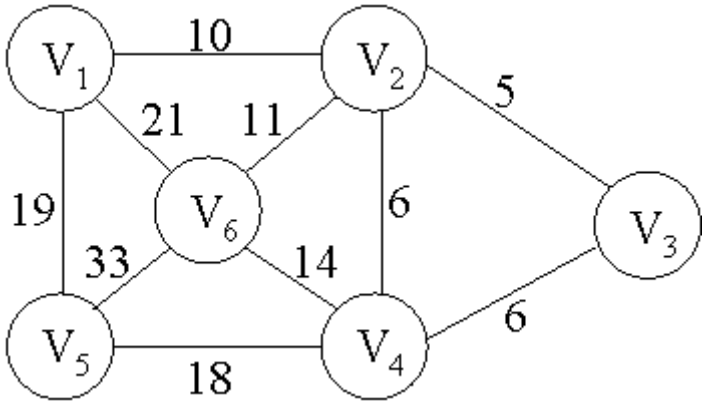


Prim算法与Dijkstra算法的区别

- 相同点：都是贪心的思路
- 不同点
  - Prim算法要寻找的是离已加入顶点距离最近的点
  - 而Dijkstra是寻找离源点距离最近的点
- 其时间复杂度分析与Dijkstra算法相同

2、Kruskal算法

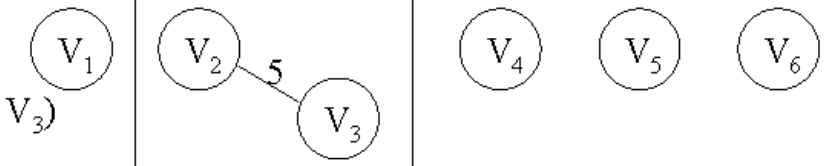
- Kruskal算法的基本思想
  - 对于图G=（V，E），开始时，将顶点集分为|V|个等价类，每个等价类包括一个顶点
  - 然后，以权的大小为顺序处理各条边，如果某条边连接两个不同等价类的顶点，则这条边被添加到MST，两个等价类被合并为一个；
  - 反复执行此过程，直到只剩下一个等价类



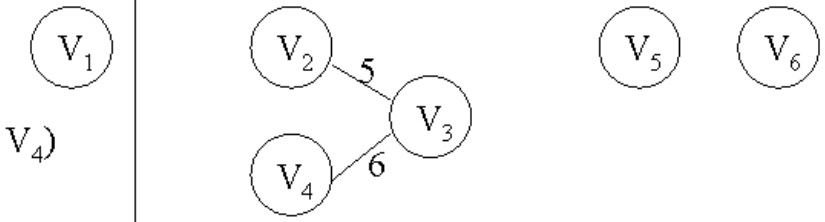
初始状态：



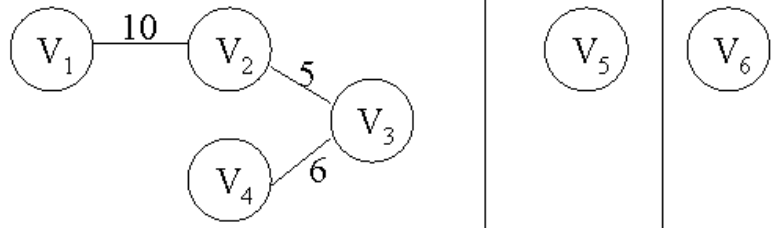
步骤1：
处理边(V_2, V_3)



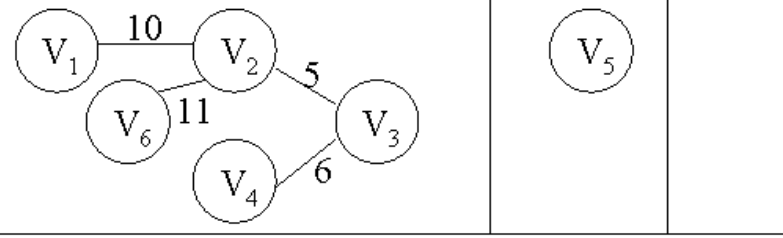
步骤2：
处理边(V_3, V_4)



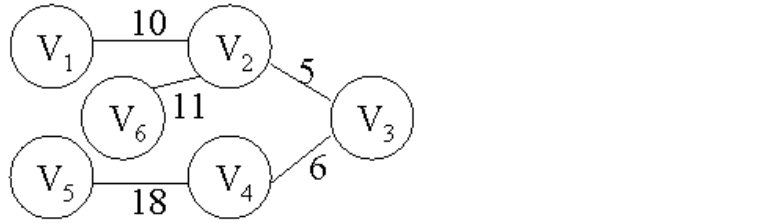
步骤3：
处理边(V_1, V_2)



步骤4：
处理边(V_2, V_6)

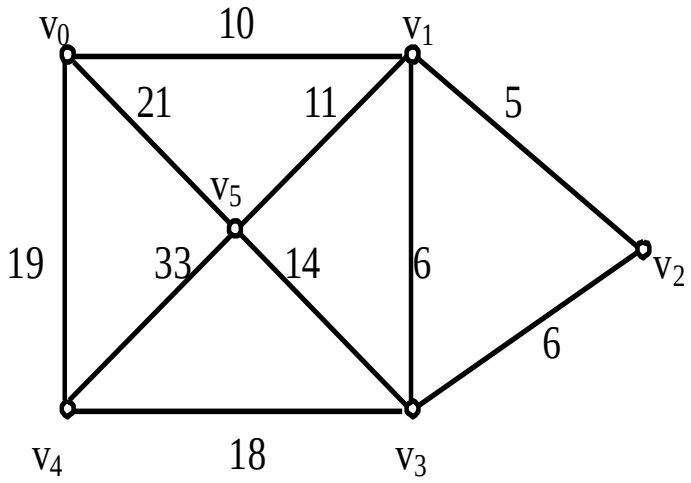


步骤5：
处理边(V_4, V_5)



Kruskal算法 - 例子

- 用Kruskal方法构造下图的最小生成树



$$\begin{bmatrix} 0 & 10 & \infty & \infty & 19 & 21 \\ 10 & 0 & 5 & 6 & \infty & 11 \\ \infty & 5 & 0 & 6 & \infty & \infty \\ \infty & 6 & 6 & 0 & 18 & 14 \\ 19 & \infty & \infty & 18 & 0 & 33 \\ 21 & 11 & \infty & 14 & 33 & 0 \end{bmatrix}$$

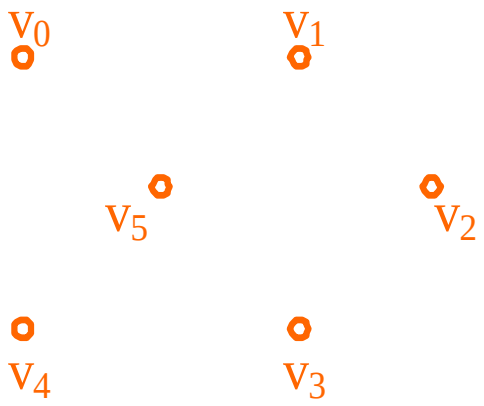
- 集合E中的边按递增顺序排列为：

$(v_1, v_2), (v_1, v_3), (v_2, v_3), (v_0, v_1), (v_1, v_5), (v_3, v_5), (v_3, v_4), (v_0, v_4), (v_0, v_5), (v_4, v_5)$

Kruskal算法 - 例子

(v1, v2), (v1, v3), (v2, v3), (v0, v1), (v1, v5),
(v3, v5), (v3, v4), (v0, v4), (v0, v5), (v4, v5)

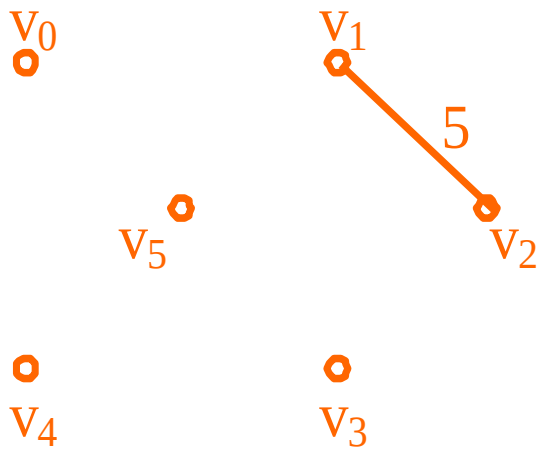
①．初始时，T为只有6个顶点的非连通图

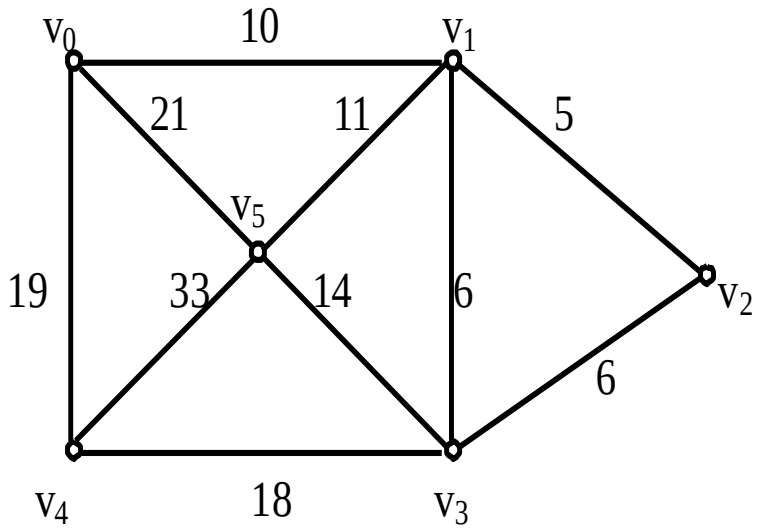


Kruskal算法 - 例子

(v1, v2), (v1, v3), (v2, v3), (v0, v1), (v1, v5),
(v3, v5), (v3, v4), (v0, v4), (v0, v5), (v4, v5)

②．边(v_1, v_2)的两个顶点v_1，v_2分别属于两个连通分量，将边(v_1, v_2)加入T

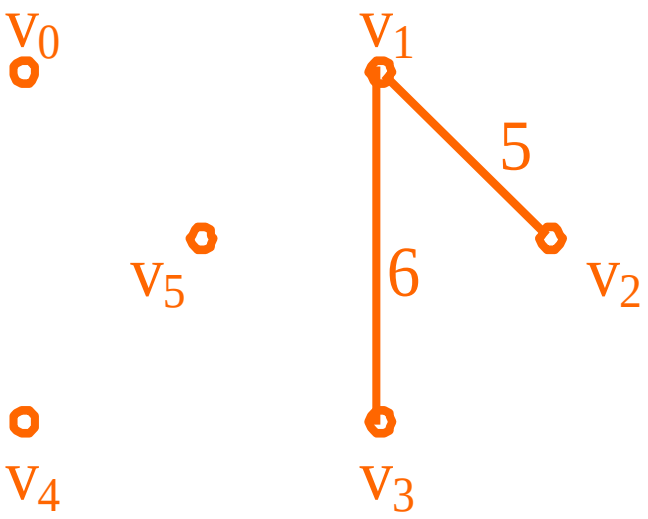


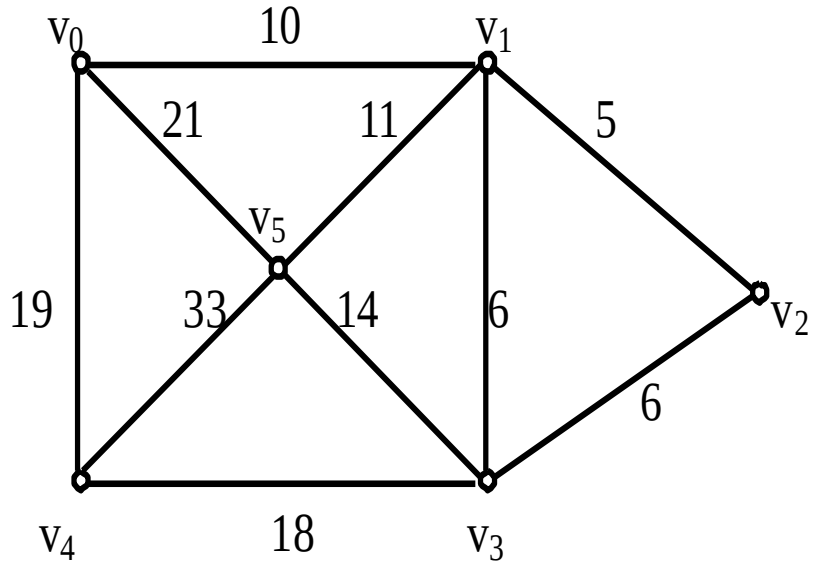


Kruskal算法 - 例子

(v1, v2), (v1, v3), (v2, v3), (v0, v1), (v1, v5),
(v3, v5), (v3, v4), (v0, v4), (v0, v5), (v4, v5)

③. 同理，将边(v_1, v_3)加入T

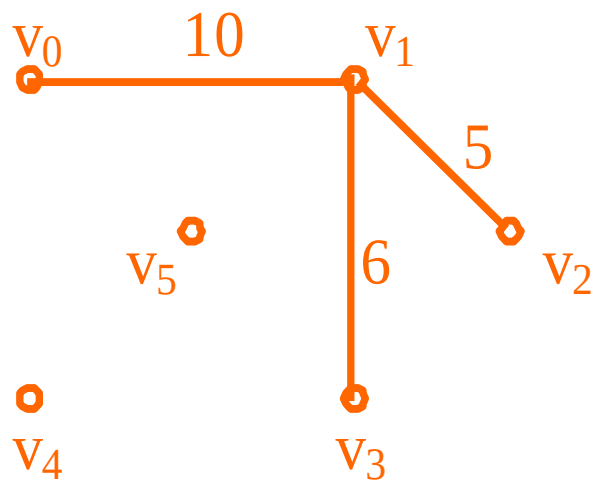


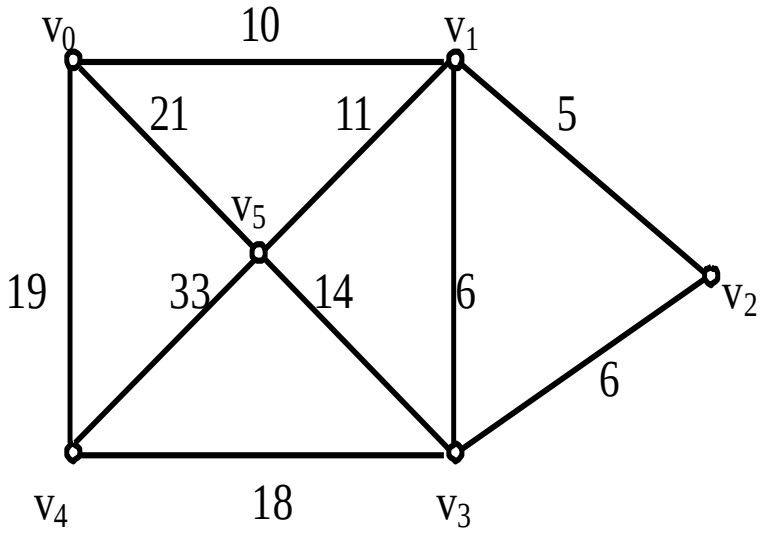


Kruskal算法 - 例子

(v1, v2), (v1, v3), (v2, v3), (v0, v1), (v1, v5),
(v3, v5), (v3, v4), (v0, v4), (v0, v5), (v4, v5)

④ 由于边(v_2, v_3)的两个顶点v_2，v_3属于同一个连通分量，因此，舍去这条边，将 (v_0, v_1)加入



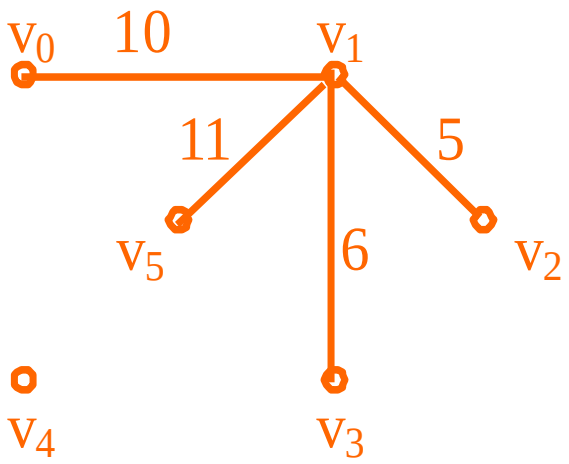


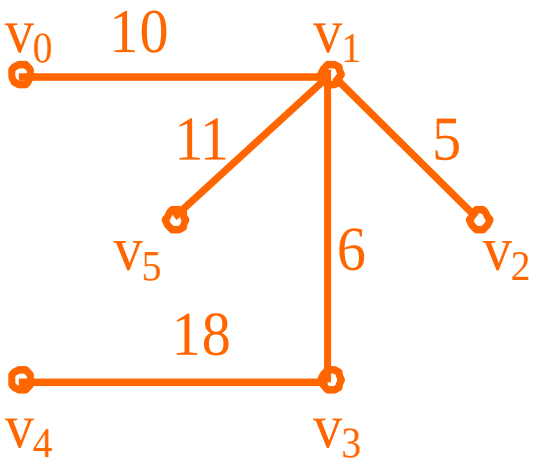
Kruskal算法 - 例子

(v1, v2), (v1, v3), (v2, v3), (v0, v1), (v1, v5),
(v3, v5), (v3, v4), (v0, v4), (v0, v5), (v4, v5)

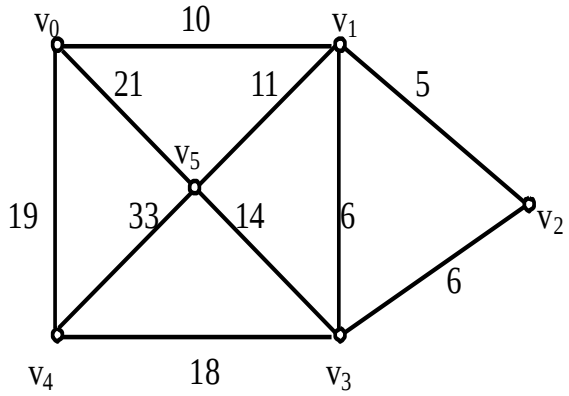
⑤同理将边 (v_1, v_5)加入T，边(v_3, v_5)舍去，边(v_3, v_4)加入T

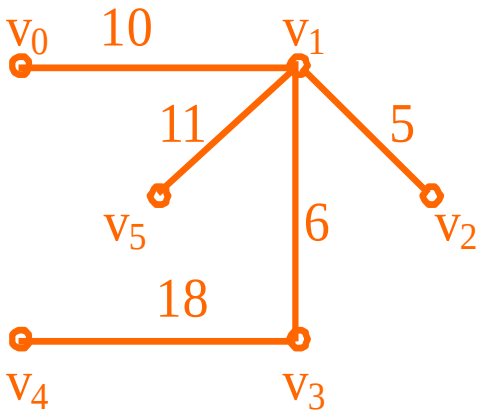
这时T中含的边数为5条，成为一个连通分量，T就是G的一棵最小生成树

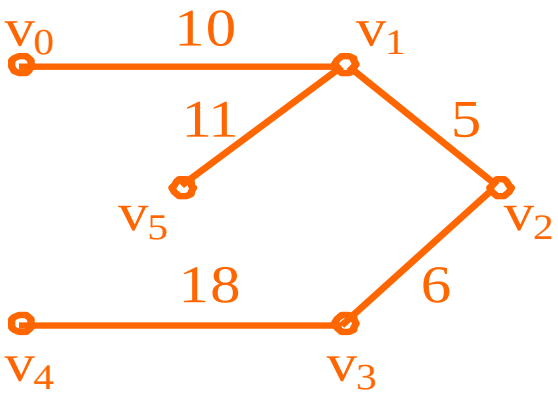




图的最小生成树并不一定是唯一的，例如，上图中边(v_1, v_3)和(v_2, v_3)的长度相同，该图的另一棵最小生成树如下图所示







Kruskal算法描述

```
void Kruskal(Graph& G, Edge* &MST{
    Partree A(G.VerticesNum());                          //等价类
    MinHeap<Edge> H(G.EdgesNum());                       //声明一个最小堆
    MST=new Edge[G.VerticesNum()-1];          //最小支撑树
    int MSTtag=0;                                  //最小支撑树边的标号
    for(int i=0; i<G.VerticesNum(); i++) {   //将图的所有边插入最小值堆H中
        for(Edge e= G.FirstEdge(i); G.IsEdge(e);e=G. NextEdge(e))
            if(G.FromVertex(e)< G.ToVertex(e))
                H.Insert(e);
    int EquNum=G.VerticesNum();        //开始时有|V|个等价类
    while(EquNum>1) {                          //合并等价类
        Edge e=H.RemoveMin();          //获得下一条权最小的边
```

边集入堆

```
            int from=G.FromVertex(e);          //记录该条边的信息
            int to= G.ToVertex(e);
            if(A.differ(from,to)) {      //如果边e的两个顶点不在一个等价类
                  //将边e的两个顶点所在的两个等价类合并为一个
                  A.UNION(from,to);
                  AddEdgetoMST(e,MST,MSTtag++); //将边e加到MST
                  EquNum--;                              //将等价类的个数减1
            }
      }//end while
}
```

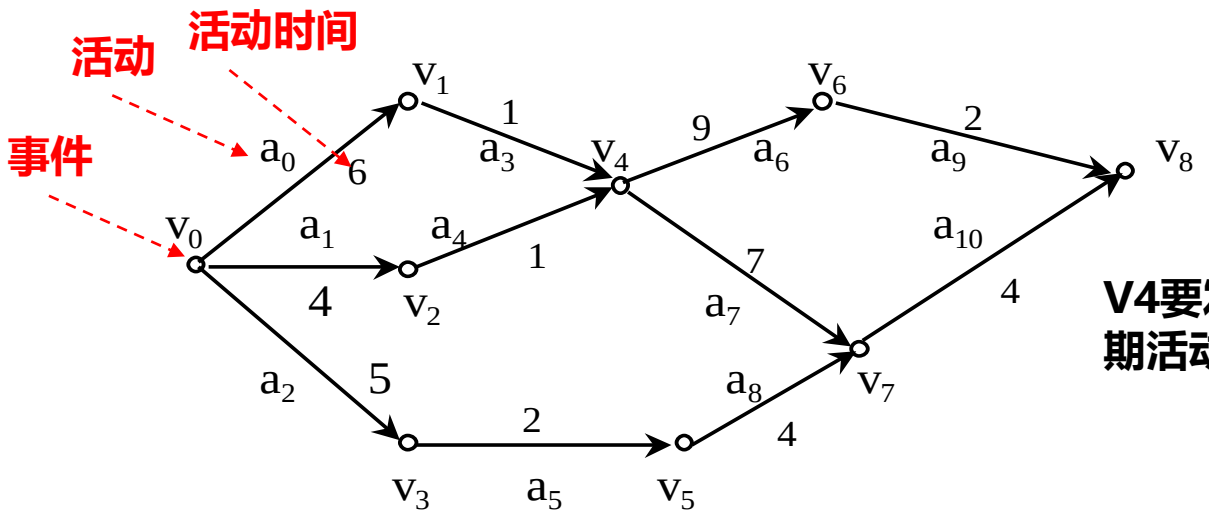
性能分析

- 使用了路径压缩，differ和UNION函数几乎是常数
- 假设可能对几乎所有边都判断过了，则最坏情况下算法时间代价为 O(|E|log |E|)，即堆排序的时间
- 适合于稀疏图

*关键路径 与 AOE网

- 如果在带权的有向图中，用顶点表示事件，用有向边表示活动，边上的权值表示活动持续的时间，则此带权的有向图称为边活动网（Activity On Edge network，简称AOE网）。

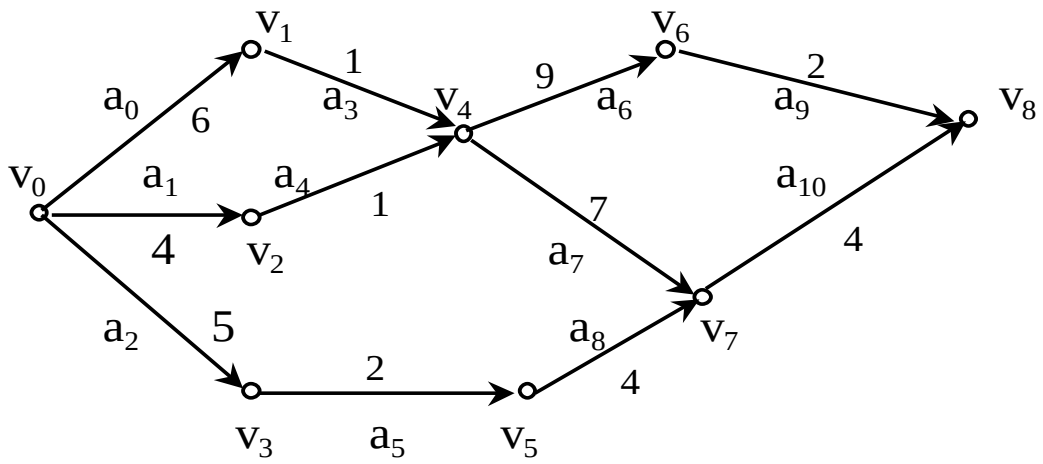
- 顶点所表示的事件就是它的入边所表示的活动都已完成，它的出边所表示的活动可以开始这样一种状态。

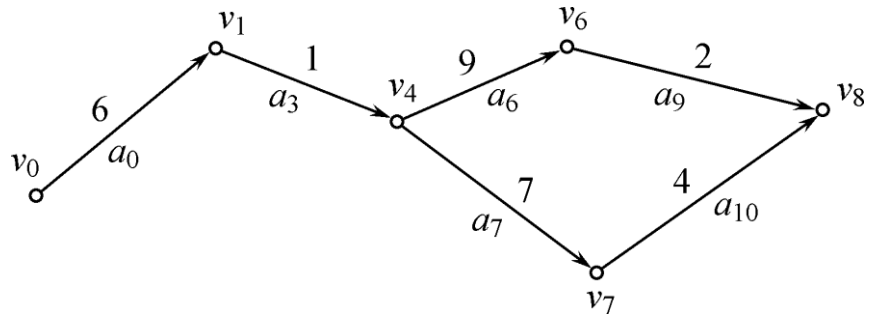


V4要发生，则他对应的两个前期活动a3 a4都要结束才可以

关键路径

- AOE网中有些活动可以并行进行，所以完成整个工程的**最短时间是从开始顶点到完成顶点的最长路径长度**，路径长度为路径上各边的权值之和。
- 把开始顶点到完成顶点的**最长路径**称为关键路径。
  - 例如，路径v0, v1, v4, v7, v8是一条关键路径，长度为18，也就是整个工程至少要18天才能完成。
  - AOE网的关键路径可能不止一条





关键活动

- 哪些活动将影响工程的进度呢?
  - 关键路径上的活动如果延期，都将导致整项工程不能按时完成
  - 要提前完成工程，需要在某些关键活动上投入更多的人力物力

- 关键路径上的活动为关键活动
- 分析关键路径的目的是确定关键活动

几个概念

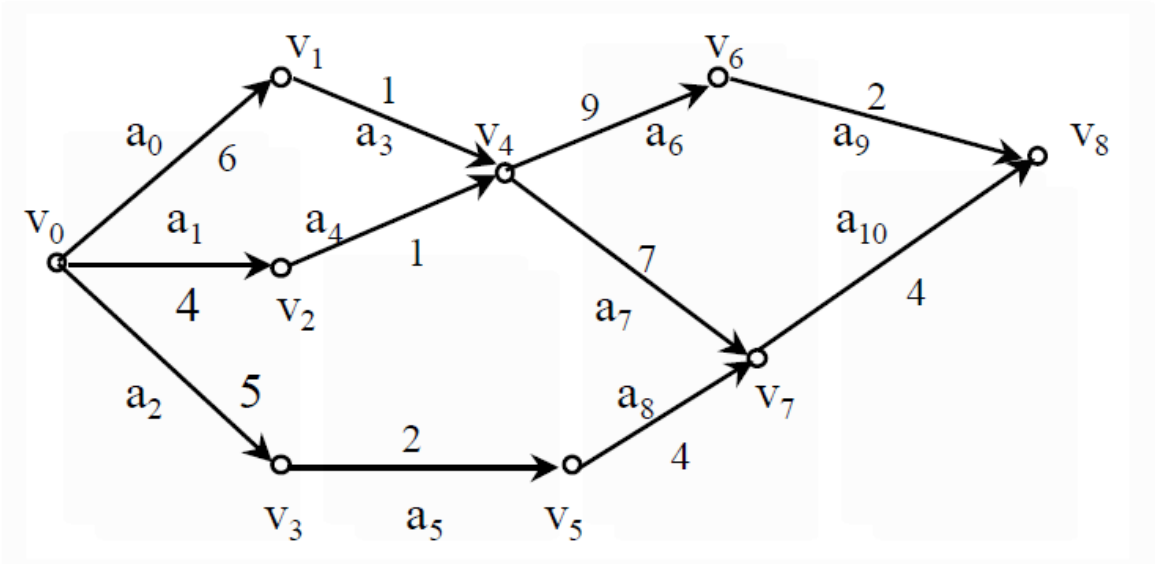
- **关键活动**
  - 事件v_j可能的最早发生时间ee(j)
  - 事件v_i允许的最迟发生时间le(i)
  - 活动a_k=< v_i,v_j>的最早开始时间e(k)
  - 活动a_k=< v_i,v_j>的最晚开始时间l(k)

- e(k)=l(k)的那些活动均为关键活动

- l(k)-e(k)表示完成活动a_k的时间余量，是在不延误工期的前提下，活动a_k可以延迟的时间。

关键路径的求解示例

- 分别求出事件的最早发生时间和最迟发生时间、活动的最早开始时间和最晚开始时间



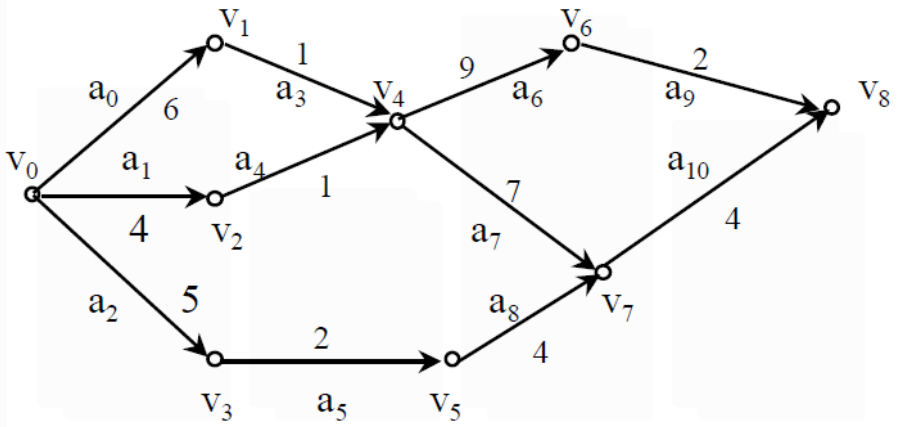
某事件可能的最早发生时间

- 从开始顶点到顶点v_j（事件v_j）的**最长路径长度**
  - 所有进入v_j的活动$< v_i, v_j >$都结束时，v_j所代表的事件**才可立即发生（所以是最早发生时间）**，即

$$ee(0) = 0$$

$$ee(j) = \max\{ee(i) + weight(< v_i, v_j >)\}$$

$$< v_i, v_j > \in T, 1 \le j \le n-1$$



其中T为所有以v_j为终点的入边的集合

关键路径的求解示例

ee(0)=0

ee(1)=ee(0)+weight(<v_0,v_1>)=0+6=6

ee(2)=ee(0)+ weight(<v_0,v_2>)=0+4=4

ee(3)=ee(0)+ weight(<v_0,v_3>)=0+5=5

ee(4)=max{ee(1)+ weight(<v_1,v_4>), ee(2)+ weight(<v_2,v_4>)

=max{6+1, 4+1}=7

ee(5)=ee(3)+ weight(<v_3,v_5>)=5+2=7

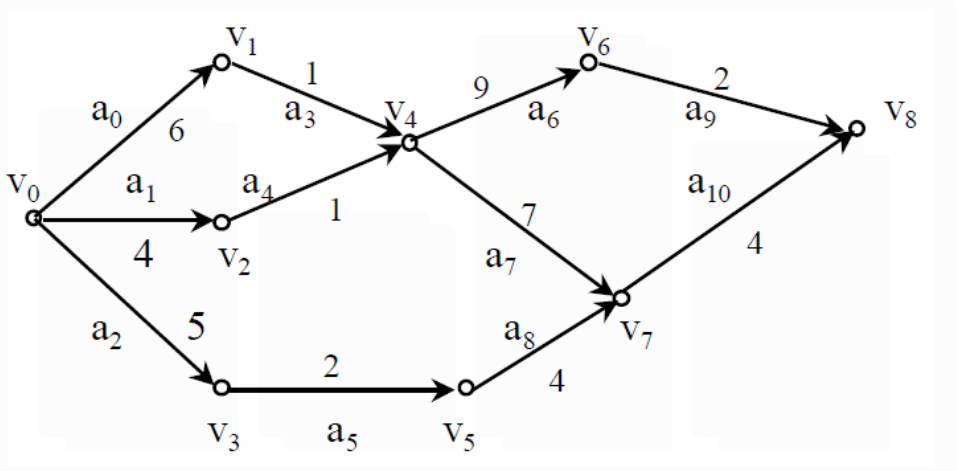
ee(6)=ee(4)+ weight(<v_4,v_6>)=7+9=16

ee(7)=max{ee(4)+ weight(<v_4,v_7>), ee(5)+ weight(<v_5,v_7>)

=max{7+7, 7+4}=14

ee(8)=max{ee(6)+ weight(<v_6,v_8>), ee(7)+ weight(<v_7,v_8>)

=max{16+2, 14+4}=18



某事件允许的最晚发生时间

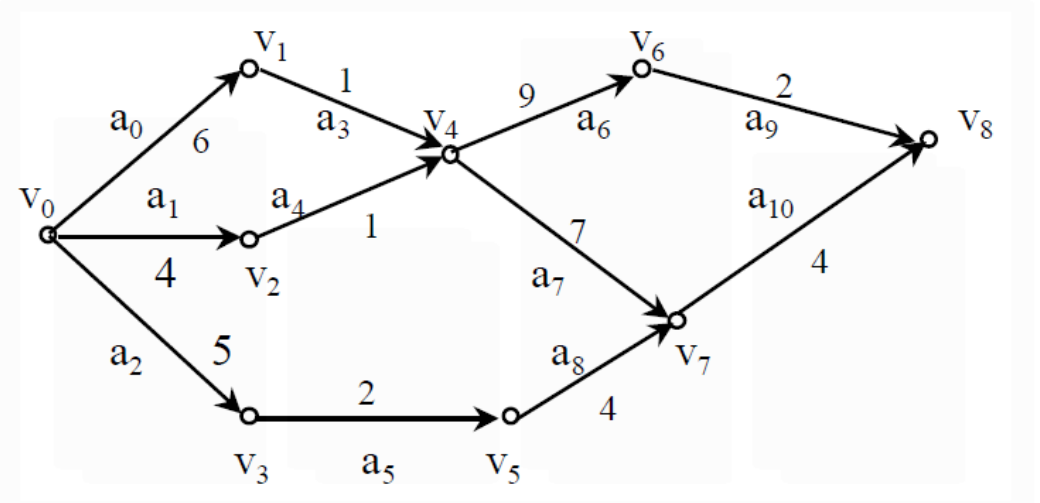
- 在不推延整个工期的前提下，事件v_i允许的**最晚发生时间**
  - 不得迟于其后继事件v_j的最晚发生时间减去活动$< v_i, v_j >$的持续时间（不延误后续的活动前提下的最晚开始时间）

不能延误工期，所以最后一个事件的最晚发生时间=最早发生时间

$$le(n-1) = ee(n-1)$$

$$le(i) = \min\{le(j) - weight(< v_i, v_j >)\}$$

$$< v_i, v_j > \in S, 0 \le i \le n-2$$



其中S为所有以v_i为始点的出边的集合

关键路径的求解示例

le(8)=ee(8)=18

le(7)=ee(8)−weight(<v_7,v_8>)=18−4=14

le(6)=ee(8)−weight(<v_6,v_8>)=18−2=16

le(5)=ee(7)−weight(<v_5,v_7>)=14−4=10

le(4)=min{le(7)—weight(<v_4,v_7>), le(6)—weight(<v_4,v_6>)}

=min{14−7, 16−9 }=7

le(3)=le(5)−weight(<v_3,v_5>)=10−2=8

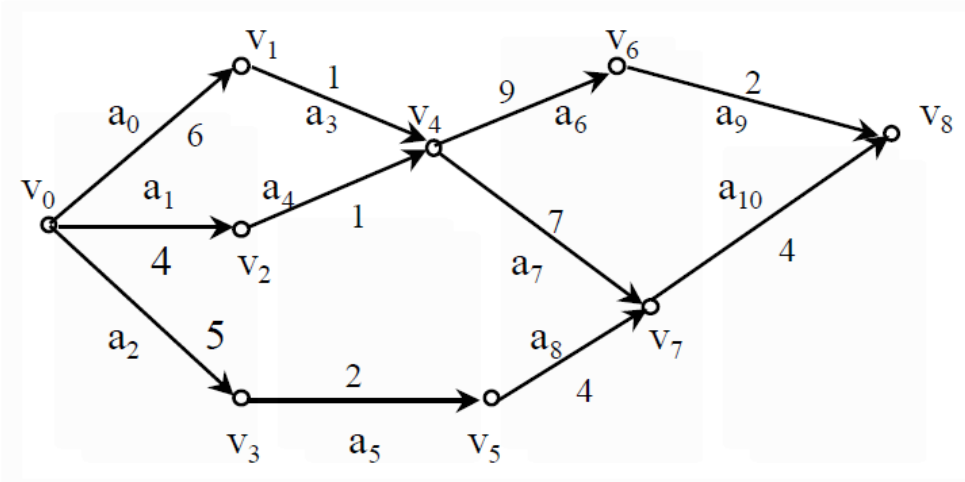
le(2)=le(4)−weight(<v_2,v_4>)=7−1=6

le(1)=le(4)−weight(<v_1,v_4>)=7−1=6

le(0)=min{le(1)—weight(<v_0,v_1>), le(2)—weight(<v_0,v_2>),

le(3)—weight(<v_0,v_3>)}

=min{6−6, 6−4, 8−5 }=0



某活动的最早/晚开始事件

- 活动$a_k = < v_i, v_j >$的最早开始时间e(k)
  - 只有事件v_i发生了，活动a_k才能开始，也即它的最早开始时间为事件v_i的最早发生时间

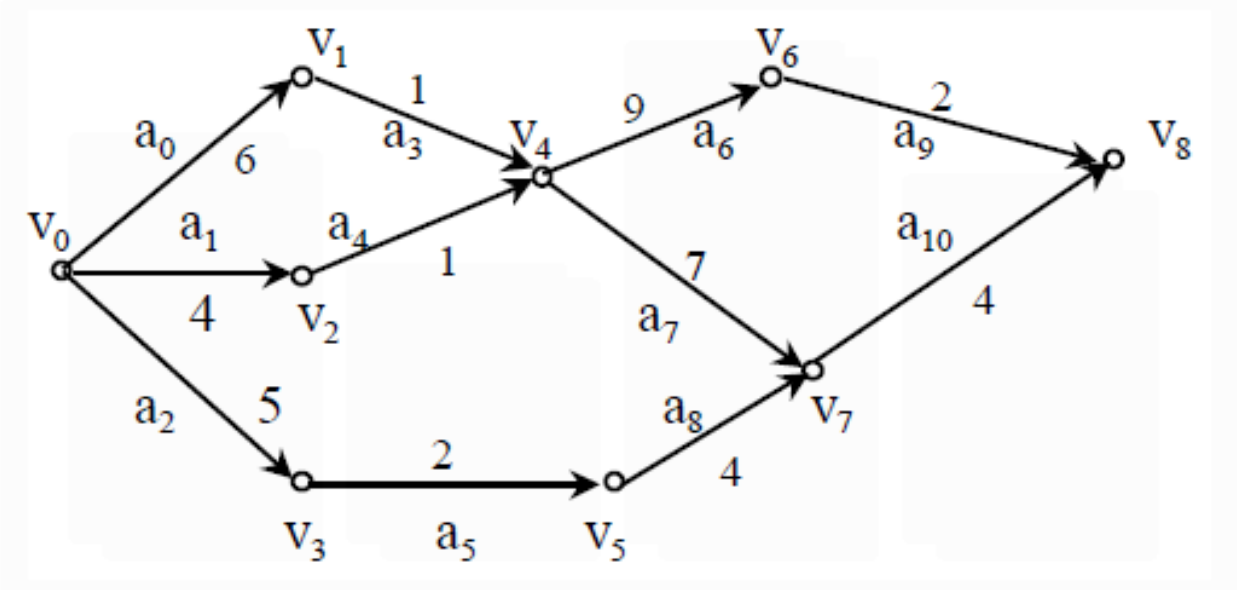
$$e(k) = ee(i)$$

- 活动$a_k = < v_i, v_j >$的最晚开始时间l(k)
  - 活动a_k的最晚开始时间为事件v_j的最晚完成时间减去a_k的持续时间

$$l(k) = le(j) - weight(< v_i, v_j >)$$

关键路径求解示例

e(0)=ee(0)=0
e(1)=ee(0)=0
e(2)=ee(0)=0
e(3)=ee(1)=6
e(4)=ee(2)=4
e(5)=ee(3)=5
e(6)=ee(4)=7
e(7)=ee(4)=7
e(8)=ee(5)=7
e(9)=ee(6)=16
e(10)=ee(7)=14



关键路径求解示例

$l(0)=le(1)-weight(<v_0,v_1>)=6-6=0$

$l(1)=le(2)-weight(<v_0,v_2>)=6-4=2$

$l(2)=le(3)-weight(<v_0,v_3>)=8-5=3$

$l(3)=le(4)-weight(<v_1,v_4>)=7-1=6$

$l(4)=le(4)-weight(<v_2,v_4>)=7-1=6$

$l(5)=le(5)-weight(<v_3,v_5>)=10-2=8$

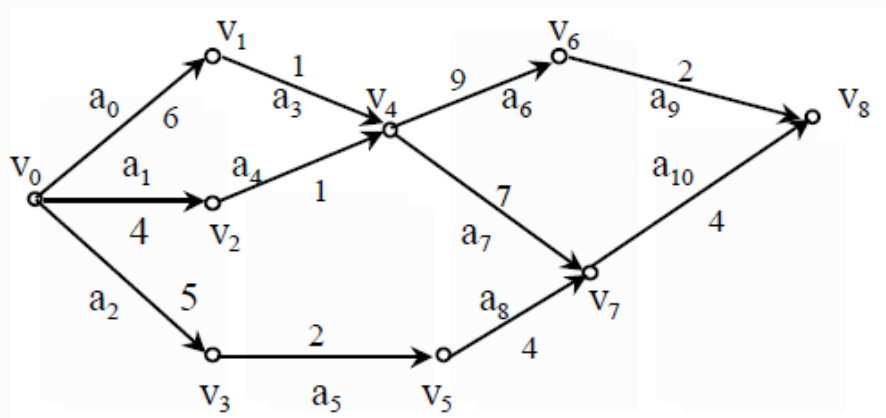
$l(6)=le(6)-weight(<v_4,v_6>)=16-9=7$

$l(7)=le(7)-weight(<v_4,v_7>)=14-7=7$

$l(8)=le(7)-weight(<v_5,v_7>)=14-4=10$

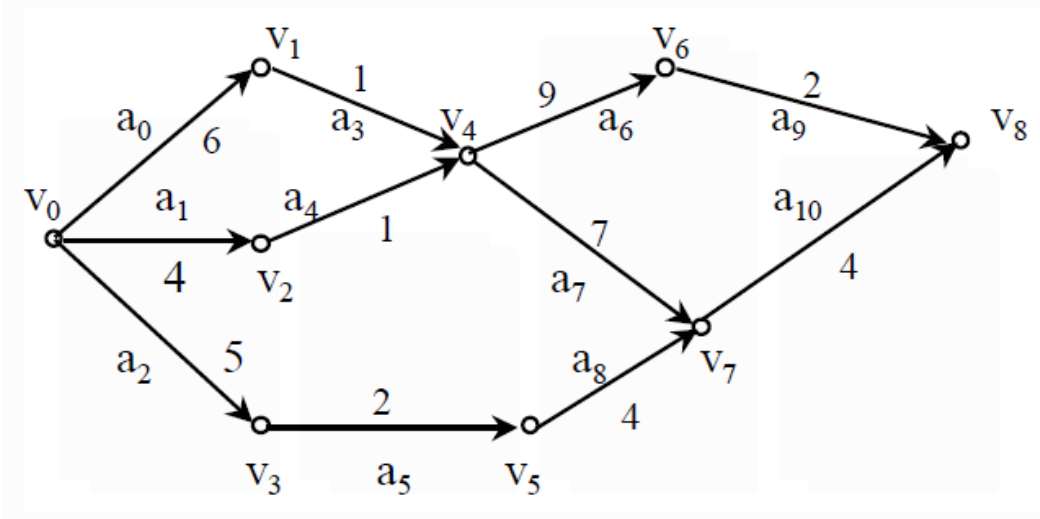
$l(9)=le(8)-weight(<v_6,v_8>)=18-2=16$

$l(10)=le(8)-weight(<v_7,v_8>)=18-4=14$



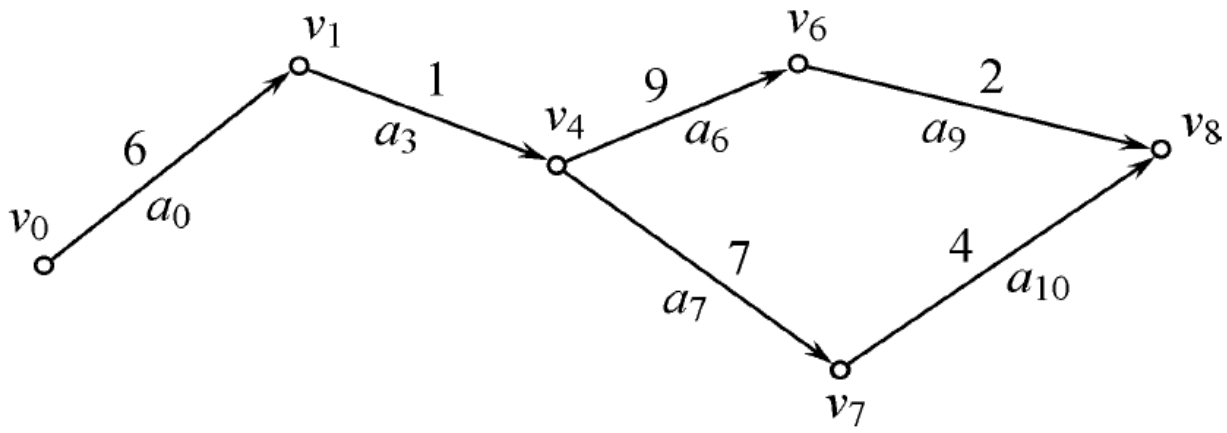
关键路径求解示例

l(0)－e(0)=0－0=0
l(1)－e(1)=2－0=2
l(2)－e(2)=3－0=3
l(3)－e(3)=6－6=0
l(4)－e(4)=6－4=2
l(5)－e(5)=8－5=3
l(6)－e(6)=7－7=0
l(7)－e(7)=7－7=0
l(8)－e(8)=10－7=3
l(9)－e(9)=16－16=0
l(10)－e(10)=14－14=0



关键路径求解示例

- 结果：活动a_0, a_3, a_6, a_7, a_9, a_{10}为关键活动



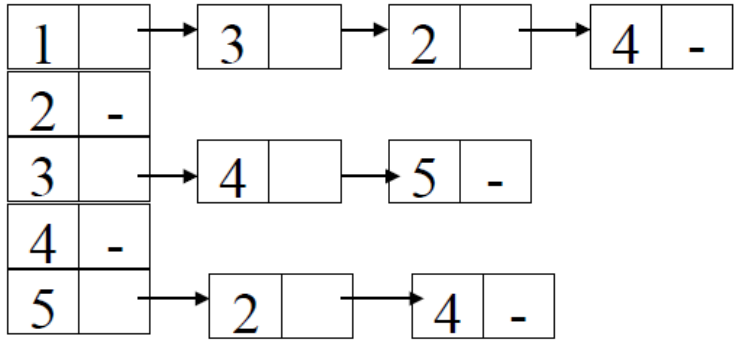
复杂度分析

- 在n个顶点，e条边的AOE网中
- 求事件可能的最早发生时间及允许的最迟发生时间，以及活动的最早开始时间和最晚开始时间时，都要对图中所有顶点及每个顶点边表中所有的边结点进行检查，时间花费为O(n+e)
- 故关键路径算法的时间复杂度为O(n+e)

练习

- 已知一个有向图的邻接表存储结构,如遍历算法从顶点v1 出发图所示，根据有向图的深度优先,所得的顶点序列是( )，按广度优先遍历算法得到序列( )。

A. v1,v2,v3,v5,v4
B. v1,v2,v3,v4,v5
C. v1,v3,v4,v5,v2
D. v1,v4,v3,v5,v2
E. v1,v3,v2, v4,v5



练习

- 设计一个算法判断无向图G是否是一棵树?
  - 若是树，返回true；否则，返回false

- 解题思路
  - 图G是一棵树的条件是G必须是无回路的连通图或者是有n-1条边的连通图。
  - 深度优先遍历，是否可以访问到n个顶点和n-1条边，且访问的顶点不重复

奖金(reward)分配问题

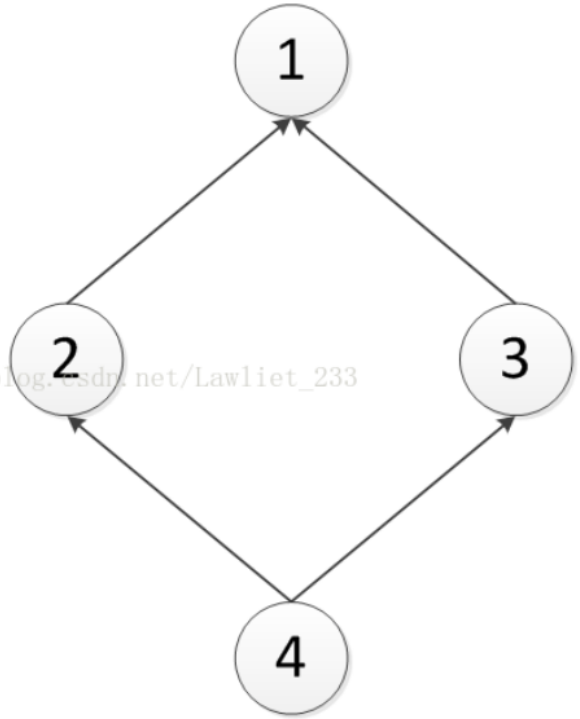
- 由于Yali Company总经理 Mr. Z 决定给n位员工发奖金。公司决定以每个人本年在公司的贡献为标准来计算他们得到奖金的多少。

- 于是Mr. Z下令召开m方会谈。每位参加会谈的代表提出了自己的意见：我认为员工a的奖金应该比b高！

- HR决定要找出一种奖金方案，满足各位代表的意见，且同时使得总奖金数最少。每位员工奖金最少为100元。

思路

- 构建具有n个顶点，m条边的有向图
- 之后进行拓扑排序
  - 首先统计入度为0顶点，这些顶点的奖金就是100元
  - 删除这些顶点相邻边后，入度变为0的顶点的奖金就是101元。
  - 依次类推，直到所有顶点入度都为0
  - 在排序过程中，如果发现成环，无法进行拓扑排序，则退出，无法分配奖金。



//blog.csdn.net/Lawliet_233



数据结构与算法